

MOST50 Intelligent Network Interface Controller

Features

- Complete MOST® Network Interface on a single chip
- Embedded MOST Network management
 - Network protection mode
 - Hardware and application watchdog timer
 - Intelligent muting
- Media Local Bus (MediaLB®) Port
 - Supports all MOST data types
 - Eases inter-chip communication and streaming
 - MediaLB 3-pin interface at speeds up to 1024×Fs
 - Legacy MediaLB 5-pin interface at speeds up to 512×Fs
- Control Port supports I²C format for control messages and asynchronous packets
- Streaming Port supports up to four serial interfaces for synchronous data
- Dedicated application reset output
- Persistent memory for storing internal INIC values during low-power operation (MOST Sleep Mode)
- One-Time Programmable (OTP) memory for initial configuration settings (*INIC Configuration String*)
- General Purpose I/O (GPIO) Port
- Operating voltages 3.3/1.8 V
- 48-pin QFN package with exposed pad for extended temperature support

Applications

- MOST50 Electrical Networks (MOST50 ePHY)

General Description

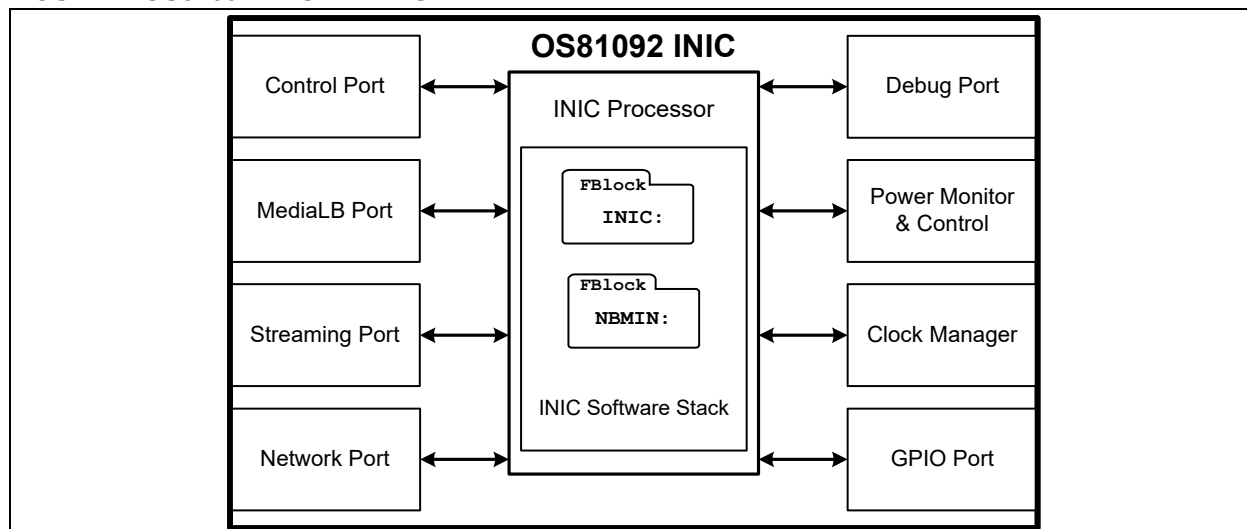
The OS81092 is a highly integrated *Intelligent Network Interface Controller* (INIC) for the 50 Mbps MOST Network (MOST50) with an electrical physical layer (ePHY) optimized for unshielded twisted pair (UTP) copper wire.

INIC provides encapsulation of all low-level functions necessary to develop a MOST-compliant device, significantly simplifying MOST implementation in a node. Supervision of the application is also provided, including a protection mode that is entered when an application is not present (i.e. start-up) or the External Host Controller (EHC) malfunctions. This protection mode prevents application malfunctions from influencing the integrity of the network and the system.

INIC includes the *INIC Software Stack*, which provides MOST-compliant real-time behavior. Integration of the *INIC Software Stack* significantly relieves the EHC from real-time processing tasks.

INIC makes a MOST device accessible according to the rules of the MOST Specification, even without participation from the EHC. When the EHC is engaged, a message-based interface, as opposed to a register-based interface, is available for communication with INIC. Configuration is encapsulated within a Function Block (FBlock INIC) which allows the EHC to manage INIC in the same way as other functions in the system.

FIGURE: OS81092 BLOCK DIAGRAM



ORDERING INFORMATION

Order Number	Package
OS81092AM-rrr-vvvvvv	48-pin QFN
OS81092AMR-rrr-vvvvvv	48-pin QFN, Tape and Reel
rrr - Product Revision Code vvvvv - Firmware Revision Code	

TO OUR VALUED CUSTOMERS

It is our intention to provide our valued customers with the best documentation possible to ensure successful use of your Microchip products. To this end, we will continue to improve our publications to better suit your needs. Our publications will be refined and enhanced as new volumes and updates are introduced.

If you have any questions or comments regarding this publication, please contact the Marketing Communications Department via E-mail at docerrors@microchip.com. We welcome your feedback.

Most Current Data Sheet

To obtain the most up-to-date version of this data sheet, please register at our Worldwide Web site at:

<http://www.microchip.com>

You can determine the version of a data sheet by examining its literature number found on the bottom outside corner of any page. The last character of the literature number is the version number, (e.g., DS30000000A is version A of document DS30000000).

Errata

An errata sheet, describing minor operational differences from the data sheet and recommended workarounds, may exist for current devices. As device/documentation issues become known to us, we will publish an errata sheet. The errata will specify the revision of silicon and revision of document to which it applies.

To determine if an errata sheet exists for a particular device, please check with one of the following:

- Microchip's Worldwide Web site; <http://www.microchip.com>
- Your local Microchip sales office (see last page)

When contacting a sales office, please specify which device, revision of silicon and data sheet (include literature number) you are using.

Customer Notification System

Register on our web site at www.microchip.com to receive the most current information on all of our products.

TABLE OF CONTENTS

1	OS81092 PACKAGE OUTLINE	5
2	OS81092 PINOUT	6
3	EQUIVALENT SCHEMATICS FOR PINS	9
4	OVERVIEW	11
5	INIC PROCESSOR	17
6	PORT MESSAGE PROTOCOL	19
7	CONTROL PORT	22
8	MediaLB PORT	27
9	STREAMING PORT	35
10	EXTERNAL POWER MANAGEMENT	39
11	CLOCK MANAGER	41
12	GENERAL PURPOSE INPUT/OUTPUT (GPIO) PORT	42
13	JTAG PORT	45
14	ELECTRICAL PHYSICAL (ePHY) INTERFACE	47
15	ELECTRICAL CHARACTERISTICS	50
16	APPLICATION INFORMATION	65
APPENDIX A REFERENCES		81
APPENDIX B REVISION HISTORY		82
APPENDIX C FEATURE MATRIX		84
APPENDIX D LIST OF TABLES		85
APPENDIX E LIST OF FIGURES		86

Conventions

The following abbreviations and symbols are used to improve readability.

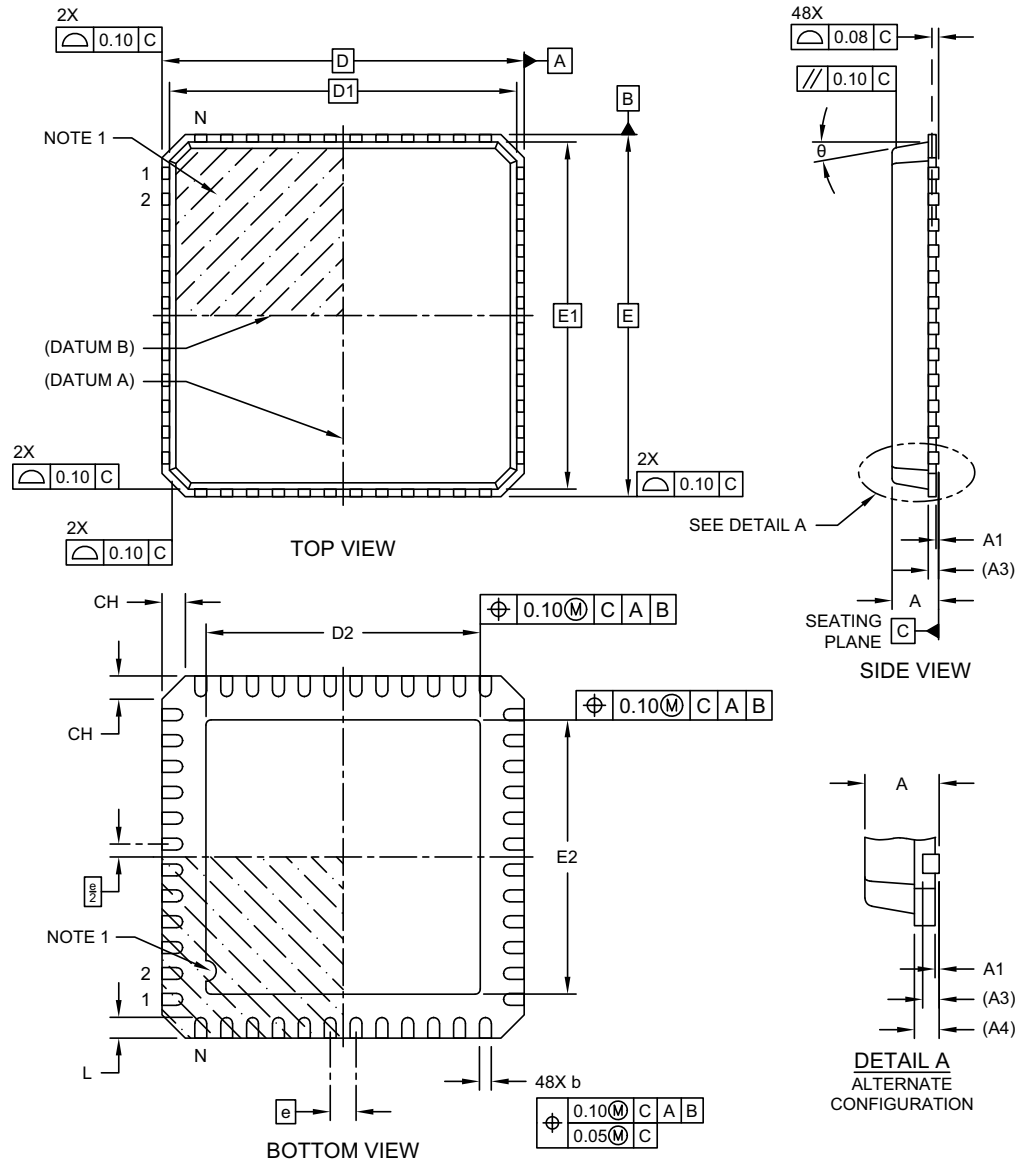
Example	Description
BIT	Name of a single bit within a field
FIELD.BIT	Name of a single bit (BIT) in FIELD
x...y	Range from x to y, inclusive
BITS[m:n]	Groups of bits from m to n, inclusive
PIN	Pin Name
<i>SIGNAL</i>	Signal Name
msb, lsb	Most significant bit, least significant bit
MSB, LSB	Most significant byte, least significant byte
zzzzb	Binary number (value zzzz)
0zzzz	Hexadecimal number (value zzz)
zzh	Hexadecimal number (value zz)
rsvd	Reserved memory location. Must write 0, read value indeterminate
code	Instruction code, or API function or parameter
<i>Multi Word Name</i>	Used for multiple words that are considered a single unit, such as: <i>Resource Allocate</i> message, or <i>Connection Label</i> , or <i>Decrement Stack Pointer</i> instruction.
<i>Section Name</i>	Emphasis, Reference, Section or Document name.
$\overline{\text{VAL}}$	Over-bar indicates active low pin or register bit
x	Don't care
<Parameter>	<> indicate a Parameter is optional or is only used under some conditions
{,Parameter}	Braces indicate Parameter(s) that repeat one or more times.
[Parameter]	Brackets indicate a nested Parameter. This Parameter is not real and actually decodes into one or more real parameters.

1.0 OS81092 PACKAGE OUTLINE

FIGURE 1-1: 48-PIN QFN

**48-Lead Plastic Quad Flat, No Lead Package (RSX) - 7x7x0.9 mm Body [VQFN]
With Exposed Pad; Punch Singulated (AIS Package HZH)**

Note: For the most current package drawings, please see the Microchip Packaging Specification located at <http://www.microchip.com/packaging>



Microchip Technology Drawing C04-223-RSX Rev E Sheet 1 of 2

© 2022 Microchip Technology Inc.

THERMAL CHARACTERISTICS

Parameter	Symbol	Value	Unit
Typical Junction to Package	Ψ_{JT}	0.2	°C/W
Typical Junction to Ambient	θ_{JA}	25	°C/W

OS81092

2.0 OS81092 PINOUT

Input pins must not be left floating; therefore, they must be driven, have pull-ups or pull-downs, or be tied directly to a power or ground pin.

Digital pins that can be configured as outputs (e.g. $D_{I/O}$, D_{OUT} , $D_{I/OD}$) are high impedance during power-up/reset. After power-up/reset:

- pins with more than one configuration option are as follows:
 - Pin 27 - **DSCL** functionality ($D_{I/OD}$) - assumes **TMS** high
 - Pin 31 - **SDSA** functionality ($D_{I/OD}$) - assumes **TMS** high
 - Pin 34 - **MLBSIG** functionality ($D_{I/O}$) - default can be changed via the *INIC Configuration String*
 - Pin 35 - **MLBDAT** functionality ($D_{I/O}$) - default can be changed via the *INIC Configuration String*
 - Pin 37 - High impedance until configured
 - Pin 38 - High impedance until configured
 - Pin 45 - **ERR** functionality (D_{OUT}) - unless held low at the rising edge of reset
 - Pin 46 - **DINT** functionality (D_{OUTD})
- output pins with a disable option remain in the high impedance state, unless specifically enabled by software (e.g. **RMCK**, **MLBCLK**), and
- all other pin types are as shown in [Table 2-1](#).

FIGURE 2-1: 48-PIN QFN DIAGRAM FOR OS81092

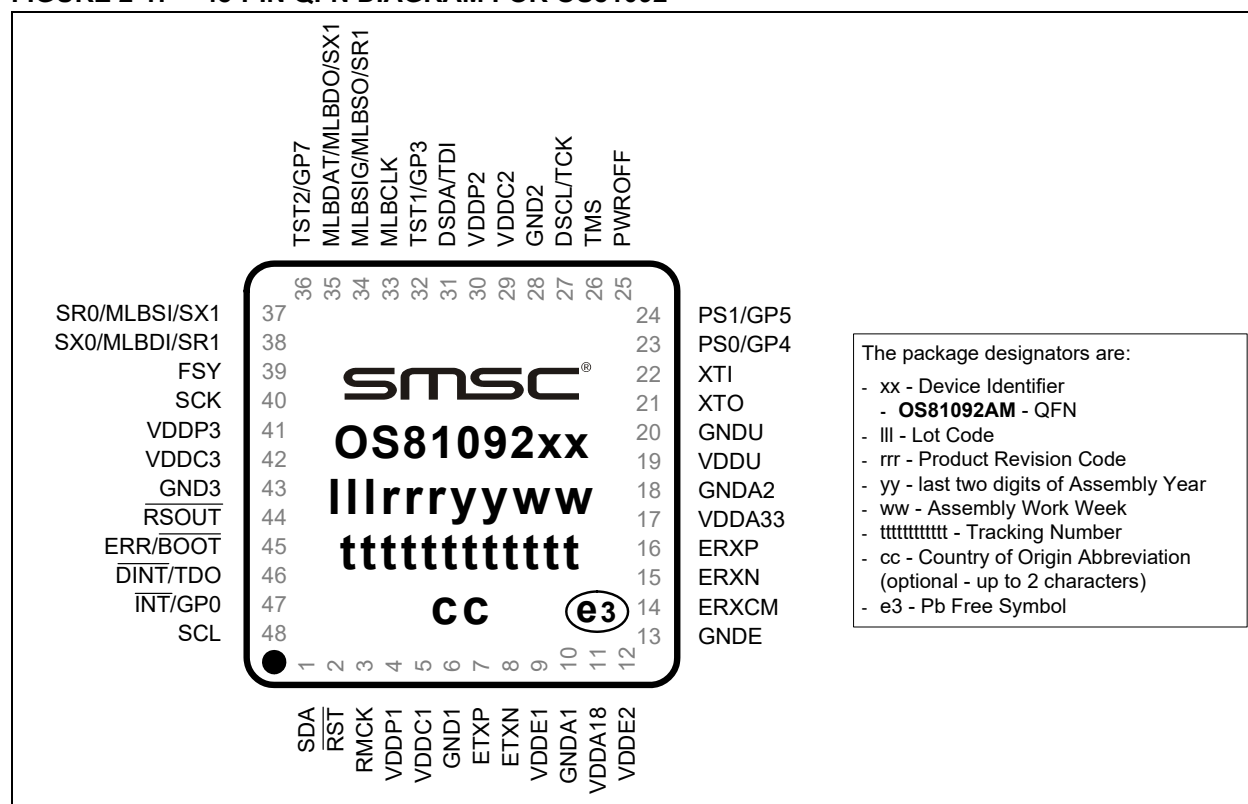


TABLE 2-1: OS81092 PIN ALLOCATION TABLE

Pin	Name	Type	Pin Description
1	SDA ¹	D _{I/OD}	Control Port I ² C data
2	RST	D _{IN}	Hardware reset input. (Pull-up resistor to VDDP should be used when not driven high by an external device. A series resistor should be used in lieu of the pull-up when always driven by an external device.)
3	RMCK	D _{OUT}	Recovered master clock output for synchronizing external devices
4	VDDP1		3.3 V periphery power supply (digital)
5	VDDC1		1.8 V core power supply (digital)
6	GND1		Ground (digital)
7	ETXP	A _{IO}	Positive (differential) ePHY network transmitter output
8	ETXN	A _{IO}	Negative (differential) ePHY network transmitter output
9	VDDE1		3.3 V ePHY power supply (analog)
10	GND A1		Ground (analog)
11	VDDA18		1.8 V power supply (analog)
12	VDDE2		3.3 V ePHY power supply (analog)
13	GNDE		Ground (analog)
14	ERXCM	A _{IO}	ePHY common-mode voltage output
15	ERXN	A _{IO}	Negative (differential) ePHY network receiver input
16	ERXP	A _{IO}	Positive (differential) ePHY network receiver input
17	VDDA33		3.3 V power supply (analog)
18	GND A2		Ground (analog)
19	VDDU		3.3 V continuous power supply (digital)
20	GNDU		Ground (digital)
21	XTO	A _{IO}	Crystal oscillator output
22	XTI	A _{IO}	Crystal oscillator input - or external CMOS clock input
23	PS0	D _{IN}	External Power Management status bit 0. (Tie to ground when external power management is not used.)
	GP4	D _{I/O}	GPIO 4
24	PS1	D _{IN}	External Power Management status bit 1. (Tie to ground when external power management is not used.)
	GP5	D _{I/O}	GPIO 5
25	PWROFF ¹	D _{OUTD}	External Power Management power-off indicator. This pin is driven low by INIC after initialization. When high, indicates that the INIC Processor is ready to be shut down. (Pull-up resistor required when used. If not used, this pin may be left unconnected.)
26	TMS ¹	D _{IN}	JTAG Test Mode Select
27	DSCL ¹	D _{I/OD}	Customer Configuration Interface clock
	TCK ¹	D _{IN}	JTAG Test Clock
28	GND2		Ground (digital)
29	VDDC2		1.8 V core power supply (digital)
30	VDDP2		3.3 V periphery power supply (digital)
31	DSDA ¹	D _{I/OD}	Customer Configuration Interface data
	TDI ¹	D _{IN}	JTAG Test Data Input

Note 1: Pull-up resistor required.

2: Pull-down resistor required.

3: A pull-up or pull-down resistor recommended, based on the default mode.

OS81092

TABLE 2-1: OS81092 PIN ALLOCATION TABLE (CONTINUED)

Pin	Name	Type	Pin Description
32	TST1		Internal test pin (connect directly to ground)
	GP3	D _{I/O}	GPIO 3
33	MLBCLK ²	D _{OUT}	MediaLB Port clock output
34	MLBSIG ²	D _{I/O}	MediaLB 3-pin Port signal information line
	MLBSO ²	D _{OUT}	MediaLB 5-pin Port signal information output line
	SR1	D _{IN}	Streaming Port serial data input 1
35	MLBDAT ²	D _{I/O}	MediaLB 3-pin Port data line
	MLBDO ²	D _{OUT}	MediaLB 5-pin Port data output line
	SX1	D _{OUT}	Streaming Port serial data output 1
36	TST2		Internal test pin (connect directly to ground)
	GP7	D _{OUT}	GPIO 7 (output only)
37	SR0	D _{IN}	Streaming Port serial data input 0
	MLBSI	D _{IN}	MediaLB 5-pin Port signal information input line
	SX1	D _{OUT}	Streaming Port serial data output 1
38	SX0	D _{OUT}	Streaming Port serial data output 0
	MLBDI	D _{IN}	MediaLB 5-pin Port data input line
	SR1	D _{IN}	Streaming Port serial data input 1
39	FSY	D _{I/O}	Streaming Port frame sync
40	SCK	D _{I/O}	Streaming Port bit clock
41	VDDP3		3.3 V peripheral power supply (digital)
42	VDDC3		1.8 V core power supply (digital)
43	GND3		Ground (digital)
44	$\overline{\text{RSOUT}}$ ¹	D _{OUTD}	Optional reset for External Host Controller. (Pull-up resistor required when used. If not used, this pin may be left unconnected.)
45	ERR	D _{OUT}	Error indicator output. (See the OS81092 INIC API User's Manual [2] for information on configuring the behavior of this pin.)
	$\overline{\text{BOOT}}$ ¹	D _{IN}	Configuration pin. This pin is connected to the configuration/debug header and is used by Microchip's INIC Explorer Interface Box [4] to load configuration data into INIC. This pin may also be connected to the EHC to allow for configuration via the Control Port.
46	$\overline{\text{DINT}}$ ¹	D _{OUTD}	Customer Configuration Interface interrupt (active low)
	TDO ¹	D _{OUTZ}	JTAG Test Data Output
47	$\overline{\text{INT}}$ ³	D _{OUTD}	Control Port interrupt (active low)
		D _{IN}	Configuration pin - When <code>PortConfiguration.DefPort()</code> is set to <code>INT_Select</code> : • a pull-up resistor enables the Control Port at power-up/reset, and • a pull-down resistor enables the MediaLB by default at power-up/reset.
	GP0	D _{I/O}	GPIO 0
48	SCL ¹	D _{I/OD}	Control Port I ² C clock
ePAD			The exposed paddle of the QFN package should be soldered to the ground plane for efficient heat dissipation.

Note 1: Pull-up resistor required.

2: Pull-down resistor required.

3: A pull-up or pull-down resistor recommended, based on the default mode.

3.0 EQUIVALENT SCHEMATICS FOR PINS

FIGURE 3-1: PIN-EQUIVALENT FOR ANALOG I/O PIN - A_{IO}

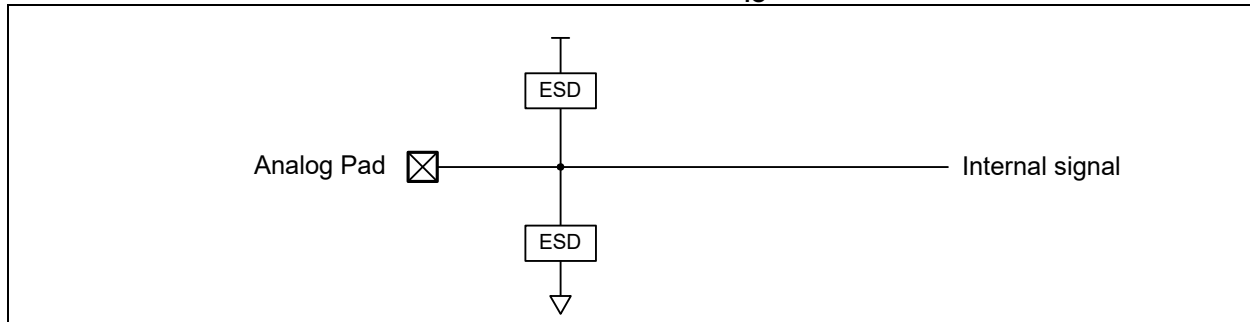


FIGURE 3-2: PIN-EQUIVALENT FOR DIGITAL INPUT PIN - D_{IN}

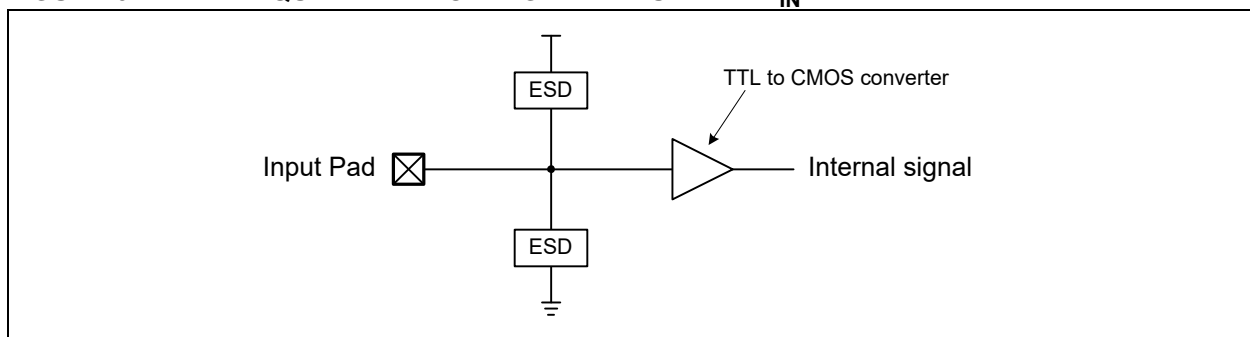


FIGURE 3-3: PIN-EQUIVALENT FOR DIGITAL OUTPUT PIN - D_{OUT}

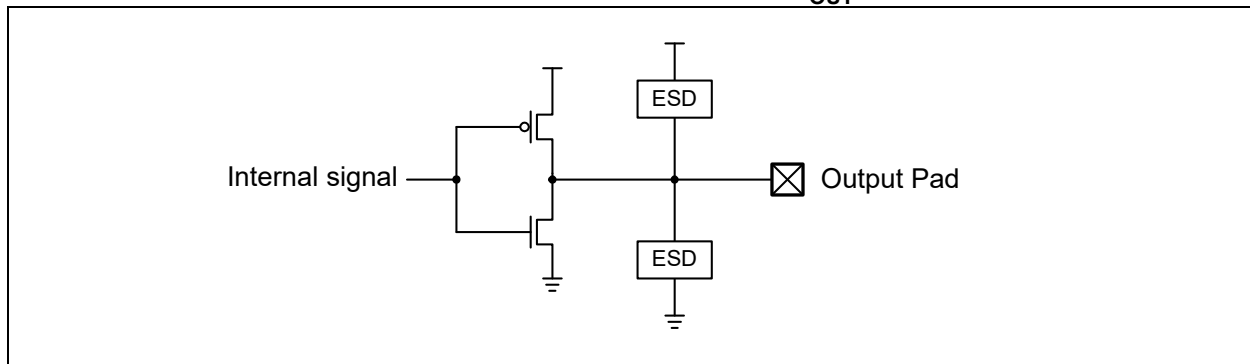


FIGURE 3-4: PIN-EQUIVALENT FOR OPEN-DRAIN DIGITAL OUTPUT PIN - D_{OUTD}

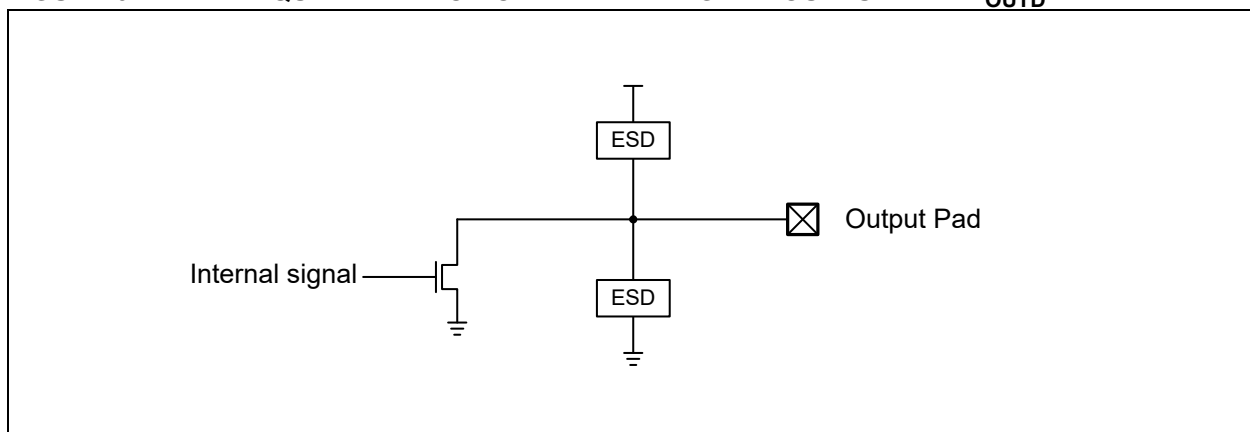


FIGURE 3-5: PIN-EQUIVALENT FOR DIGITAL OUTPUT PIN WITH HIGH-Z CONTROL - D_{OUTZ}

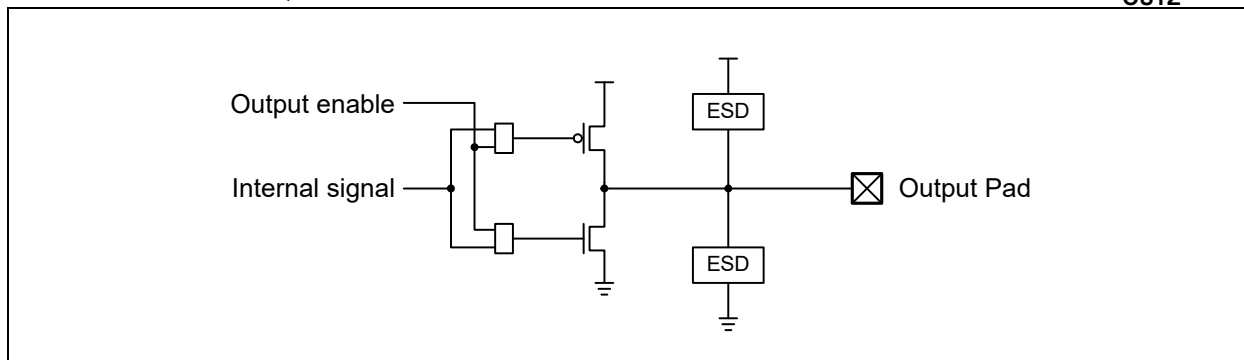


FIGURE 3-6: PIN-EQUIVALENT FOR DIGITAL I/O PIN - $D_{I/O}$

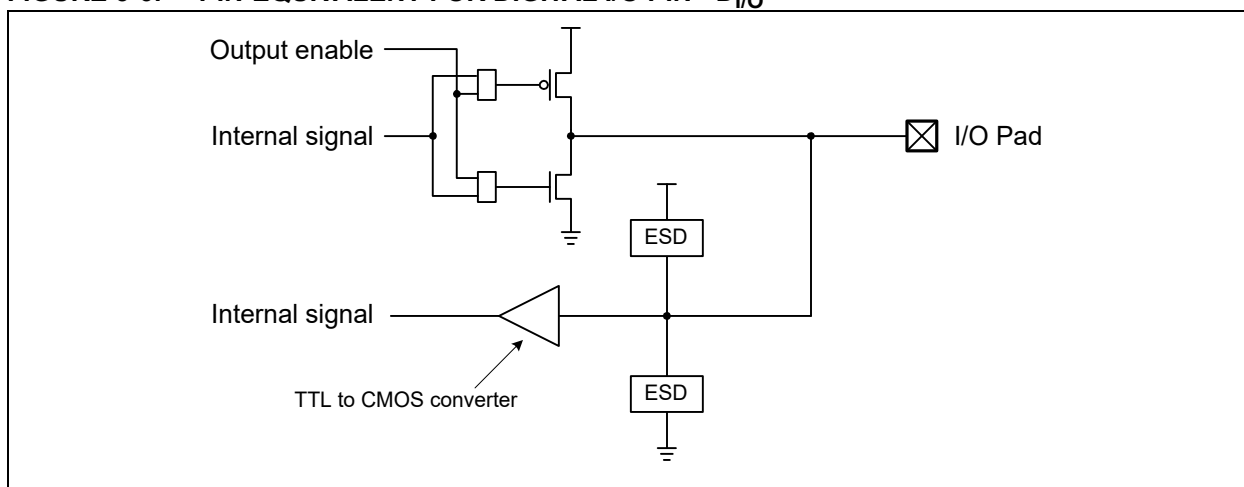
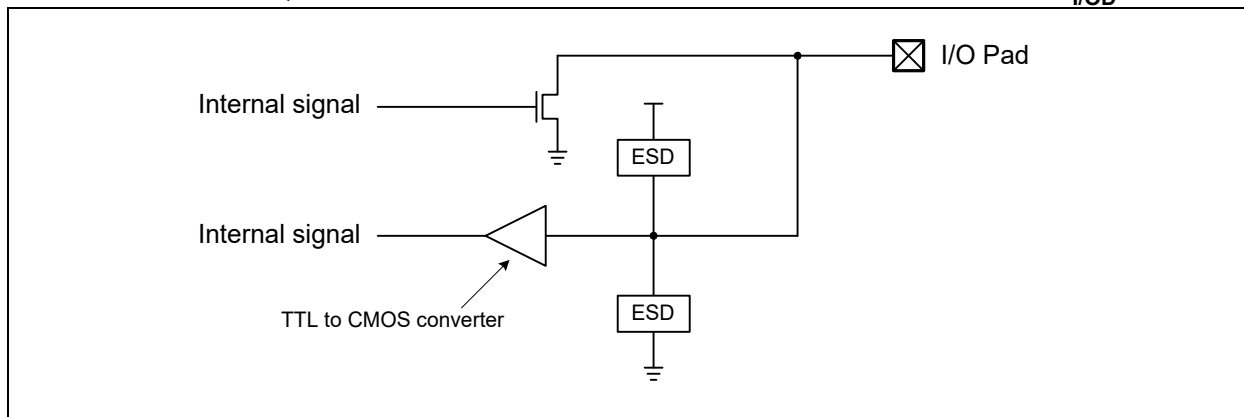


FIGURE 3-7: PIN-EQUIVALENT FOR DIGITAL INPUT/OPEN-DRAIN OUTPUT PIN - $D_{I/OD}$



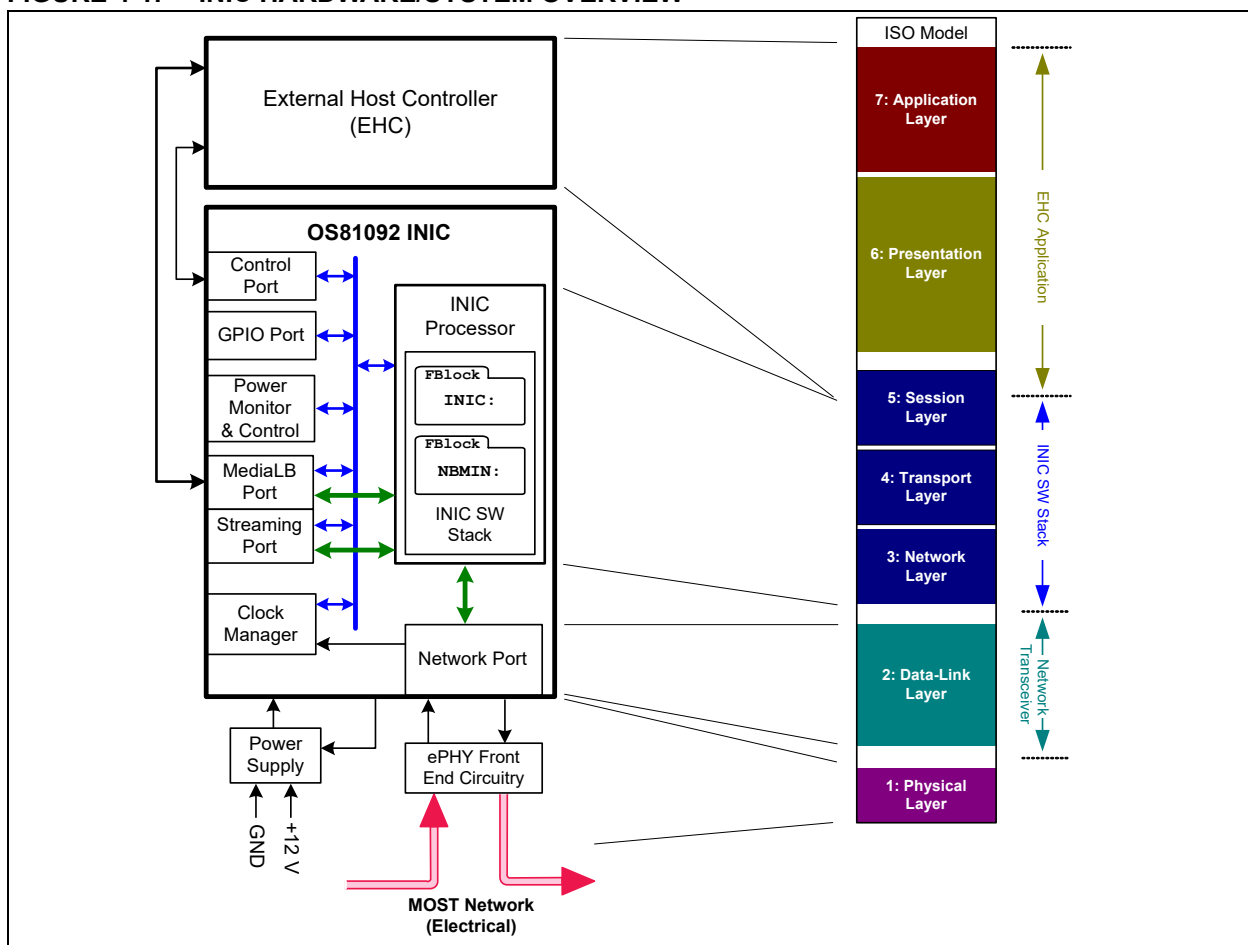
4.0 OVERVIEW

The OS81092 is a network transceiver device belonging to the Microchip *Intelligent Network Interface Controller* (INIC) family. The OS81092 INIC (also known as INIC50) supports an Electrical Physical (ePHY) interface, optimized for unshielded twisted pair (UTP) copper wire. A network data rate of 128 bytes/frame is supported (49.2 Mbps - also known as MOST50) when interfacing to a *Media Oriented Systems Transport* (MOST) Network. Any reference to MOST in this data sheet implies MOST compliance at the MOST50 data rate.

All relevant network management functions are handled on-chip, providing a complete system interface to the physical layer components. Minimal additional components are required due to the high-level of integration. An on-chip PLL with ultra-low jitter guarantees accurate audio and video transmission and clock recovery over a wide frequency range.

A typical MOST Network node consists of the front-end circuitry connected to the MOST Network, the OS81092 to handle the low-level protocols, and an *External Host Controller* (EHC) for the mid- to high-level functions. This architecture eliminates the need for the user to implement the lower levels of the [MOST Specification \[1\]](#) protocol, thereby drastically shrinking development time. Network management functions are off-loaded from the programmer, allowing full concentration on the application being developed. [Figure 4-1](#) illustrates the OS81092, with the protocol stack implementation.

FIGURE 4-1: INIC HARDWARE/SYSTEM OVERVIEW



4.1 MOST50 Network

To minimize costs, the MOST Network supports a peer-to-peer methodology, eliminating excessive hardware overhead such as a hub (although hub-based architectures can also be implemented). In addition to handling network interface and communication management functions, the OS81092 INIC also handles all of the important low-level network management functions such as node position sensing (Plug-and-Play), network delay detection, start up and shut down. Other features include: error reporting, fail-safe operation, and channel allocation. Placing these features in hardware off-loads the EHC, allowing it to focus on higher-level network functions.

The MOST Network contains multiple simultaneous networks operating across UTP. Once bandwidth is allocated, each network (or *data transfer method*) operates independently (not affecting the others), providing a robust, dependable, and deterministic system architecture. The OS81092 INIC supports the following data transfer methods, including:

- Control Network - consists of a single *control channel*
- Asynchronous (Packet) Network - consists of a single *packet channel*
- Synchronous Streaming Network - consists of one or more *synchronous streaming channels*

4.1.1 CONTROL NETWORK

The *MOST Control Network* consists of a single control channel, shared by all network nodes. Messages transported across this channel adhere to the *MOST Control Message* (MCM) format and are used to manage the network and exchange control data among nodes. Any network node that needs to transmit a control message must arbitrate for the channel.

INIC monitors the control channel on the Network Port by default and uses information in the message header to determine whether the node is the intended receiver of the control message. When a control message is to be received (e.g. node address match), it is buffered within INIC. For MCMs, the application is notified of the message reception and the packet is retrieved by the EHC via one of the OS81092 hardware ports that supports control message exchange (see [Table 4-1](#)).

4.1.2 ASYNCHRONOUS (PACKET) NETWORK

The *MOST Asynchronous Network* consists of a single packet channel, shared by all network nodes. Applications use this channel to transmit large data packets (e.g. navigation maps) in bursts to other network nodes. Data packets on the packet channel are referred to as *asynchronous packets* and adhere to the *MOST Data Packet* (MDP) format. Any network node that needs to transmit a data packet can arbitrate for the packet channel.

INIC monitors the packet channel on the Network Port by default and uses information in the packet header to determine whether the node is the intended receiver. When an asynchronous packet is to be received (e.g. node address match), it is buffered within INIC and made available to the application at one of the OS81092 hardware ports that supports asynchronous packets (see [Table 4-1](#)).

4.1.3 SYNCHRONOUS STREAMING NETWORK

The *MOST Synchronous Streaming Network* can consist of multiple synchronous streaming channels, each transporting raw, real-time synchronous data (e.g. audio, video). The data on a synchronous streaming channel is sourced by a single transmitting node that has been granted channel bandwidth. Synchronous streaming channels are broadcast, the data available for reception by one or more sink nodes.

For high-speed synchronous data, INIC acts like a cross-point switch to connect synchronous streaming channels to sources/sinks attached to OS81092 hardware ports that support synchronous data (see [Table 4-1](#)).

4.2 INIC Processor

The INIC Processor manages the transfer of data to and from the network. The Network Port connects the INIC Processor to the network using the OS81092 network transmission and reception pins (**ETXP/ETXN** and **ERXP/ERXN**) to exchange data over the ePHY interface. The ePHY interface of the OS81092 requires reception and transmission circuitry to connect the INIC Processor to the electrical network and increase system immunity to EMI. This external circuitry is referred to as the *ePHY Front End Circuitry*.

Working in conjunction with the Clock Manager, the Network Port recovers the network clock, decodes received data, and passes data to the INIC Processor. The INIC Processor then routes data to the appropriate destinations on and off the chip.

The network frame to be transmitted by the OS81092 is constructed by the INIC Processor by combining incoming network data from the Network Port with outgoing data from other on-chip resources. The Network Port then encodes the data for network transmission.

The *INIC Software Stack* runs on the INIC Processor and configures the Network Port at power-up, which allows the network to become fully operational without any external protocol stack. This has been accomplished by placing all the lower layers of the ISO protocol stack inside of the device. In addition to the *INIC Software Stack*, the INIC Processor supports two groups of functions: FBlock NBMIN and FBlock INIC. [Figure 4-1](#) illustrates the position of the *INIC Software Stack* within the ISO software model.

Each network node contains a general set of functions, referred to as the FBlock *NetBlock* functions. INIC manages network-related *NetBlock* functions and the EHC manages the higher-level *NetBlock* functions. To discern between these two sections of the FBlock *NetBlock*, the INIC portion is referred to as *NBMIN* and the EHC portion is labeled *NBEHC*.

FBlock *INIC* contains functions used to configure the chip. This includes the setup of hardware ports and the utilization of network bandwidth for transporting streaming data.

The upper levels of the network protocol must be implemented by an EHC that is capable of processing the user application with the minimal overhead imposed by the INIC API. During real-time operation, the EHC and the *INIC Software Stack* within the INIC Processor communicate using control messages via the Control Port or MediaLB Port.

4.3 Hardware Ports

Each OS81092 hardware port transports different network data types. [Table 4-1](#) shows the data types that can be transported across each hardware interface.

TABLE 4-1: HARDWARE PORT DATA TRANSPORT

Hardware Port	Control Messages*	Asynchronous Packets	Synchronous Data
Network	X	X	X
MediaLB *	X	X	X
Streaming			X
Control *	X	X	

* In addition to MCMs, Control and MediaLB Ports can exchange *INIC Control Messages* (ICMs). ICMs specifically target INIC and are used for internal configuration and control only (never routed to another hardware port or target a network node).

Using the OS81092 API, a *socket* is created for each data stream transported across a hardware port. The specific data type to be exchanged is specified when calling the `CreateSocket()` function. Data routing between hardware ports is setup using the `ConnectSockets()` function, where sockets of like data types are linked together. For more information on sockets, refer to the [OS81092 INIC API User's Manual \[2\]](#).

4.3.1 NETWORK PORT

The Network Port receiver (*ERXP/ERXN*) is over-sampled at a high-frequency and data is recovered by a digital state machine. When the port is configured as a timing-slave, the OS81092 recovers the network clock, which the Clock Manager then uses to generate other internal clocks. When the port is configured as the timing-master, the internal clocks and the network clock are generated by the Clock Manager based on an external crystal (*XTI/XTO*). The Network Port transmitter (*ETXP/ETXN*) works in conjunction with the INIC Processor to encode data for network transmission.

After reset, the OS81092 automatically interacts with the network, performing all necessary low-level network management functions. This enables the EHC to configure its interface to the OS81092 when the application is ready.

4.3.2 MediaLB PORT

The MediaLB Port supports communication between the EHC and the OS81092 INIC by means of the *Media Local Bus* (MediaLB) protocol. MediaLB provides a low cost, easy to implement hardware interface that standardizes and simplifies network application development. MediaLB has one bus master, referred to as the *MediaLB Controller* (or simply *Controller*), which is always the OS81092. All other connected components (including the EHC) are known as *MediaLB Devices* (or simply *Devices*). MediaLB Device functionality is a subset of Controller functionality; the Controller (OS81092) functions as a Device when it is receiving data from other connected components.

The OS81092 supports the MediaLB protocol in either MediaLB 3-pin mode or MediaLB 5-pin mode. Both modes support the exchange of all network data types, including: control messages, asynchronous packets, and synchronous data. The transmitting Device sends data on a dedicated data line, while information about the data is simultaneously transported on an independent signal information line.

When using the MediaLB Port in 3-pin mode, the OS81092 output clock is configurable as 256×Fs (12.288 MHz at 48 kHz), 512×Fs (24.576 MHz at 48 kHz), or 1024×Fs (49.152 MHz at 48 kHz). When configured for 256×Fs, the OS81092 can transmit and receive up to 28 bytes (7 quadlets) per frame of data. When configured for 1024×Fs, the OS81092 can transmit and receive up to 124 bytes (31 quadlets) per frame of data. When using the MediaLB Port in 5-pin mode, the maximum bus clock frequency is 512×Fs (1024×Fs not supported).

MediaLB is a token-passing bus, where the Controller manages the token and generates the data clock. The Controller passes the token by sending out a *ChannelAddress* on the signal information line. The *ChannelAddresses* are pre-assigned by the board integrator and are associated with a specific transmitting Device and a receiving Device. Once the transmitting Device associated with the *ChannelAddress* is granted bus access, it can send one quadlet of data on the MediaLB data line after a defined time delay. The Device provides information about the data being transmitted by simultaneously sending out a *Command* on the signal information line. After receiving the *ChannelAddress*, the receiving Device must decide whether to receive the associated message or reject it. A status byte from the receiving Device is placed on the signal information line, one byte after the *Command* from the sending Device is received. Once per network frame, the Controller generates a unique pattern (*FRAMESYNC*) on the signal information line. This defines the MediaLB frame edge, as well as the byte boundary of signal and data lines for all Devices.

Since MediaLB is a high-speed bus, connections to external devices should be implemented in hardware and not *bit-banged* on generic ports. A MediaLB Device interface, in the form of VHDL code, is available from Microchip. For more information about the Media Local Bus, refer to the [MediaLB Specification \[3\]](#).

EHC devices without an integrated MediaLB interface can use one of the Microchip *I/O Companion* devices (IOCs) as a communication bridge between the OS81092 MediaLB Port and a parallel, SRAM-like EHC interface. For example, the IOC *Host Bus Interface* (HBI) is a 16-bit parallel communication interface for exchange of all network data types. This HBI Port is easily accommodated by EHCs with a generic SRAM port. In addition to serving as a communication bridge between its MediaLB and HBI Ports, the IOC provides hardware port expansion to the application via the MediaLB Port, as described in [Section 4.3.6 “Port Expansion”](#).

4.3.3 STREAMING PORT

The Streaming Port provides a gateway for synchronous data exchange between the OS81092 and external legacy devices. Depending on the configuration of the OS81092, the Streaming Port supports up to four serial data pins, which share a common synchronization signal and bit clock. The data format of the Streaming Port is compatible with industry-standard serial interfaces found on many ADCs, DACs and other devices.

4.3.4 CONTROL PORT

The Control Port operates as an I²C bus slave and supports asynchronous packet and control message exchange. An interrupt pin (*INT*) is used to notify the EHC when Control Port servicing is required (e.g. control message or asynchronous packet received). While the EHC can communicate with the OS81092 over the Control Port alone, the I²C data rate limits throughput on the Application Layer. Application performance can be maximized by using the MediaLB Port to transport all network data types. Configuring and enabling of the MediaLB Port is supported via the I²C Control Port.

The OS81092 Control Port may be configured as an I²C master by changing the settings in the *INIC Configuration String*. In master mode, the OS81092 performs read and write operations with on-board I²C slave devices based on remote commands from the MOST Network or from a local EHC.

4.3.5 GPIO PORT

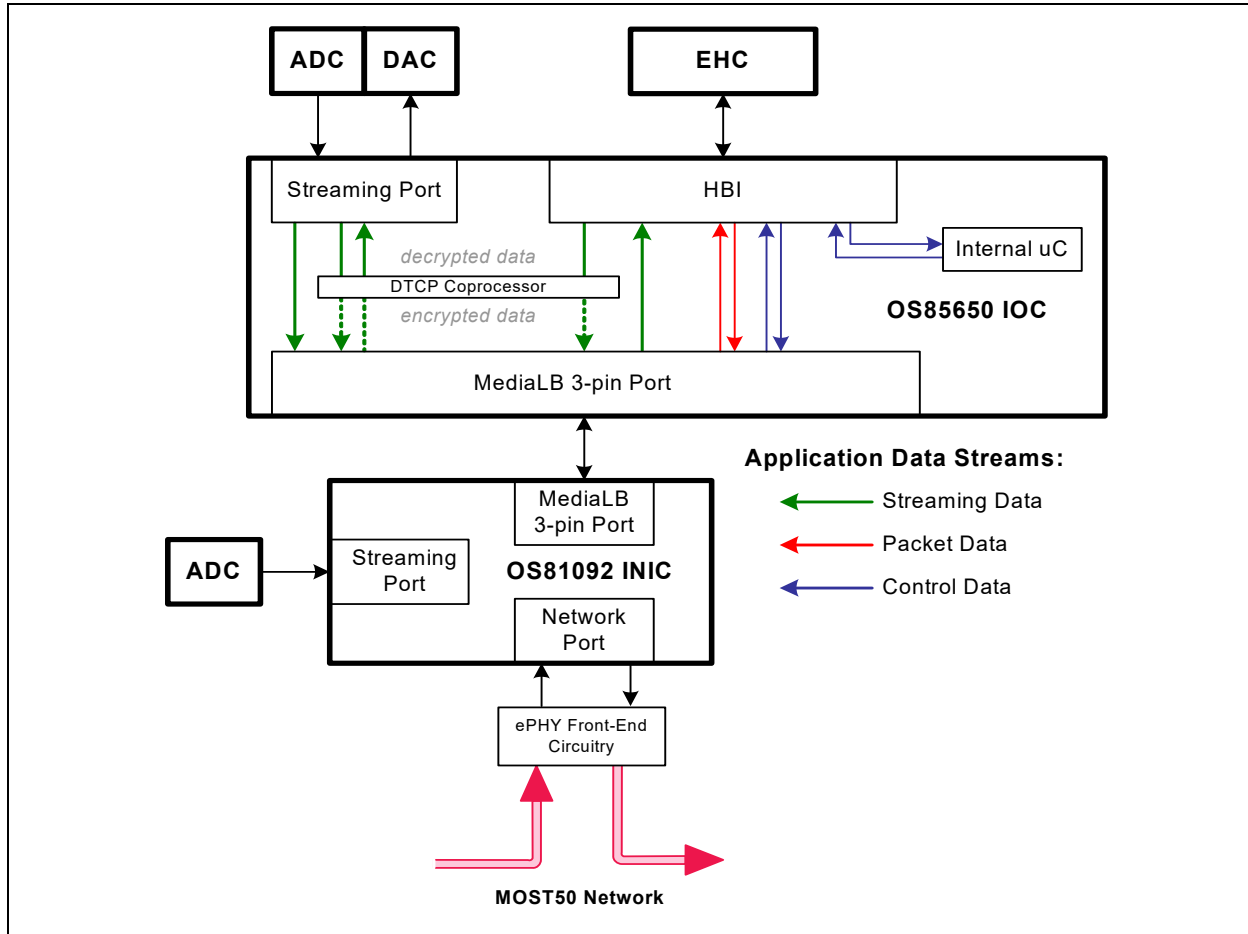
Depending on the *INIC Configuration String* settings loaded in the OS81092, the *General Purpose Input/Output* (GPIO) Port may be available to support various external operations. Some of the OS81092 default functions or ports may not be available when the GPIO Port is used.

4.3.6 PORT EXPANSION

Devices belonging to the Microchip *I/O Companion* (IOC) family can be used as companions to INIC, providing expanded hardware ports to the application. The OS85650 IOC, for example, is a high-performance routing engine that supports IO expansion and a *Digital Transmission Content Protection* (DTCP) coprocessor. The primary function of the OS85650 is routing of data streams between various hardware ports, while the integrated DTCP coprocessor provides optional encryption/decryption of data streams routed through the IOC device. Various services are available to support full *Authentication and Key Exchange* (AKE) implementation. For applications that do not require DTCP, the OS85652 IOC provides the same functionality as the OS85650, albeit without the DTCP coprocessor.

When the OS85650 IOC is used as a companion to the OS81092 INIC, the MediaLB bus is typically the medium used to exchange data between the two devices. Data exchanged over MediaLB is routed to another IOC hardware interface on which external devices may reside (e.g. ADC, DSP, EHC). The EHC for the application may reside on the IOC's *Host Bus Interface* (HBI), which provides an SRAM-like parallel port for easy communication with various microcontrollers. In this way, the IOC allows an EHC to exploit the efficiency and bandwidth of the MediaLB bus, even when the EHC does not support a MediaLB interface.

FIGURE 4-2: INIC-IOC APPLICATION EXAMPLE



Contact Microchip for more information about the IOC family of devices, including the OS85650.

4.4 MOST NetServices

The division of services between the OS81092 INIC and the EHC in MOST applications is illustrated in [Figure 4-3](#). To accelerate development of MOST applications that use INIC, Microchip offers the MOST NetServices API, which provides a seamless software interface between INIC and the EHC. A software library is available to provide all services that are relevant for:

- exchanging application data on the MOST Network,
- managing hardware port data connections (e.g. sockets),
- managing high-level system tasks, and
- using control messages to access INIC for specific operations.

The MOST NetServices code incorporated into the EHC is divided into two distinct components: *Basic Services* (Layer I) and *Applications Socket* (Layer II).

The *Basic Services* component provides the facilities that enable direct communication with INIC, including: a low-level driver for message exchange, message management services, and data wrappers. When transmitting data from the EHC to INIC, the wrapper modules pass data to the message management services. Formatted messages are then sent by the low-level driver across the interface between the EHC and INIC. When receiving data from INIC, the wrappers reformat the data for compatibility with the MOST NetServices code that resides on the EHC.

The *Applications Socket* component resides on top of *Basic Services* and provides the FBlock NBEHC services required to implement the full FBlock NetBlock on the node. This component also contains a command interpreter, which provides a simple API for developing application FBlocks (e.g. FBlock AudioAmp).

MOST NetServices code is modular, allowing it to be customized for a particular application. Implemented in ANSI C, the MOST NetServices API can be adapted for individual requirements through configuration files.

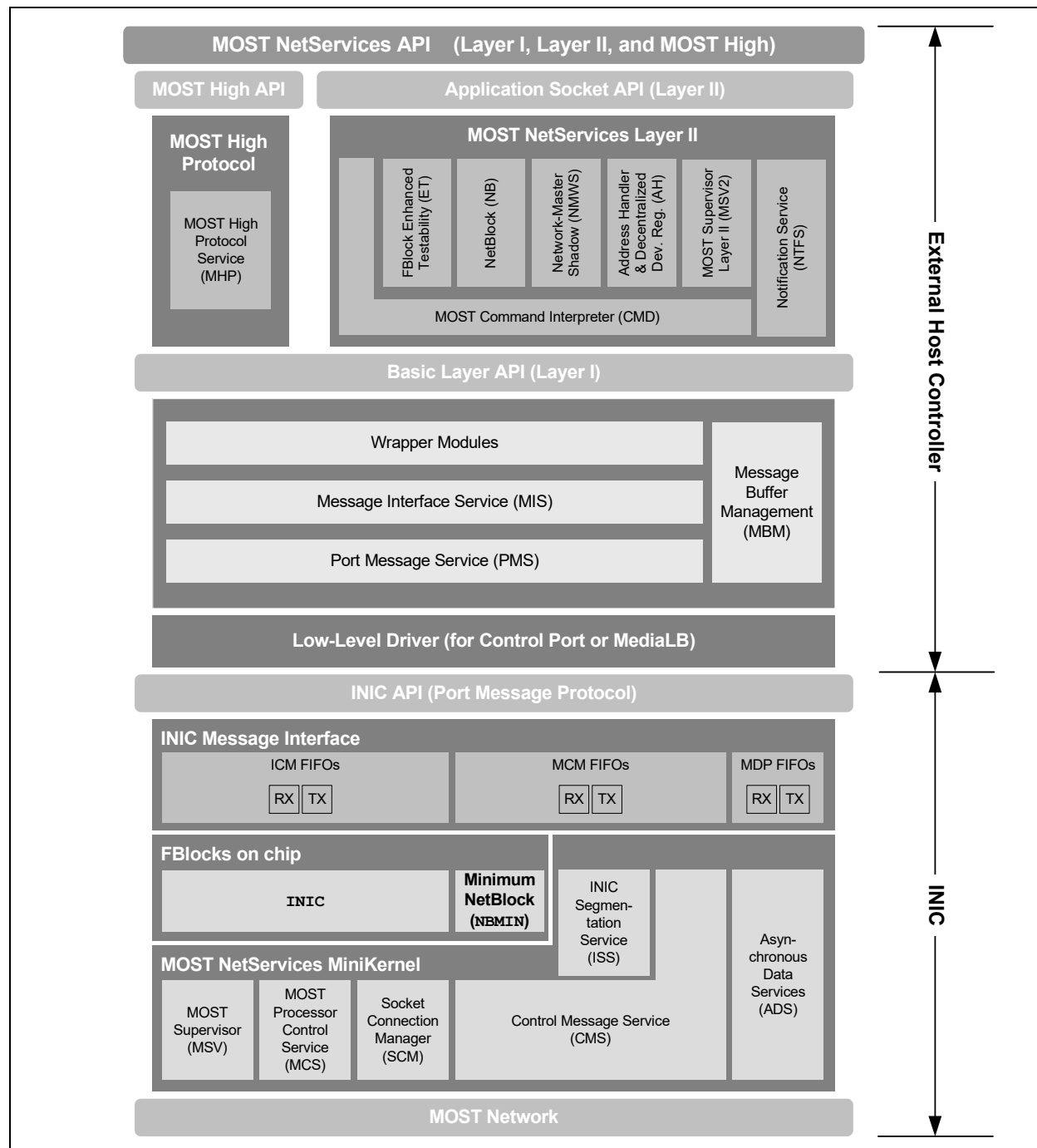
OS81092

With respect to the ISO communications model (shown in [Figure 4-1](#)):

- External ePHY front-end circuitry connects the OS81092 and the MOST Network, supporting the *Physical Layer*.
- OS81092 supports the *Data Link Layer* up to a portion of the *Session Layer*.
- The EHC must provide the remaining portion of the *Session Layer*, the *Presentation Layer*, and the *Application Layer*. When the MOST NetServices code is integrated:
 - the *Basic Services* component provides the remaining portion of the *Session Layer*, and
 - the *Application Socket* component provides the *Presentation Layer* and a portion of the *Application Layer*.

The [OS81092 INIC API User's Manual \[2\]](#) defines the FBlocks (and all associated functions) that are supported by the OS81092.

FIGURE 4-3: DIVISION OF SERVICES BETWEEN INIC AND THE EHC

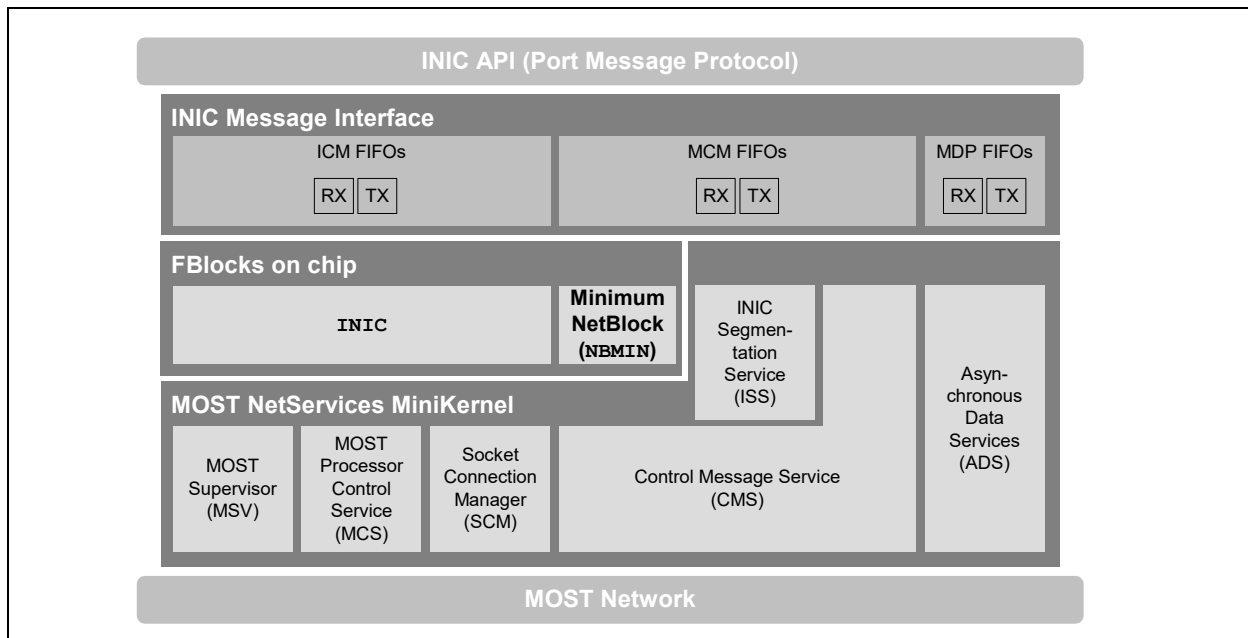


5.0 INIC PROCESSOR

The OS81092 is a valid network node even without an EHC, due to the on-chip INIC Processor. The *INIC Software Stack*, running on the INIC Processor, provides a message-based interface that is easily implemented in a high-level language. This message-based interface not only simplifies the interface between the application and the network, but allows the EHC software to be completely driven by message events. Using the INIC API, as described in the [OS81092 INIC API User's Manual \[2\]](#), the EHC has access to the same functionality that would be available on a register-based interface, but the programmer is spared the effort of making software align with the chip architecture. Much of the *INIC Software Stack* is common across INIC chips, minimizing the effort of upgrading the software for different application platforms. The power-up default settings of the INIC Processor, such as timing-master/timing-slave functionality and other configuration options, can be changed using OTP memory (see [Section 16.6 "Configuration"](#)). The power-up configuration settings stored in OTP memory are collectively referred to as the *INIC Configuration String*.

Partitioning the node architecture between INIC and the EHC results in an efficient implementation, allowing INIC to respond to many network events without intervention by the EHC. Network initialization is optimized, not requiring the network transceiver at each node to wait for the Application Layer to respond. Application software runs efficiently since the number of interrupts to the EHC is minimized. Interaction between the EHC and INIC can be further reduced if the EHC enables notification for INIC properties. [Figure 5-1](#) illustrates the OS81092 *INIC Software Stack*.

FIGURE 5-1: INIC SOFTWARE STACK



5.1 INIC Message Interface

Internal OS81092 FIFOs are used for *message-based* data exchange between INIC and the EHC. Message-based data includes: control messages (e.g. MCMs, ICMs) and asynchronous packets (e.g. MDPs). The *Port Message Protocol* (described in [Section 6.0 "Port Message Protocol"](#)) defines the format for all messages passing through the INIC Message Interface.

Six independent FIFOs support the INIC Message Interface, including:

- *MCM Rx FIFO* - used to receive *MOST Control Messages* (MCMs) from the EHC that are intended for either the local FBlock `NBMIN` or the MOST Network.
- *MCM Tx FIFO* - used to transmit MCMs to the EHC from the local FBlock `NBMIN` or the MOST Network.
- *ICM Rx FIFO* - used to receive *INIC Control Messages* (ICMs) from the EHC intended only for the local FBlock `INIC`.
- *ICM Tx FIFO* - used to transmit ICMs to the EHC from the local FBlock `INIC`.
- *MDP Rx FIFO* - used to receive *MOST Data Packet* messages (MDPs) from the EHC that are intended for transmission onto the MOST Network.
- *MDP Tx FIFO* - used to transmit MDPs to the EHC from the MOST Network.

Each type of message-based data targets a specific INIC FIFO based on direction of transport. These FIFOs are used for message-based data exchange, regardless of the actual hardware port used for communication.

The OS81092 INIC supports four external interfaces (INIC Ports) for exchanging messages. Message FIFOs are used during transmission and reception of data at each port. Most ports can be opened and configured using the `OpenPort()` function within the MOST NetServices MiniKernel's *Socket Connection Manager* (SCM).

- MediaLB Port (`PortID 0`) - The MediaLB Port can be opened by default at power-up/reset based on the *INIC Configuration String*. If not automatically opened by default, this port can be opened and configured through the Control Port, using the `OpenPort()` function described in the API manual.

Note: When the MediaLB Port is automatically opened out of reset, one socket pair is created for Control message transport between INIC and the EHC. A sink socket is assigned <i>ChannelAddress</i> 0004h for EHC to INIC exchange. A source socket is assigned <i>ChannelAddress</i> 0002h for INIC to EHC exchange. Additional socket pairs for asynchronous and synchronous data transfer between the MediaLB Port and the Network Port can be opened by the user with the <code>CreateSocket()</code> INIC function.

- Control Port (`PortID 1`) - The Control Port is not configurable via the API. This port can only be opened at power-up/reset based on the *INIC Configuration String*.
- Network Port (`PortID 2`). The Network Port is configurable via the API. This port is always opened by default at power-up/reset.
- Streaming Port (`PortID 3`). The Streaming Port is never opened by default at power-up/reset. This port can be opened and configured through the Control Port or MediaLB Port, using the `OpenPort()` function described in the API manual.

6.0 PORT MESSAGE PROTOCOL

All message-based data (includes control messages and asynchronous packets) transferred up or down the *INIC Software Stack* adheres to the same low-level message format (defined as the *Port Message*) regardless of the physical hardware port used.

Control messages (e.g. MCMs, ICMs) are used to access the FBlock of a network node. MCMs are typically routed *through* INIC (between the network control channel and the INIC Message Interface), using either the *MCM Rx FIFO* or the *MCM Tx FIFO*, depending on the direction of transport. ICMs target the FBlock *INIC* and are exchanged exclusively between INIC and the EHC, using either the *ICM Rx FIFO* or the *ICM Tx FIFO*, depending on the direction of transport.

Asynchronous packets (e.g. MDPs) are routed between the network packet channel and the INIC Message Interface, using either the *MDP Rx FIFO* or the *MDP Tx FIFO*, depending on the direction of transport.

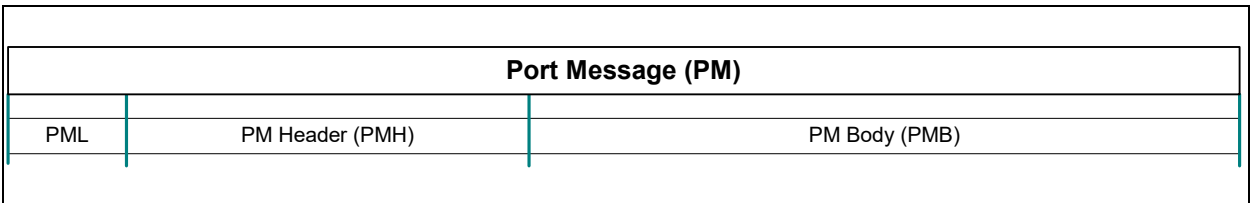
6.1 Generic Port Messages

A generic *Port Message* consists of three main fields:

- *Port Message Length* (PML) - This 16-bit field indicates the total number of bytes that follow in the Port Message, including the message overhead and payload.
- *Port Message Header* (PMH) - This field contains the Port Message overhead and defines the format of the payload data that follows.
- *Port Message Body* (PMB) - This field contains the message payload whose format is dependent on the preceding header.

Figure 6-1 illustrates the generic format of the *Port Message*.

FIGURE 6-1: GENERIC PORT MESSAGE FORMAT



Five types of specific *Port Messages* are defined, each associated with a specific operation to be performed on the targeted FIFO:

- *FIFO Command* - Writes commands to the INIC Message Interface
- *FIFO Status Read* - Reads FIFO status information from the INIC Message Interface
- *FIFO Status Write* - Sends an acknowledge to the INIC Message Interface when a data message is read
- *FIFO Data Write* - Writes a message into a particular FIFO
- *FIFO Data Read* - Reads a message out of a particular FIFO

The following pages outline the basic structure of the *Port Messages*. For more information on the software- controlled, sub-fields of the *Port Messages*, see the [OS81092 INIC API User's Manual \[2\]](#).

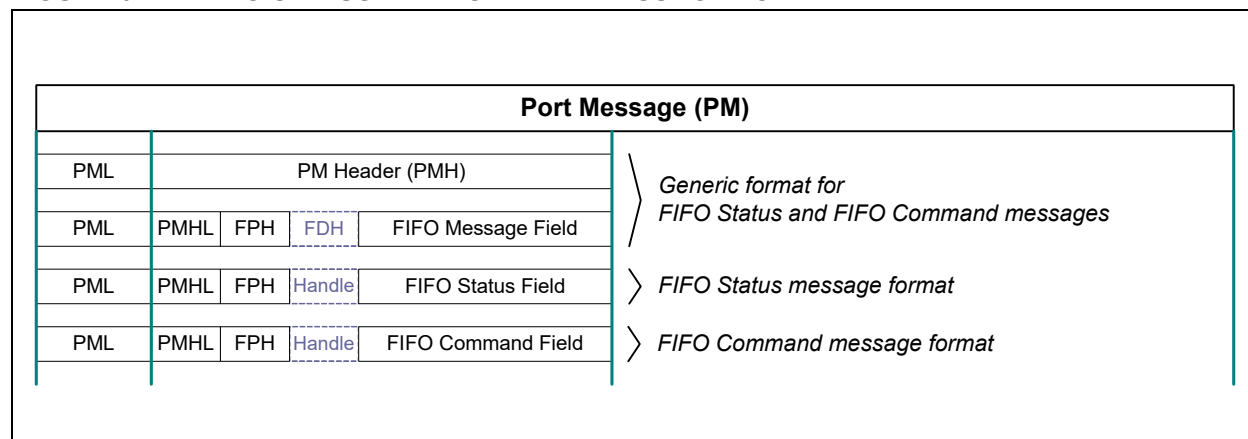
6.2 FIFO Status and Command Messages

FIFO Status and Command messages are comprised of the following fields:

- *Port Message Length* (PML) - This 16-bit field specifies the total byte length of the message, not including this field.
- *Port Message Header Length* (PMHL) - This single byte field specifies the total byte length of the two fields that follow (*FIFO Protocol Header* and *FIFO Message Field*).
- *FIFO Protocol Header* (FPH) - Typically one byte in length, this field defines the message type and the FIFO to be targeted by the message. Support for an additional header byte is available.
- *FIFO Data Header* (FDH) - Optional field (based on the FPH setting) that supports a one-byte Handle.
- *FIFO Message Field* - This field contains the message payload for FIFO Command and Status messages. In FIFO Command messages, this field specifies INIC behavior for failed messages, along with a FIFO synchronization command. In FIFO Status Read messages, this field contains status information about the receive and transmit FIFOs, as well as results of the last message transmitted.

Figure 6-2 illustrates the main fields of the FIFO Status and FIFO Command messages.

FIGURE 6-2: FIFO STATUS AND COMMAND MESSAGE FORMAT



6.3 FIFO Data Messages

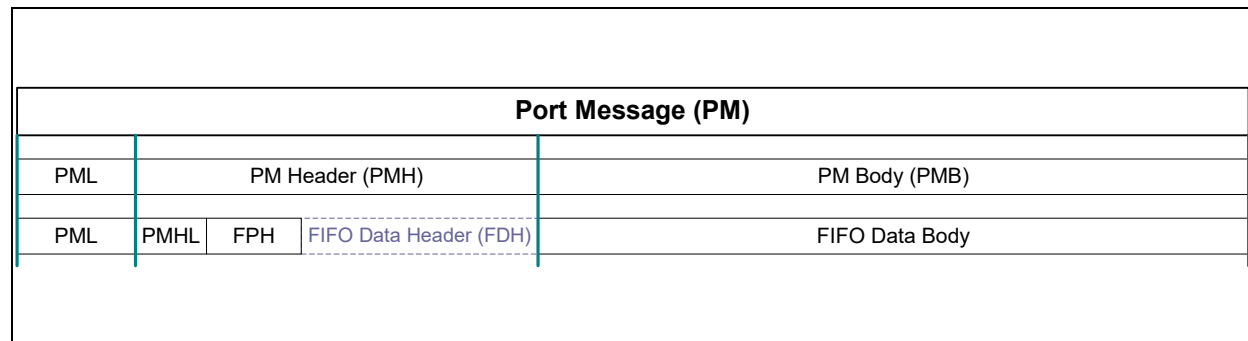
FIFO Data messages, which indicate either a read or write of a control message or asynchronous packet, are comprised of the following fields:

- **Port Message Length (PML)** - This 16-bit field specifies the total byte length of the message, not including this field.
- **Port Message Header Length (PMHL)** - This single byte field specifies the total byte length of the two fields that follow (*FIFO Protocol Header* and *FIFO Data Header*).
- **FIFO Protocol Header (FPH)** - At least one byte in length, this field defines the message type and the FIFO to be targeted by the message. The length is extensible up to three bytes on a byte-basis.
- **FIFO Data Header (FDH)** - This field contains message information, such as Target Address, Source Address, Functional Address (`FBlockID.InstID` part of the control message), message priority, and retry parameters. This is a variable-length field, based upon FPH settings.
- **Port Message Body (PMB)** - This field (also known as the *FIFO Data Body*), contains the actual message, and has a total length defined by $(PML - PMHL - 1)$. In addition to payload, this field defines the `FktID.OpType(Parameters)` part of the control message, required by the [MOST Specification \[1\]](#) for typical control messages.

For MOST Data Packets (MDP), this field contains a 16-bit length sub-field and a maximum MOST Packet message payload of 1014 bytes when the message is sent via the MediaLB interface. When sent via the Control Port, the maximum MOST Packet payload is 50 bytes.

Figure 6-3 illustrates the main fields of the FIFO Data (Read/Write) messages.

FIGURE 6-3: FIFO READ AND WRITE MESSAGE FORMAT



6.4 FIFO Message Exchange

All FIFO Data messages require a status message response from the receiving device. For example, an ICM FIFO Data message sent to INIC results in the following status message being sent to the EHC:

```
ICM:Status(XmitSuccess)
```

This status message indicates the OS81092 received the message successfully. Likewise, when the OS81092 sends a FIFO Data message to the EHC (from the network or as a response to a previous request), the EHC is required to respond with a status message indicating message reception was successful.

INIC provides an application watchdog to monitor the *Port Message* transfer during a predefined time frame. The watchdog timer times out under either of the following two cases:

- a status message response to a data message is not sent by the EHC within the timeout period, or
- INIC does not receive an MCM or ICM data message from the EHC within the timeout period.

The watchdog timer is configured through the `WatchdogMode()` INIC function. For more information on configuring the watchdog timer, refer to the [OS81092 INIC API User's Manual \[2\]](#).

6.4.1 MOST CONTROL MESSAGES (MCM)

MOST Control Messages (MCMs) are transferred between INIC and the EHC on either the MediaLB Port or the Control Port. MCMs can target the EHC FBlocks, INIC's FBlock `NBMIN`, or FBlocks that exist on the MOST Network. Segmented MCM messages are supported.

The default structure of an MCM sent from INIC to the EHC:

```
PML.06.04.TgtDevType.SrcDevID.FBlockID.InstID.FktID.OpType.TelID.Length.Data
```

The default structure of an MCM sent from the EHC to INIC:

```
PML.05.04.TgtDevID.FBlockID.InstID.FktID.OpType.TelID.Length.Data
```

6.4.2 INIC CONTROL MESSAGES (ICM)

INIC Control Messages (ICMs) are very similar to MCM messages, with the exception that ICM messages are always to/from the local FBlock `INIC` directly. Unlike MCM messages, ICM messages do not support segmentation.

ICM and MCM messages use separate FIFOs; therefore, ICM messages are never blocked by MCM messages that may need to be retried, or fail to get sent altogether. The *ICM Rx FIFO* and *ICM Tx FIFO* directly interface with the local FBlock `INIC` (which is always available) and provide faster response times for system-important functions, such as the watchdog timer.

The default structure of an ICM sent from INIC to the EHC:

```
PML.01.14.FktID.OpType.0.Length.Data
```

The default structure of an ICM sent from the EHC to INIC:

```
PML.01.14.FktID.OpType.0.Length.Data
```

6.4.3 MOST DATA PACKETS (MDP)

For *MOST Data Packets* (MDPs), INIC acts as a liaison between the EHC and the MOST Network (no INIC internal FBlocks handle MDPs). All MDP messages from the EHC must target devices that reside on the MOST Network. Similarly, INIC passes all MDPs from the MOST Network to the EHC.

The default structure of an MDP sent from INIC to the EHC:

```
PML.05.0C.TgtDevType.SrcDevID.(FillerBytes).Length.Data
```

The default structure of an MDP sent from the EHC to INIC:

```
PML.05.0C.Prio.TgtDevID.(FillerBytes).Length.Data
```

FIFO Status messages always indicate a successful transmission and are only returned for the MDP FIFO when using the Control Port to exchange asynchronous packets.

Note: MDP messages from the EHC to INIC require the entire message header (includes: PML, PMHL, FPH, and FDH) be a four-byte multiple. If the message header byte length is not divisible by four, filler bytes (containing zeros) must be added after the FDH, and PML must be updated accordingly.

7.0 CONTROL PORT

Using the default *INIC Configuration String*, the OS81092 Control Port operates as an I²C slave device that must be managed externally. The OS81092 Control Port may be configured as an I²C master by changing the settings in the *INIC Configuration String*. See the [OS81092 INIC API User's Manual \[2\]](#) for more information. In master mode, the OS81092 performs read and write operations with on-board I²C slave devices based on remote commands received from the MOST Network or from a local EHC.

Note: Control Port operation in master mode is only available when using OS81092 Revision C1D or later. Refer to [Appendix C: "Feature Matrix"](#) for more information.

7.1 Slave Operation

The Control Port supports message-based communication exclusively between the OS81092 and the EHC. These messages are used to configure and control INIC and target the local INIC FBlocks (*INIC* or *NBMIN*). The Control Port also supports the routing of asynchronous packets and control messages between the EHC and the network.

The Control Port supports the I²C protocol, where the data is received MSB-first, and is processed in bytes by the internal firmware of the OS81092. The processing time varies based on the overall load of tasks. Clock stretching provides an appropriate handshake mechanism to adapt the data transfer rate dynamically, thereby maximizing the transfer rate at any given time.

The Control Port is enabled by default after power-up/reset based on the *INIC Configuration String*. Specifically, the Control Port is enabled under one of the following two conditions:

- `PortConfiguration.DefPort()` is set to `I2C`, or
- `PortConfiguration.DefPort()` is set to `INT_Select` and $\overline{\text{INT}}$ is pulled high.

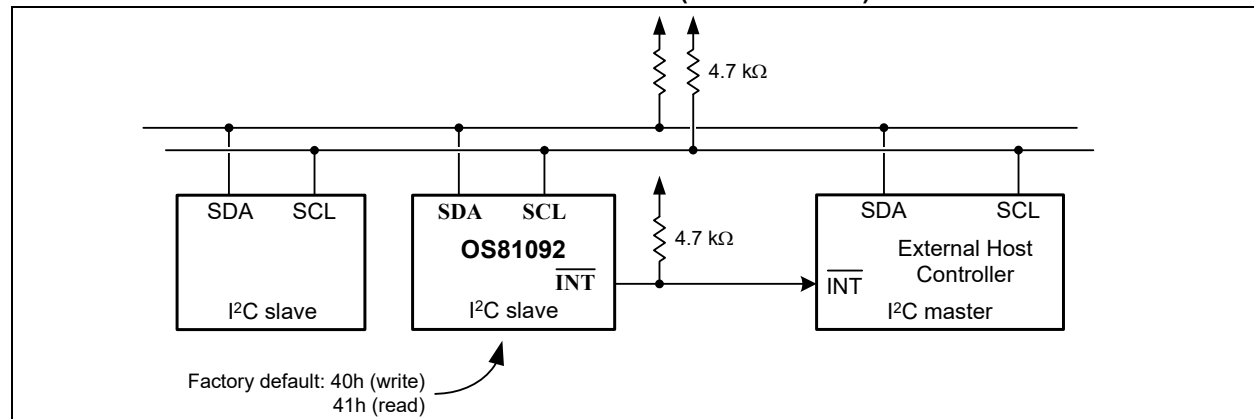
If one of these conditions is not met when the OS81092 is powered-up (or reset), the Control Port is disabled and all communication between the EHC and the OS81092 must occur through the MediaLB Port.

If OTP memory is not initialized, the OS81092 falls back to predetermined Factory defaults, where `INT_Select` is used as the default setting for `PortConfiguration.DefPort()`. If the Control Port is enabled at power-up/reset in this manner (e.g. $\overline{\text{INT}}$ pulled high) with uninitialized OTP memory, the Factory default is I²C address of 40h/41h.

The OS81092 Control Port operates as an I²C slave device that must be managed externally. The SCL pin clocks data in and out; SDA is a bi-directional data pin. $\overline{\text{INT}}$ is an open-drain, active low output that can be wire-OR'ed with the interrupt outputs of other devices into the EHC's interrupt input.

Figure 7-1 illustrates the OS81092 in an I²C environment.

FIGURE 7-1: CONTROL PORT PIN CONNECTIONS (SLAVE MODE)



When the Control Port is enabled, INIC drives the $\overline{\text{INT}}$ interrupt pin low to inform the EHC when service through the port is required (e.g. message available for reading). Following reset, the EHC should wait for the first falling edge of $\overline{\text{INT}}$, which occurs when initialization is complete. The first available message through the Control Port (shown below) indicates INIC is ready to receive messages and the EHC should synchronize all FIFOs.

```
AllFIFOs:Status(SyncS, StartUp)
```


7.1.2 READING FROM THE CONTROL PORT

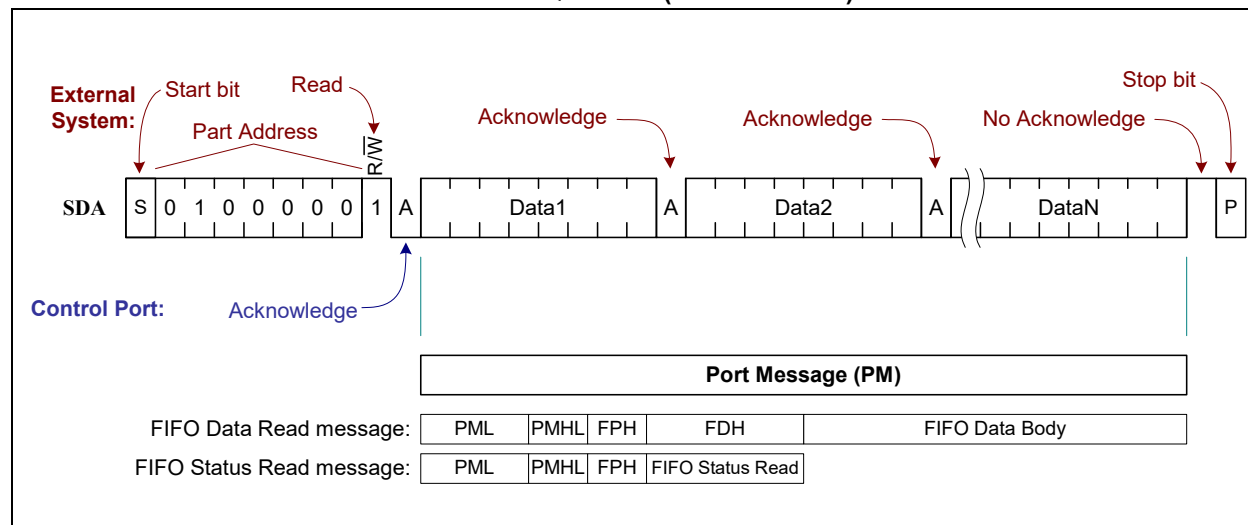
The beginning of transmission is marked by a start condition. The first byte specifies the chip address, as well as whether the operation is a read or a write. The OS81092 address (Factory default of 0100000) occupies the upper seven bits of the first byte; the R/W bit is the LSB.

When the Control Port receives the address byte of 41h (Factory default OS81092 address and R/W bit set), it acknowledges reception of the byte through an acknowledge bit and the rest of the data is read from the Control Port. The read data is defined as the *Port Message* (PM) and can be one of the following:

- FIFO Data (Read) message
- FIFO Status (Read) message

Figure 7-3 illustrates the I²C transmission read sequence. The characters “S” and “P” represent the start and stop conditions for messages.

FIGURE 7-3: CONTROL PORT READ SEQUENCE (SLAVE MODE)



The $\overline{\text{INT}}$ pin going low indicates that a *Port Message* is available in at least one of the message FIFOs. Once a message starts being read, the $\overline{\text{INT}}$ pin is de-asserted. If other messages are pending (waiting to be read), the $\overline{\text{INT}}$ pin is asserted again. This pending interrupt can occur as soon as the last byte of the current *Port Message* is read.

If a *Port Message* is not fully read (stop or repeated start bit received before the end of the message), $\overline{\text{INT}}$ is reasserted and the next read command starts reading from the beginning of the disrupted message, not the position where the read was previously terminated.

Ideally, the EHC reads the entire length of the *Port Message* in one Control Port read cycle, where the total number of bytes read is extracted from the Port Message Length (PML) during the read cycle. For I²C drivers that are not able to decipher the message length during a single read cycle, other methods of reading the *Port Message* are feasible, including the following:

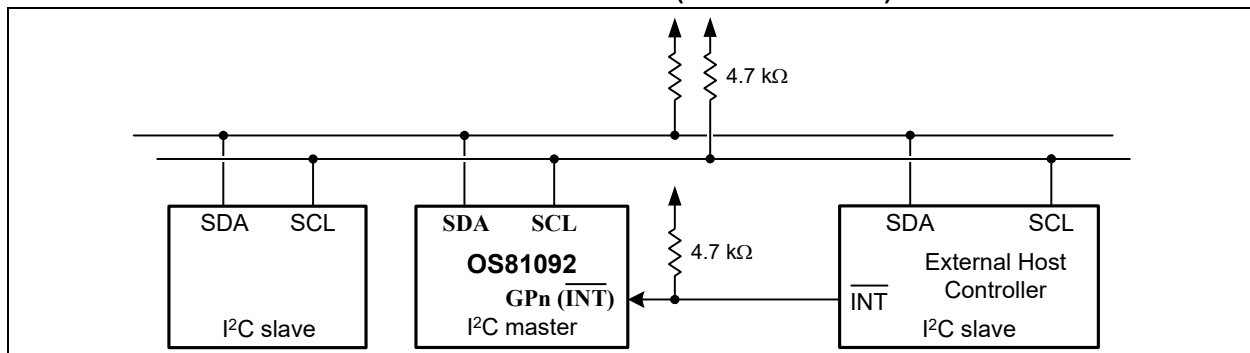
- Example 1: The EHC may issue two consecutive Control Port reads. The first cycle reads two bytes of the *Port Message*, where PML is extracted. Based on the message length obtained from PML in this first read cycle, a second cycle reads the entire *Port Message* (including the first two bytes again).
- Example 2: The EHC may issue read cycles that always read the *maximum* length of a *Port Message*, regardless of the actual PML. Once the *Port Message* is read, application software can later determine the relevant portion of the read data using the actual PML. The OS81092 returns 00h data when a read cycle extends beyond the actual length of the *Port Message*.

7.2 Master Operation

Note: When used in I²C master mode, the Control Port is unavailable for use as an I²C slave device and cannot be used for Port Message exchange with an EHC.

When operating as an I²C bus master, the OS81092 supports standard 7-bit device addressing mode as well as the enhanced 10-bit I²C addressing mode. 10-bit addressing expands the number of possible device addresses while maintaining backwards-compatibility with devices that only support 7-bit addressing. The OS81092 supports full interoperability for systems with mixed 7-bit and 10-bit slave device addresses. Per the [I²C-Bus Specification \[5\]](#), multi-master configurations allow more than one master to control the bus at the same time without corrupting data exchange. [Figure 7-4](#) illustrates the OS81092 as a master in an I²C environment.

FIGURE 7-4: CONTROL PORT PIN CONNECTIONS (MASTER MODE)



I²C bus master mode offers two different settings for I²C transaction speed. The SCL clock frequency can be set to either 100 kHz (Standard-mode) or 400 kHz (Fast-mode). Faster speed modes defined in the [I²C-Bus Specification \[5\]](#) (e.g. Fast-mode plus, High-speed mode) are not supported by the OS81092. The speed setting is selected when the port is opened.

Clock stretching is a handshaking mechanism used by slave devices to adapt the data transfer rate dynamically. A slave device is permitted to hold SCL low after receiving (or sending) a bit to indicate it is not yet ready to process additional data. To ensure compatibility with all slave devices, the OS81092 supports clock stretching at the bit and byte boundary.

Some I²C slave devices implement an interrupt pin to indicate to the I²C master that service is needed. The OS81092 GPIO Port can be configured to support devices that implement interrupt functionality. Refer to [Chapter 12.0 "General Purpose Input/Output \(GPIO\) Port"](#) for more information.

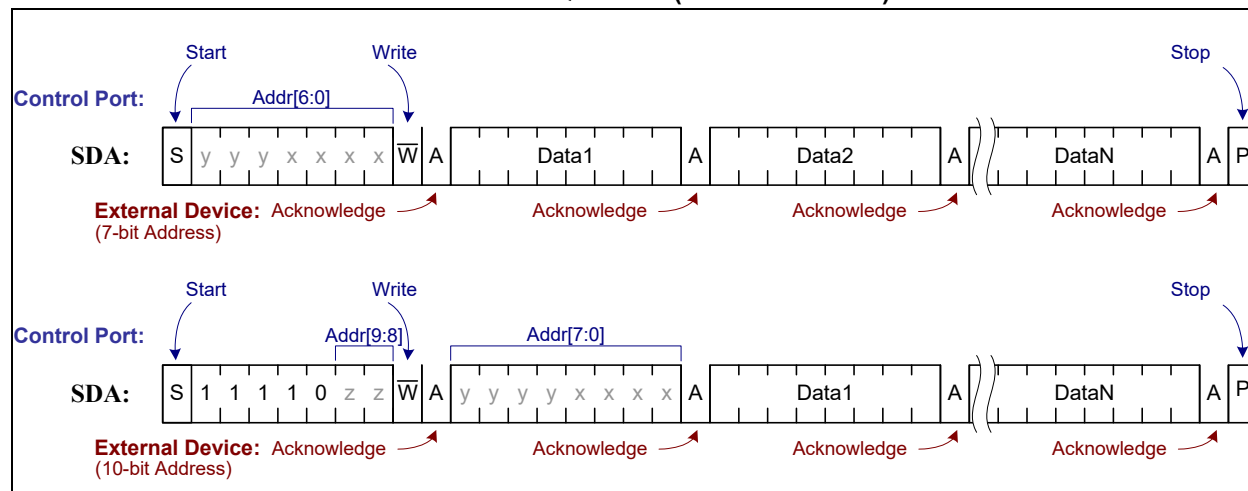
Once master mode is enabled, remote I²C read and write sequences are initiated by a remote node on the MOST Network. The read/write methods specify the transfer mode, the slave target address, and the number of bytes to be read/written. The maximum block size for a read/write operation is 32 bytes of data. A successful read transaction returns the device address, the number of bytes read, and the list of data bytes. When a write operation successfully finishes, the device address and number of bytes written is returned.

Refer to the [OS81092 INIC API User's Manual \[2\]](#) for the function reference and more information on using the OS81092 Control Port in master mode.

7.2.1 WRITING TO AN I²C SLAVE DEVICE

Per the [I²C-Bus Specification \[5\]](#) a '11110' following a start condition indicates use of the 10-bit addressing scheme. For a write operation the first byte contains the two most significant bits of the 10 bit device address as well as the R/W flag. Following acknowledgment from the external device, the remaining eight address bits are sent in the second byte. Message payload (Data1 - DataN) begins in the third byte of the message for 10-bit addressing. [Figure 7-5](#) below shows a write operation for both 7-bit and 10-bit addressing modes.

FIGURE 7-5: CONTROL PORT WRITE SEQUENCE (MASTER MODE)

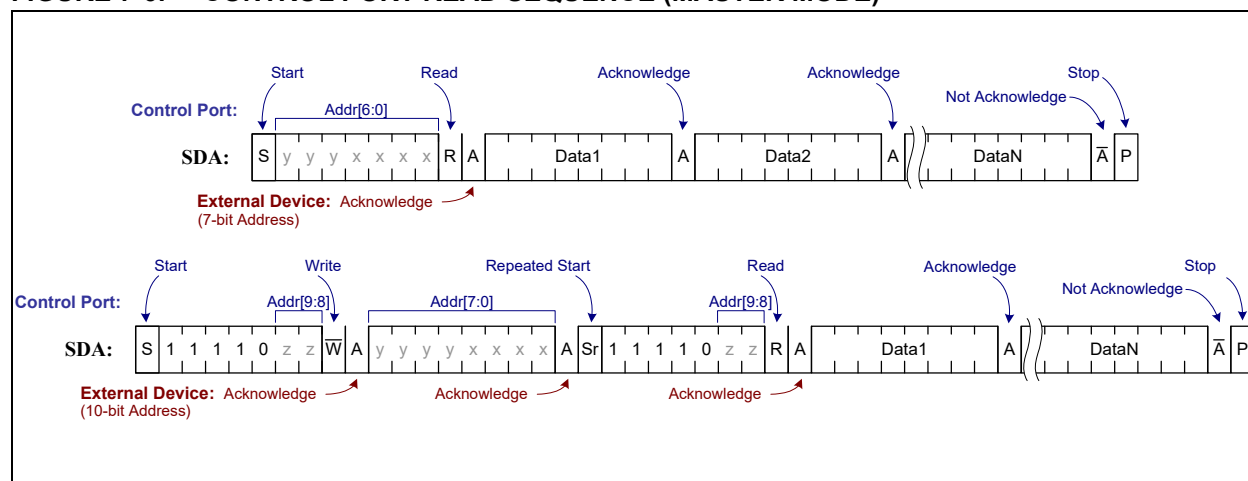


When using 10-bit addressing, the second address byte is actually sent as the first data byte. Therefore, the total length of the message to be sent on the Control Port must be adjusted appropriately. Refer to the [OS81092 INIC API User's Manual \[2\]](#) for more information.

7.2.2 READING FROM AN I²C SLAVE DEVICE

Read operations on slave devices with 10-bit addresses require a Repeated Start condition. Refer to the [I²C-Bus Specification \[5\]](#) for more information. The master first fully addresses the 10-bit slave device with a write operation and then issues a Repeated Start condition. After the Repeated Start, the direction is changed and the master completes the read transaction with an abbreviated version (only the two most significant bits) of the address. [Figure 7-6](#) below shows a read operation for both 7-bit and 10-bit addressing modes.

FIGURE 7-6: CONTROL PORT READ SEQUENCE (MASTER MODE)



8.0 MediaLB PORT

The *Media Local Bus* (MediaLB) Port gives external devices (e.g. the External Host Controller) real-time access to all supported types of data on the MOST Network. The MediaLB Port and the Streaming Port share some pins; therefore, some Streaming and MediaLB modes are mutually exclusive.

The MediaLB Port can be configured for one of the following operational modes:

- MediaLB 3-pin Mode - When configured in this mode, the MediaLB Port provides a 3-pin interface that supports a maximum data rate of $1024 \times F_s$ (49.152 MHz at $F_s = 48$ kHz). In this mode, the Streaming Port may be configured with up to two serial interfaces for accessing synchronous network data.
- MediaLB 5-pin Mode - When configured in this mode, the MediaLB Port provides a 5-pin interface that supports a maximum data rate of $512 \times F_s$ (24.576 MHz at $F_s = 48$ kHz). In this mode, the Streaming Port is not available.

MediaLB bandwidth is managed in quadlets (4 bytes) and consists of a combination of streaming, packet, and control data. The MediaLB Port rate of $256 \times F_s$ supports up to seven quadlets of data, divided among streaming, packet, and control data types. The MediaLB Port rate of $512 \times F_s$ supports up to 15 quadlets of data. The MediaLB Port rate of $1024 \times F_s$ supports up to 31 quadlets of data. A maximum of 64 bytes can be transferred per frame, in either direction, for the Streaming Port and MediaLB Port combined.

When transmitting data onto the network, the MSB is transmitted first. To comply with the [MOST Specification \[1\]](#), when transmitting multi-byte words, the most significant byte should be transmitted first. In addition, when transmitting stereo data, left should be sent before right.

The MediaLB Port can be enabled by default after power-up/reset based on the *INIC Configuration String*. If the MediaLB Port is not enabled by default at power-up/reset, it can be manually enabled and configured via ICMs through the Control Port, if desired. The MediaLB Port is enabled automatically at power-up/reset under one of the following two conditions:

- `PortConfiguration.DefPort()` is set to MediaLB, or
- `PortConfiguration.DefPort()` is set to `INT_Select` and $\overline{INT}$ is pulled low.

If OTP memory is uninitialized, the OS81092 falls back to predetermined Factory defaults, where `INT_Select` is used as the default setting for `PortConfiguration.DefPort()`. If the MediaLB Port is enabled at power-up/reset in this manner (e.g. $\overline{INT}$ pulled low) with uninitialized OTP memory, the Factory default mode and speed is 3-pin mode at $1024 \times F_s$.

Although $\overline{INT}$ is not used for MediaLB communications, if pulled high, it can be used to wake up the EHC after a device shut-down. See the `ShutDown()` function in the [OS81092 INIC API User's Manual \[2\]](#) for more information.

After reset, and following MediaLB Port initialization, the first INIC message sent to the EHC indicates the OS81092 is initialized and ready to communicate with the EHC.

```
AllFIFOs:Status(SyncS, StartUp)
```

In common applications, this message is used to instruct the EHC to synchronize its FIFOs. Additionally, the EHC may start synchronization of the OS81092 by sending the following message:

```
AllFIFOs:Command(SyncC)
```

The OS81092 responds to the above message once the synchronization is complete. This procedure ensures that OS81092 and the EHC can synchronize communication under various circumstances. The status message sent by the OS81092 is:

```
AllFIFOs:Status(SyncS, SyncRcvd)
```

8.1 MediaLB 3-pin Mode

The MediaLB Port provides a conduit for accessing the content of the MOST Network via a single hardware interface. All data types exchanged across network channels are supported across MediaLB.

The MediaLB Port also allows access to the internal functions of the OS81092, eliminating the need for Control Port access. With MediaLB, only one interface is needed for data and control. When combined with the MediaLB software API library, the system designer is insulated from much of the complexity of managing the interface between the application software and the network.

The MediaLB 3-pin interface consists of:

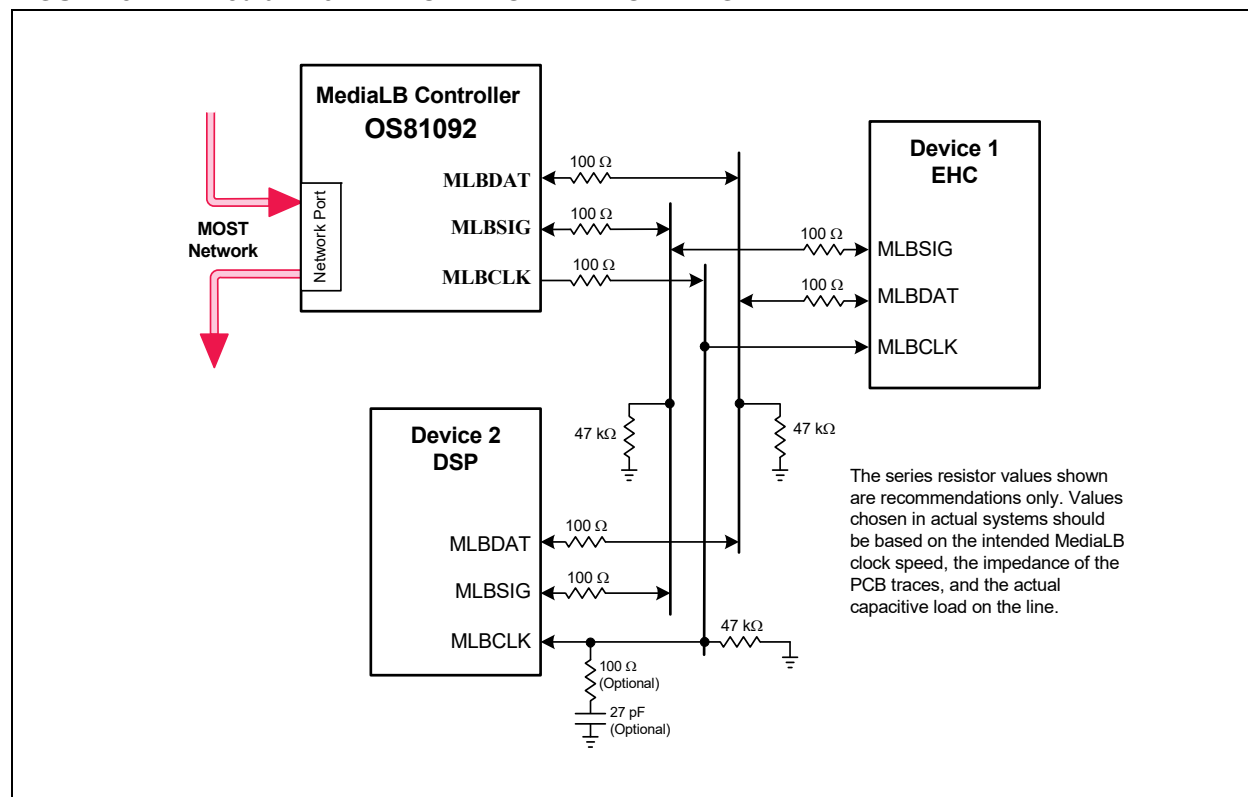
- **MLBCLK** - MediaLB Controller output signal that clocks data in and out of the MediaLB Port
- **MLBDAT** - bi-directional signal that transports data to and from the MediaLB Port
- **MLBSIG** - bi-directional signal that transports signal information to and from the MediaLB Port

Note: When the MediaLB Port is configured in MediaLB 3-pin mode, the Streaming Port can be simultaneously configured with two serial interfaces.

The MediaLB Controller (OS81092) uses **MLBSIG** to grant bus access to a MediaLB Device (e.g. EHC). That Device can then send data to other Devices (including the Controller). The OS81092 **MLBDAT** pin receives data for configuration or transmission onto the network, and transmits data to other Devices from the network receiver. Data and signal information is shifted into Devices by **MLBCLK**. A complete description of the Media Local Bus can be found in the [MediaLB Specification \[3\]](#).

In the MediaLB 3-pin connection diagram shown in [Figure 8-1](#), each MediaLB line has a weak pull-down to keep the signals in a known state when no Device is driving the line. MediaLB Devices that transmit on the line also have a series resistor near the Device for series termination and rise/fall time control. The clock line may optionally have AC-parallel termination at the farthest point from the Controller (clock source) to minimize reflections and ensure a clean clock.

FIGURE 8-1: MediaLB 3-PIN MODE CONNECTION DIAGRAM - EXAMPLE



The MediaLB protocol supports a set of physical channels for sending data over the Media Local Bus. The physical channels (PCn) are four bytes in length (one quadlet) and can be grouped into logical channels. Each logical channel represents a unidirectional data path (one or more quadlets in length) and is defined by a unique *ChannelAddress*.

On each logical channel, the source (transmitter) is a unique location on a Device, with a destination (receiver) of one or more Devices. In a system, the *ChannelAddresses* are generally static and defined at the development stage; however, the MediaLB Controller can dynamically define additional *ChannelAddresses* during operation to communicate with other Devices.

The MediaLB Controller initiates communication over the Media Local Bus by sending out a *ChannelAddress* on **MLBSIG**. This indicates to the addressed Device that it can begin transmitting on the logical channel associated with that *ChannelAddress*. The structure of the *ChannelAddress* is described in the [MediaLB Specification \[3\]](#), *Link Layer Section*.

The number of *ChannelAddresses* supported is based on the MediaLB Port speed and the size of the logical channels associated with the *ChannelAddresses*. The total number of physical channels available at a given MediaLB Port speed is indicated in [Table 8-1](#). The mapping of specific physical channels into a logical channel is arbitrary and managed by the MediaLB Controller. Generally, logical channels are created by the EHC using the `CreateSocket()` INIC function.

TABLE 8-1: AVAILABLE MediaLB BANDWIDTH

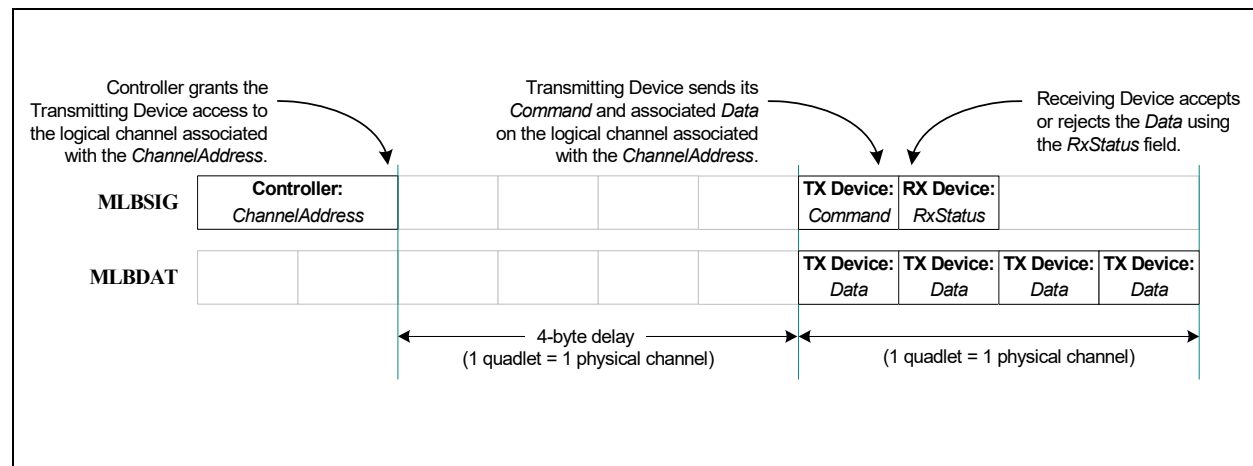
MediaLB Speed	Quadlets per MediaLB Frame	Physical Channels	Available Physical Channels *	Bytes Available per Frame *
256×Fs	8	PC0 - PC7	7	28
512×Fs	16	PC0 - PC15	15	60
1024×Fs	32	PC0 - PC31	31	124

* PC0 (first physical channel of the MediaLB frame) is always used as the *SystemChannel*; therefore, the bytes of this physical channel are not available for application data.

Once per network frame, the MediaLB Controller generates a unique 16-bit *FRAMESYNC* pattern on the signal information line (**MLBSIG**). The end of the *FRAMESYNC* pattern defines byte and physical channel boundaries, and indicates that the MediaLB frame boundary occurs one quadlet later. Physical Channel 0 (PC0), defined as the first physical channel of the MediaLB frame, is always used as the *SystemChannel*. The *FRAMESYNC* pattern represents a special *ChannelAddress*, which grants the MediaLB Controller (OS81092) PC0 for system administration.

As illustrated in [Figure 8-2](#), one quadlet after the *ChannelAddress* is sent by the Controller, the Device associated with that *ChannelAddress* sends out a MediaLB *Command* on **MLBSIG**, while simultaneously sending the corresponding *Data* over **MLBDAT**. All MediaLB Devices, including the OS81092 Controller, compare the *ChannelAddress* to their internal Channel Address table to determine whether they are the intended receiver.

FIGURE 8-2: MediaLB DATA STRUCTURE



When the OS81092 Controller is a receiving Device, it places the *RxStatus* response on **MLBSIG** one byte after the MediaLB *Command* is sent by the transmitting Device. To accept the *Data*, an *RxStatus* response of *ReceiverReady* is sent. To reject the *Data*, an *RxStatus* response of *ReceiverBusy* is sent.

Note: In accordance with the [MediaLB Specification \[3\]](#), the OS81092 does not acknowledge synchronous data transmissions (*RxStatus* default response of *ReceiverReady*).

When the OS81092 is the receiving Device for a logical channel and recognizes a reception error, it responds with an *RxStatus* of *ReceiverProtocolError* in the next physical channel of that logical channel. The following conditions cause INIC to generate *ReceiverProtocolError*, when receiving control messages and asynchronous packets:

- Inappropriate MediaLB *Command* for a particular channel
(e.g. a synchronous command is received when a logical channel is setup for control messages).
- *ControlStart*, *AsyncStart*, *ControlEnd*, or *AsyncEnd* is received while in the middle of a packet reception.
- *ControlEnd* or *AsyncEnd* is not received when the end of the buffer is expected, based on *Port Message Length* (PML).
- *Port Message Header Length* (PMHL) + 3 bytes is not divisible by 4 (applies to packet messages only).

Figure 8-4 illustrates the MediaLB interface, as various Devices drive the bus (**MLBSIG/MLBDAT**) during a given network frame. This figure depicts the 256×Fs MediaLB speed; however, the logic also applies to the 512×Fs and 1024×Fs speeds (with more PCn channels).

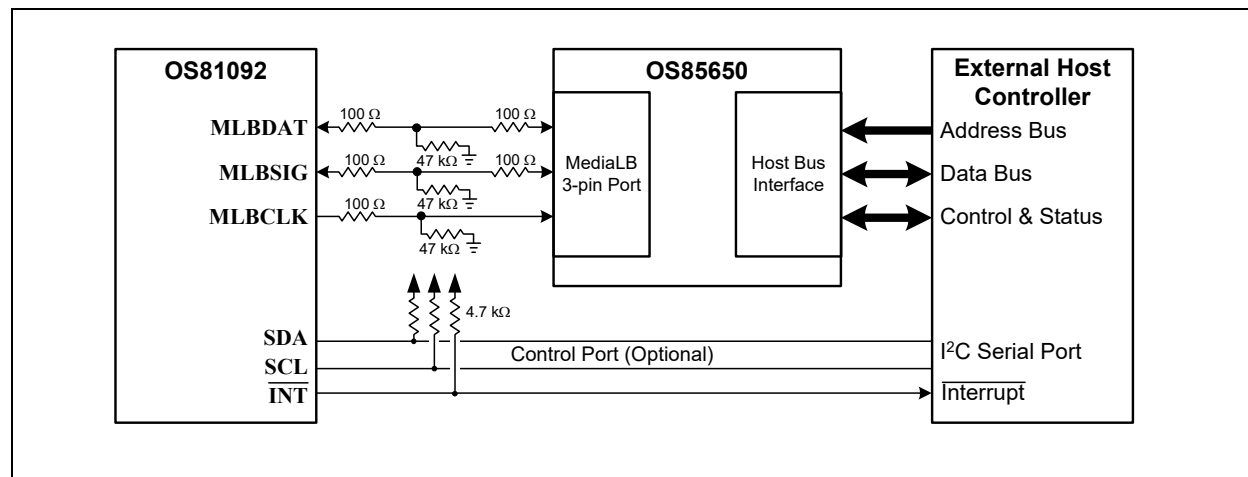
The Controller drives the *ChannelAddresses* (*ChannelAddress* B through H in this example), granting the corresponding Devices bus access. The transmitting Device for the *ChannelAddress* drives the *Command* and *Data*, and the receiving Device responds with reception status (*RxStatus*). In the second to last physical channel of the frame (PC6 at 256×Fs), the Controller drives the *FRAMESYNC* pattern onto the signal information line (**MLBSIG**) in place of a *ChannelAddress*, for synchronization to the network frame. The *Command* and *Data* associated with the *FRAMESYNC* (also known as the *SystemChannel*) is sent by the Controller one quadlet following the *FRAMESYNC* pattern, in the first physical channel of the frame (PC0).

Depending on the number of physical channels grouped into logical channels, fewer unique *ChannelAddresses* may be seen in the frame. In Figure 8-4, each logical channel is one quadlet (one physical channel), mapping to seven *ChannelAddresses* (B through H). If one logical channel consisted of two quadlets and another consisted of three quadlets, then only four unique *ChannelAddresses* would be seen on the bus (B through E).

The OS81092 opens default communication channels (0004h and 0002h) when the MediaLB Port is enabled at power-up. The OS81092 listens to *ChannelAddress* 0004h for configuration information from the EHC, and responds to the EHC on *ChannelAddress* 0002h. This *ChannelAddress* pair is used for communicating ICM and MCM messages between the EHC and INIC, and can be used to open other channels on MediaLB. A separate channel pair must be opened for MDP communications.

In applications where the EHC does not support the MediaLB interface, an IOC device (e.g. OS85650 or OS85652) can be utilized to translate between INIC's MediaLB Port and a parallel interface, as depicted in Figure 8-3. In this implementation, the IOC allows the EHC to exploit the efficiency and bandwidth of the MediaLB bus, even when the EHC does not support a MediaLB interface. See Section 4.3.6 "Port Expansion" for more information regarding the IOC family of devices.

FIGURE 8-3: MediaLB 3-PIN MODE TO PARALLEL INTERFACE



8.2 MediaLB 5-pin Mode

The MediaLB 5-pin mode is functionally equivalent to the MediaLB 3-pin mode described in [Section 8.1 "MediaLB 3-pin Mode"](#); however, two unidirectional data lines and two unidirectional signal information lines exist. Additionally, when configured for MediaLB 5-pin mode, the OS81092 supports a data rate of 256×Fs or 512×Fs (1024×Fs not supported).

The MediaLB 5-pin interface is defined as:

- **MLBDO** - unidirectional signal that transports data out of the MediaLB Port
- **MLBDI** - unidirectional signal that transports data into the MediaLB Port
- **MLBSO** - unidirectional signal that transports signal information out of the MediaLB Port
- **MLBSI** - unidirectional signal that transports signal information into the MediaLB Port
- **MLBCLK** - MediaLB Controller output signal that clocks data in and out of the MediaLB Port

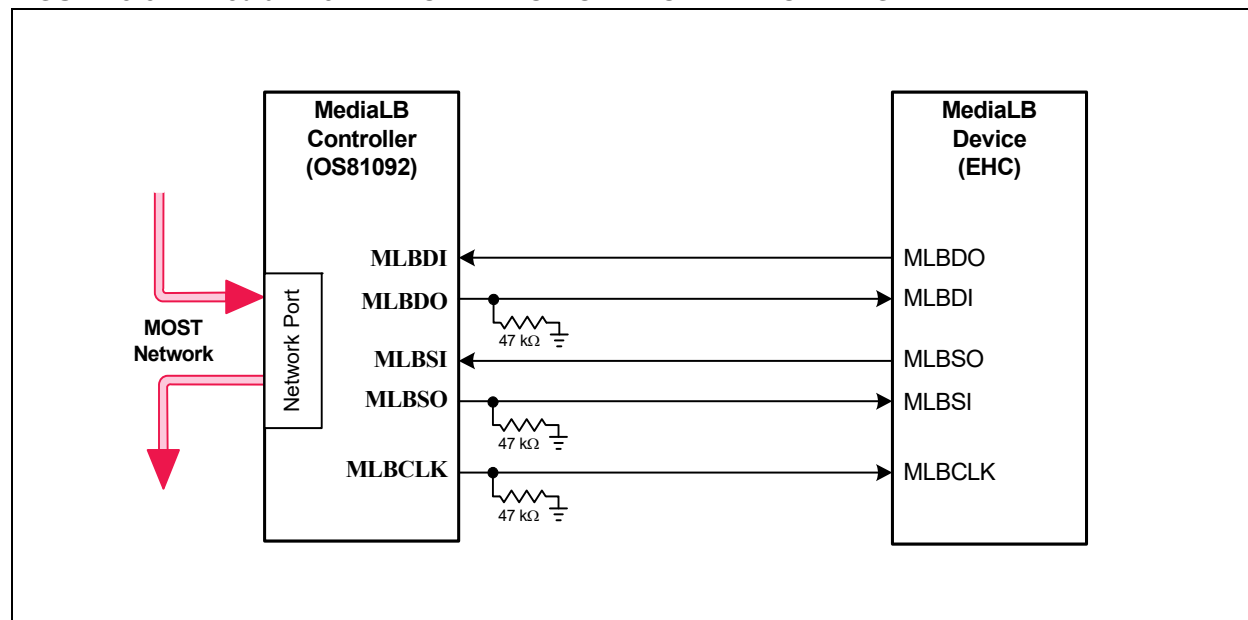
Note: The Streaming Port is not available when the MediaLB 5-pin interface is used.

The MediaLB Controller (OS81092) uses **MLBSO** to grant bus access to a MediaLB Device (e.g. EHC) and receives *Commands* and *RxStatus* responses sent out by other MediaLB Devices on **MLBSI**. The Controller receives *Data* for configuration or transmission onto the network on **MLBDI**; *Data* is transmitted to other MediaLB Devices from the network receiver on **MLBDO**.

If output signals on external MediaLB Devices can float (even during reset), then pull-down resistors are required to keep undriven data and signal information lines from affecting the other Devices driving the lines.

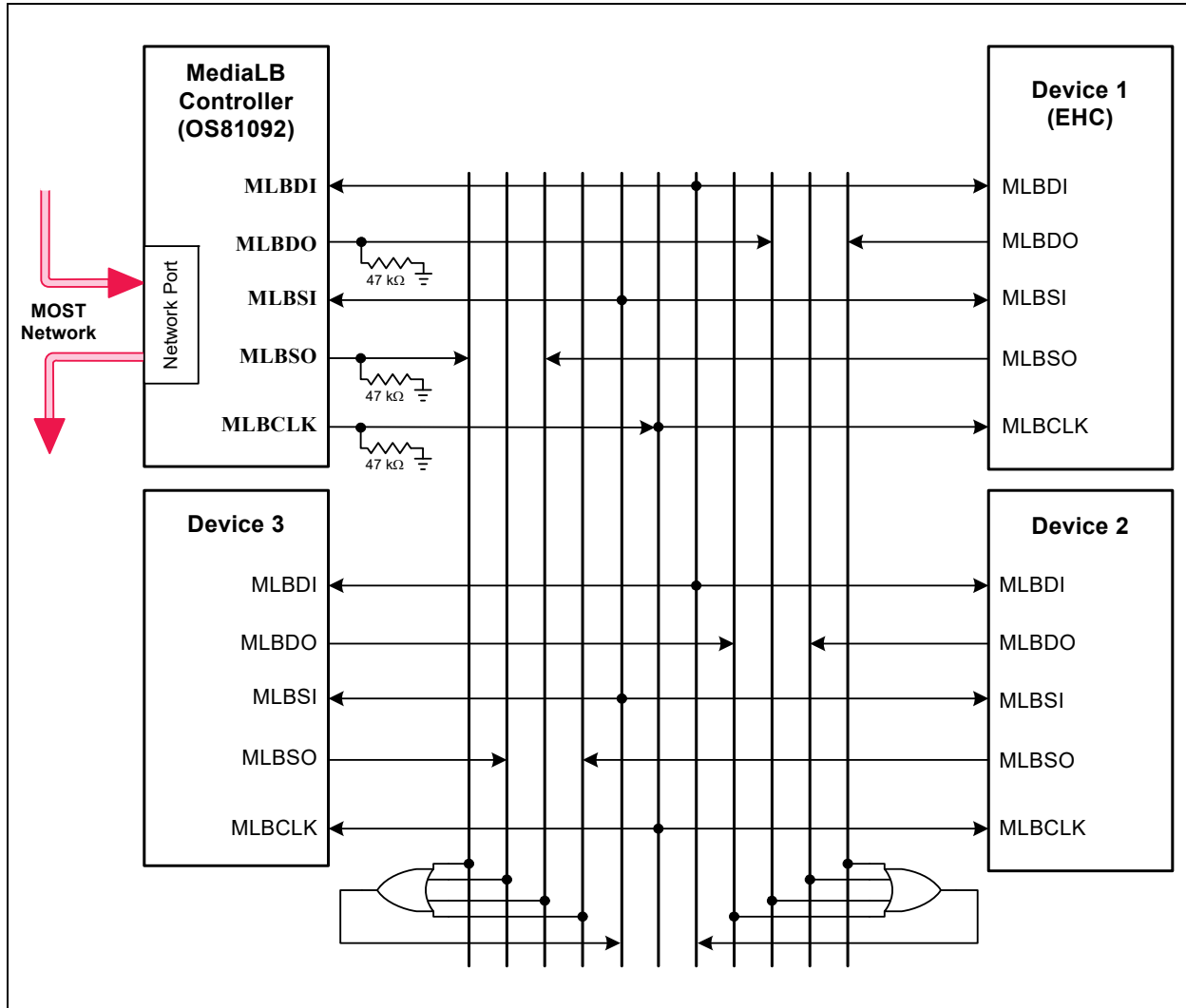
[Figure 8-5](#) shows the OS81092 as an interface between the MOST Network and the MediaLB Port, when configured for MediaLB 5-pin mode. Since the OS81092 is the interface to the MOST Network, it must be the Controller on the Media Local Bus. Using this interface, MediaLB Devices can access all MOST Network data types.

FIGURE 8-5: MediaLB 5-PIN MODE TWO-NODE CONNECTION DIAGRAM



The MediaLB Port output lines (**MLBSO** and **MLBDO**) from each Device must be OR'ed with the corresponding output lines of the other Devices on the bus. As a result, each MediaLB Device must drive their output lines low when not transmitting data. When connecting more than one MediaLB Device on the 5-pin bus, the external logic needed to sum all data and signal outputs to each Device is shown in [Figure 8-6](#).

FIGURE 8-6: MediaLB 5-PIN MODE FOUR-NODE CONNECTION DIAGRAM



8.3 MediaLB Commands and Responses Not Supported

The OS81092 INIC does not support the following MediaLB *Commands* and *RxStatus* responses:

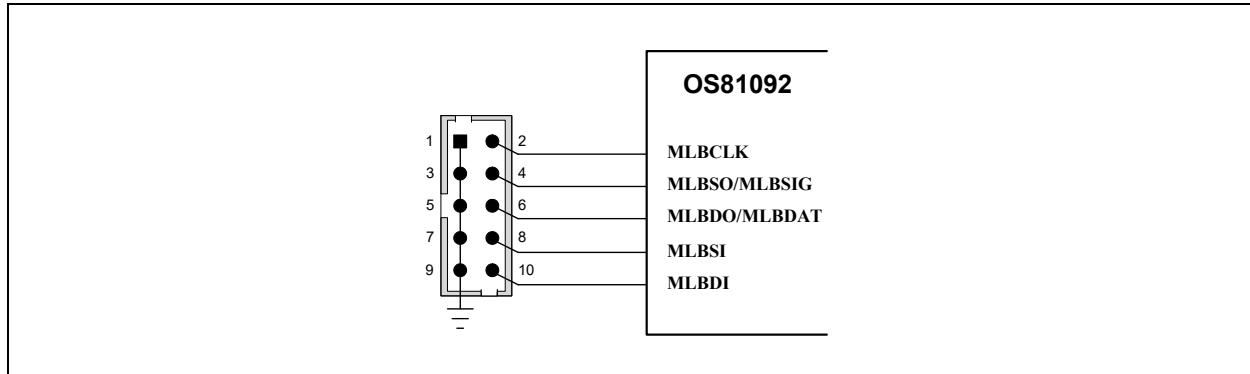
- Normal Commands:
 - All Isochronous Commands
- System Commands:
 - E0h - *MOSTLock*
 - E2h - *MOSTUnlock*
 - E4h - *MLBScan*
 - E6h - *MLBSubCmd*
 - FEh - *MLBReset*
- System Responses:
 - 80h - *DevicePresent*
 - 82h - *DeviceServiceRequest*

8.4 MediaLB Debug Header

A standard *MediaLB Debug Header* is defined to allow various MediaLB analysis and debug tools to connect directly into the application. This header is used for monitoring MediaLB in either the 3-pin or 5-pin mode, and should be placed as close as possible to the MediaLB interface to prevent unnecessary extension of MediaLB signal traces.

The pinout of the MediaLB Debug Header, as shown in [Figure 8-7](#), is compatible with *MediaLB Active Pods* for MediaLB 3-pin and 5-pin modes. Active Pods are used with various MediaLB analysis and debug tools (see the [MediaLB Monitor Adapter User's Manual \[6\]](#) and [MediaLB Analyzer User's Manual \[7\]](#)). This header should be a 10-pin, 2 mm male connector, such as Molex 87332-1020 (shrouded) or Samtec TMM-105-06-T-D-SM (not shrouded).

FIGURE 8-7: MediaLB DEBUG HEADER



9.0 STREAMING PORT

The Streaming Port is used by external devices (e.g. ADCs, DACs) to source/sink streaming data, with low overhead. The formats supported for data into and out of the Streaming Port are compatible with industry-standard serial interfaces, including: left-justified, right-justified, I²S-compatible or sequential formats.

The Streaming Port and the MediaLB Port share some pins; therefore, some modes are mutually exclusive. [Table 9-1](#) shows the relationship between the availability of Streaming Port and MediaLB Port modes.

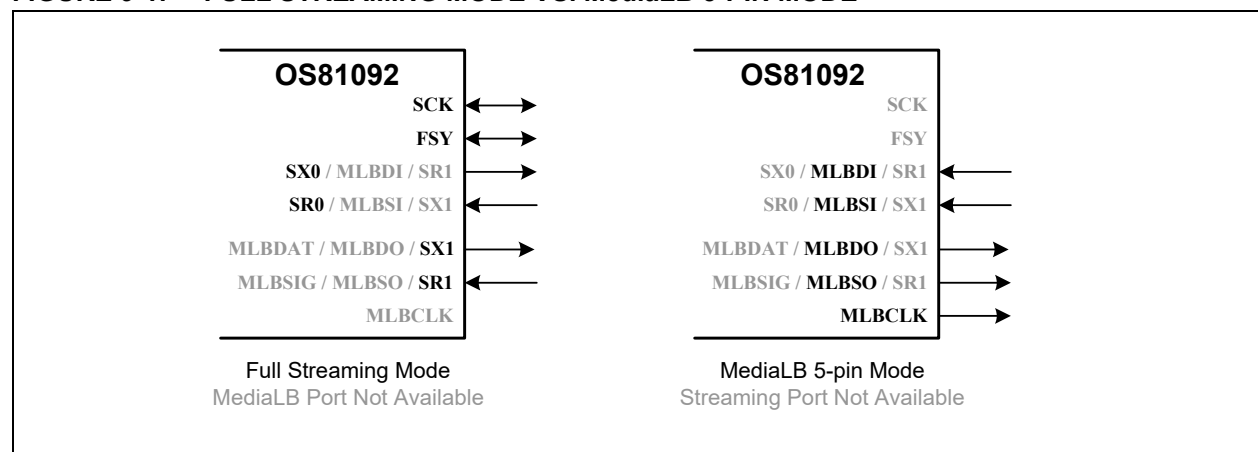
TABLE 9-1: STREAMING PORT VS. MediaLB PORT AVAILABILITY

Streaming Port	MediaLB Port
Full Streaming Mode (all 4 serial interfaces available)	Not Available
Partial Streaming Mode (2 serial interfaces available)	3-pin mode
Not Available	5-pin mode

Depending on the Streaming Port mode enabled, the OS81092 can support up to four serial data pins. All data pins on the Streaming Port share a common bit clock (SCK) and a common frame synchronization signal (FSY), which delineates the word/channel (left vs. right channel) boundaries. The bit clock (SCK) operates at 64×Fs, 128×Fs, or 256×Fs, where Fs is the frame rate of FSY. When synchronous data is exchanged on the serial data pins, the FSY and SCK pins can be configured as inputs or outputs. As inputs, they must be synchronous to the OS81092 (derived off of RMCK). As outputs, FSY and SCK are driven based on the network frame rate.

The Streaming Port can be enabled, and the format selected, through the `OpenPort()` function, while the amount of streaming data transferred is selected through the `CreateSocket()` function. [Figure 9-1](#) illustrates pin usage when the OS81092 is configured for Full Streaming Mode or MediaLB 5-pin mode.

FIGURE 9-1: FULL STREAMING MODE VS. MediaLB 5-PIN MODE



When the MediaLB Port is configured for MediaLB 3-pin mode, two serial interfaces are available through the Streaming Port. The following configuration options are available in Partial Streaming Mode:

- **InOut Option** - interface supports one serial input (SR0) and one serial output (SX0)
- **DualIn Option** - interface supports two serial inputs (SR0 and SR1)
- **DualOut Option** - interface supports two serial outputs (SX0 and SX1)

Pin usage for the various Partial Streaming Mode options is illustrated in [Figure 9-2](#).

OS81092

FIGURE 9-2: PARTIAL STREAMING MODE

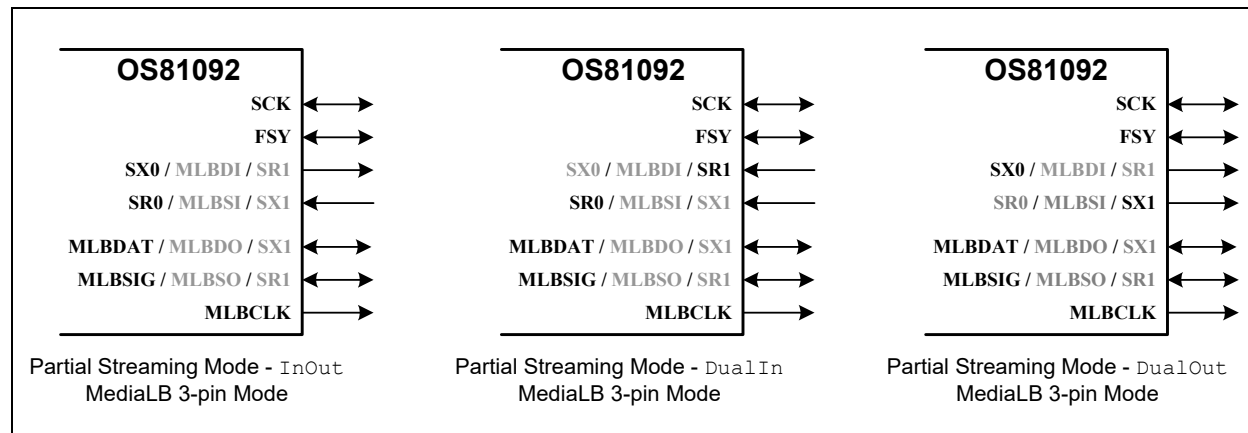


Table 9-2 shows data format and clock speed options supported by the OS81092 Streaming Port. The data alignment associated with each streaming data format is shown in the following sections, with SCK set to 64×Fs. At higher clock rates, the data alignment follows the same patterns; however, with more clocks per frame. With the sequential format (both bit-delayed and non-delayed), higher clock rates make more byte positions available.

TABLE 9-2: STREAMING DATA FORMATS

Data Format	SCK Rate	Socket Size	Streaming Data Format *
Left-Justified	64×Fs	2 bytes (1x 16 bit channel)	Left64Fs16Bit
		4 bytes (2x 16 bit channels)	
		3 bytes (1x 24 bit channel)	Left64Fs24Bit
		6 bytes (2x 24 bit channels)	
	128×Fs	4 bytes (2x 16 bit channels)	Left128Fs16Bit
		6 bytes (2x 24 bit channels)	Left128Fs24Bit
Right-Justified	64×Fs	2 bytes (1x 16 bit channel)	Right64Fs16Bit
		4 bytes (2x 16 bit channels)	
		3 bytes (1x 24 bit channel)	Right64Fs24Bit
		6 bytes (2x 24 bit channels)	
	128×Fs	4 bytes (2x 16 bit channels)	Right128Fs16Bit
		6 bytes (2x 24 bit channels)	Right128Fs24Bit
Delayed-Bit (I ² S)	64×Fs	2 bytes (1x 16 bit channel)	Delay64Fs16Bit
		4 bytes (2x 16 bit channels)	
		3 bytes (1x 24 bit channel)	Delay64Fs24Bit
		6 bytes (2x 24 bit channels)	
		up to 8 bytes	Delay64FsSeq
	128×Fs	4 bytes (2x 16 bit channels)	Delay128Fs16Bit
		6 bytes (2x 24 bit channels)	Delay128Fs24Bit
		up to 16 bytes	Delay128FsSeq
	256×Fs	up to 32 bytes	Delay256FsSeq
Sequential	8×Fs	1 byte	Seq8Fs
	16×Fs	up to 2 bytes	Seq16Fs
	32×Fs	up to 4 bytes	Seq32Fs
	64×Fs	up to 8 bytes	Seq64Fs
	128×Fs	up to 16 bytes	Seq128Fs
	256×Fs	up to 32 bytes	Seq256Fs

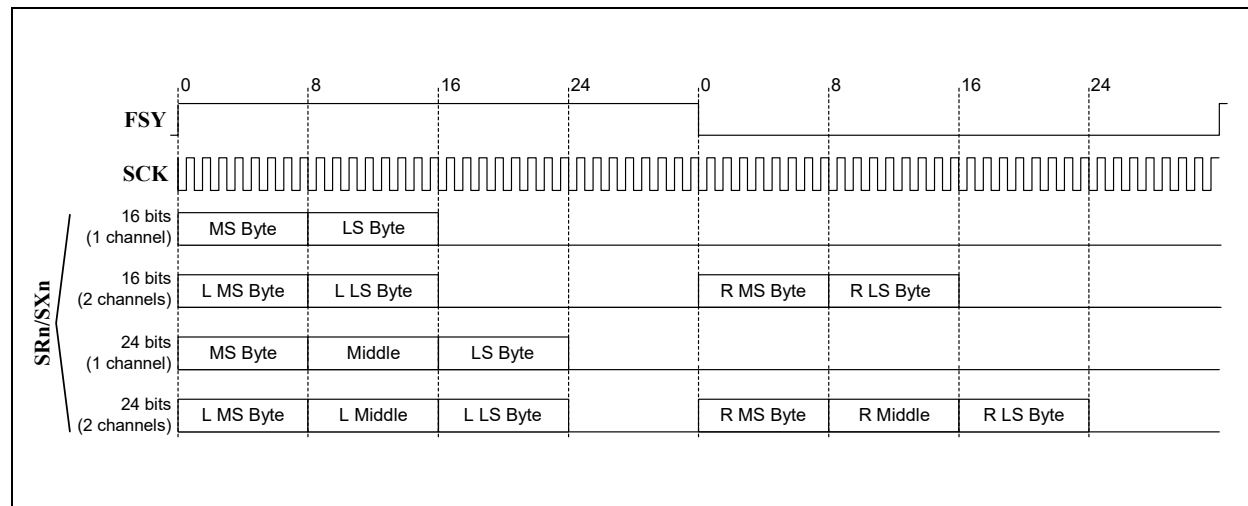
* The streaming data format is configured using the `OpenPort()` function, as outlined in the [OS81092 INIC API User's Manual \[2\]](#).

9.1 Left-Justified Alignment

When using the left-justified alignment format, the first data bit occurs on the first clock after a transition on **FSY**. The data and **FSY** signals change on the falling edges of the clock and are valid on the rising edges. Data is arranged MSB first, high byte first. The start of frame is indicated by the rising edge of **FSY**.

Figure 9-3 illustrates the data alignment and timing when the left-justified format is selected. For higher data bit rates, the channel data remains aligned with the edges of **FSY**, with more unused data bytes between channels.

FIGURE 9-3: LEFT-JUSTIFIED STREAMING DATA FORMAT



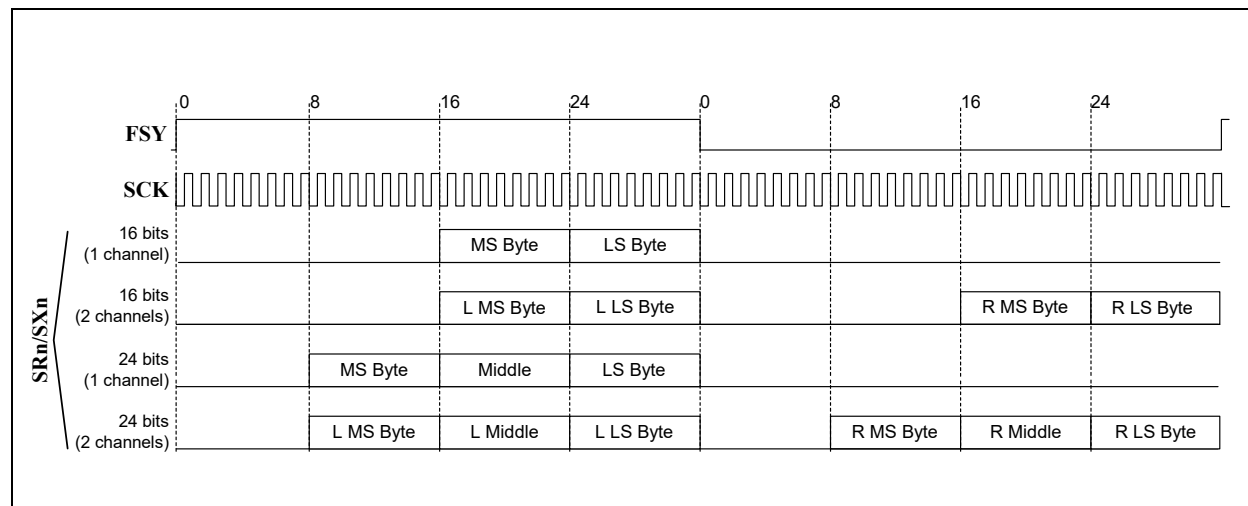
The left-justified format does not support a single (mono) channel when **SCK** is $128 \times F_s$. This mode is supported by the sequential format (see Section 9.4 “Sequential Alignment”).

9.2 Right-Justified Alignment

When using the right-justified alignment format, the last data bit occurs on the last clock before a transition on **FSY**. The data and **FSY** signals change on the falling edges of **SCK** and are valid on the rising edges. Data is arranged MSB first, high byte first. The start of frame is indicated by the rising edge of **FSY**.

Figure 9-4 illustrates the data alignment and timing when the right-justified format is selected. For higher data bit rates, the channel data remains aligned with the edges of **FSY**, with more unused data bytes between channels.

FIGURE 9-4: RIGHT-JUSTIFIED STREAMING DATA FORMAT



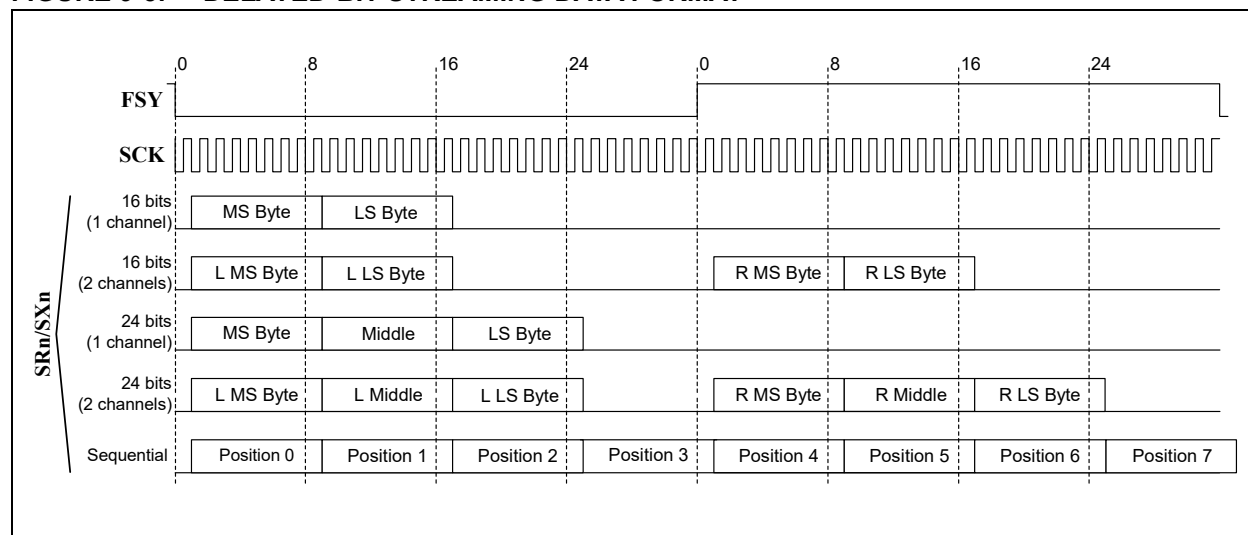
The right-justified format does not support a single (mono) channel when **SCK** is $128 \times F_s$. This mode is supported by the sequential format (see Section 9.4 “Sequential Alignment”).

9.3 Delayed-Bit Alignment

When using the delayed-bit alignment format, which is I²S compatible, there is a single **SCK** clock delay between the start of frame (falling edge of **FSY**) and the start of the frame data on **SRn/SXn**. The data and **FSY** signals change on the falling edges of **SCK** and are valid on the rising edges. Data is arranged MSB first, high byte first.

Figure 9-5 illustrates the data alignment and timing when the delayed-bit format is selected. For higher data bit rates, the channel data remains bit-delayed from the edges of **FSY**, with more unused data bytes between channels.

FIGURE 9-5: DELAYED-BIT STREAMING DATA FORMAT



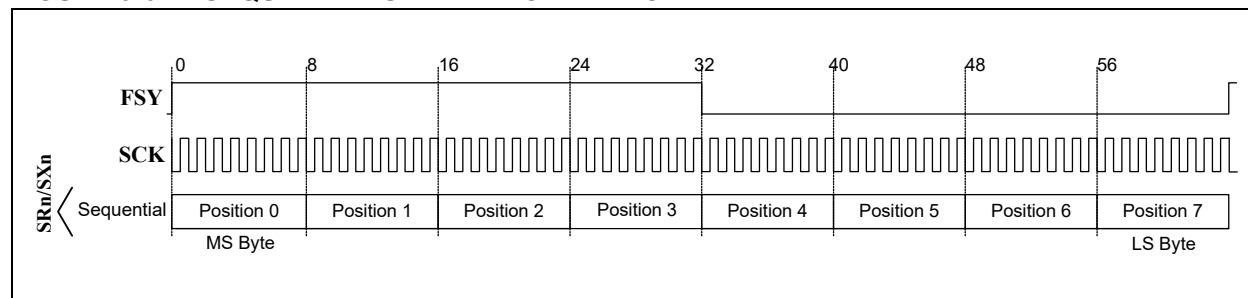
When **SCK** is 128×Fs or 256×Fs, a single (mono) channel is supported by the sequential, delayed-bit format (see Section 9.4 “Sequential Alignment”). When using the sequential, delayed-bit format, byte positions in a frame remain bit-delayed and fixed (as shown in Figure 9-5); however, the actual number of byte positions *used* in the frame is flexible. This allows sockets of various sizes to be connected.

9.4 Sequential Alignment

When using the sequential (non-delayed) alignment format, the data and **FSY** signals change on the falling edges of the clock and are valid on the rising edges. Data is arranged MSB first, high byte first. This format allows data bytes to exist at every byte position in the frame; however, the actual number of byte positions *used* is flexible. This allows a socket of custom size to be connected.

Figure 9-6 illustrates the byte positions and timing for the sequential (non-delayed) alignment format when **SCK** is 64×Fs. At higher **SCK** frequencies, more byte positions are available.

FIGURE 9-6: SEQUENTIAL STREAMING DATA FORMAT



Because the sequential alignment format does not place restrictions on socket size, this format can support a single (mono) channel. For example, with **SCK** at 128×Fs, a single 16 bit data channel could be sent in the following byte positions:

- Positions 0 and 1 (for left-justified alignment)
- Positions 6 and 7 (for right-justified alignment)

10.0 EXTERNAL POWER MANAGEMENT

The OS81092 INIC supports external power management logic by providing two inputs (**PS0** and **PS1**) and one output (**PWROFF**). The **PS0** and **PS1** inputs convey status of the external power management logic. The **PWROFF** output pin is driven low during normal operation and released when the OS81092 INIC is ready to be powered-off by the external power management logic. When using OS81092 Revision C1D (or later) devices, the functional response when INIC recognizes the various external power management states can be further adjusted as described in the [OS81092 INIC API User's Manual \[2\]](#).

Note: When the GPIO Port is enabled (only available on OS81092 Revision C1D or later devices), the **PS0** and **PS1** pins are unavailable for monitoring external power management status. Refer to [Chapter 12.0 “General Purpose Input/Output \(GPIO\) Port”](#) for more information.

The **PS0** and **PS1** inputs should indicate power management status, and communicate when an external power management event has occurred. [Table 10-1](#) describes the power management states that are defined by the [MOST Specification \[1\]](#) and should be communicated via **PS0** and **PS1**. More information on system-wide INIC power management architectures can be found in the [MOST INIC Hardware Concepts Specification \[8\]](#).

TABLE 10-1: EXTERNAL POWER MANAGEMENT STATES

PS1	PS0	Status
0	0	U _{Normal}
0	1	Switch-To-Power (STP)
1	0	U _{Critical} , U _{Super}
1	1	U _{Low}

Hardware recognizes an external power management state when **PS0** and **PS1** remain at a valid logic level for a minimum time, t_{pshold} (see [Section 15.4.2 “Reset and Power Management”](#)); however, the *INIC Software Stack* may use longer hold times for **PS0** and **PS1** states before responding to an external power management event. In addition, the *INIC Software Stack* controls how INIC responds to external power management events (e.g. releasing **PWROFF** and/or forcing network shutdown). If external power management is not used, **PS0** and **PS1** should be pulled to ground or power management handling should be disabled.

An STP event is recognized upon power-up (or reset) and on the transition to **PS[1:0] = 01**. If these pins continue to indicate an STP status (**PS[1:0] = 01**) after an STP event has already been recognized, the U_{Normal} state is assumed. An STP event can cause INIC to enter *Ring-Break Diagnosis* mode, based on the `RBDOptions()` function parameter settings. See the [OS81092 INIC API User's Manual \[2\]](#) for more information.

The `PMIConfig()` property is used to configure the behavior of the **PWROFF** pin based on the various `NetInterface` states (`NIState()`), `EHCI` states (`EHCIState()`) and the external power management states listed in [Table 10-1](#). Parameters optionally limit INIC from releasing **PWROFF** in specific `EHCI` states. Additionally, when using Revision C1D and later devices, parameters optionally limit INIC from releasing **PWROFF** when the U_{Low} state is recognized. The `PMIConfig()` property is set in the *INIC Configuration String* (see the [OS81092 INIC API User's Manual \[2\]](#) for more information).

In Revision C1C and earlier devices, the network output (**ETXP/ETXN**) is automatically switched off (and the **ERR/BOOT** pin is driven high) whenever the U_{Low} state is recognized. As long as the U_{Low} state is indicated on **PS[1:0]**, the **ETXP/ETXN** outputs remain disabled, regardless of activity on **ERXP/ERXN**. When using Revision C1D and later devices, parameters in the `PMIConfig()` property can be adjusted to keep the network output switched on when U_{Low} is recognized. Refer to the [OS81092 INIC API User's Manual \[2\]](#) for more information.

INIC supports implementation of a low-power mode through the use of *Device ShutDown*, which utilizes the `ShutDown()` function (in `FBlock NBMIN`), to place the node in a low-power state. When the EHC receives this function it should minimize power consumption, if supported. The network can then re-start the node through the `ShutDown()` function, where INIC drives **INT** low to wake the EHC.

OS81092

In a typical application, as illustrated in Figure 10-1, the power management circuitry is powered by a continuous supply and manages the switched power to the rest of the node. This circuitry also monitors the activity signal coming from the physical-layer interface (e.g. *NOACTIVITY* in Figure 10-1). When network activity is detected, the circuitry enables the switched supply and drives the *PS[1:0]* pins to the proper state, as depicted in Figure 10-2. When INIC powers up, it could take up to t_{stp} time for INIC to recognize the *PS[1:0]* state. When INIC is ready for shutdown, it releases the *PWROFF* pin (pulled high through an external resistor).

The EHC can also implement a *PWRHOLD* line that the power management circuitry detects. This signal is used to suspend node power-down until the EHC is ready. Refer to the [MOST INIC Hardware Concepts Specification \[8\]](#) for more information on typical power management architectures.

FIGURE 10-1: TYPICAL POWER MANAGEMENT DIAGRAM

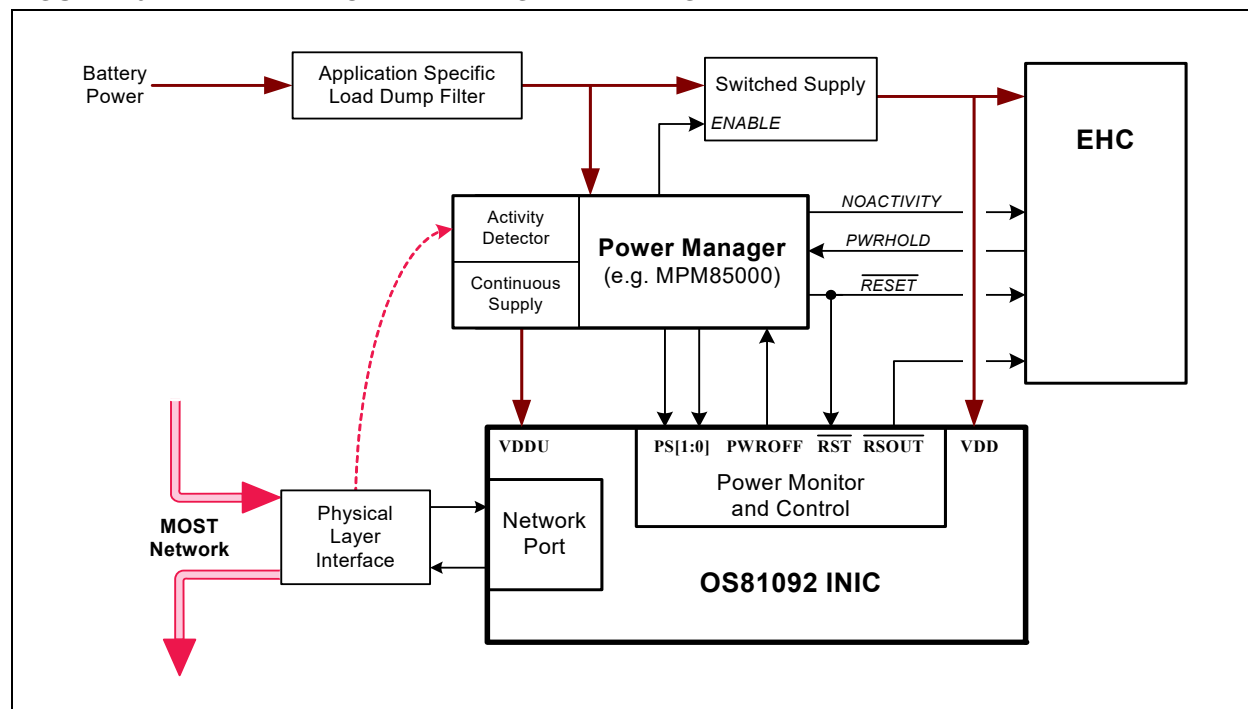
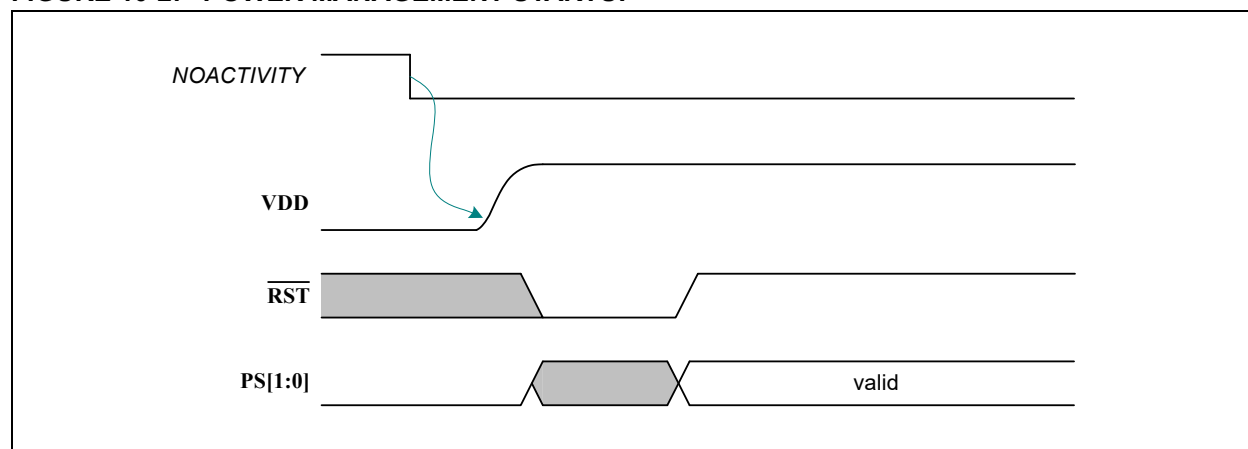


FIGURE 10-2: POWER MANAGEMENT STARTUP



Microchip's *Automotive Power Management Device* (MPM85000) performs all power management functionality required of MOST Network nodes, including: wake detection, power supply monitoring (*PS[1:0]*), reset generation, and optional *Switch-To-Power* (STP) pulse detection. The MPM85000 also supplies a 3.3 V continuous output which can be used to power *VDDU*. Integrating both the OS81092 and MPM85000 on a MOST Network node minimizes application circuitry and software complexity, allowing the application designer to focus on the application, rather than on MOST compliance. Contact Microchip for more information on the MPM85000.

11.0 CLOCK MANAGER

The Clock Manager generates internal clocks and optionally drives the **RMCK** external clock. The **RMCK** output clock supports synchronization of external devices to a clock multiple synchronous to the network frame rate.

11.1 PLL Lock Status

After power-up, the internal PLL locks to an external clock reference (**XTI/XTO**) and provides the primary timebase for the OS81092. Clock and data recovery is performed at the Network Port. For proper data and clock recovery of the incoming network bitstream, a valid reference clock must always be available.

When network lock is initially achieved, it takes three frames to synchronize streaming data. Therefore, streaming data will not be transferred properly between the network and any open source or sink ports for three frames when going from the unlock to lock state. While the PLL is unlocked, only control messages targeting the local FBlock **INIC** or **NBMIN** can be transferred (via the Control Port or MediaLB Port).

Network lock/unlock events can affect the behavior of the **ERR/BOOT** pin, based on the `MuteMode()` setting. For more information on the use of **ERR/BOOT** for lock/unlock detection, refer to the [OS81092 INIC API User's Manual \[2\]](#).

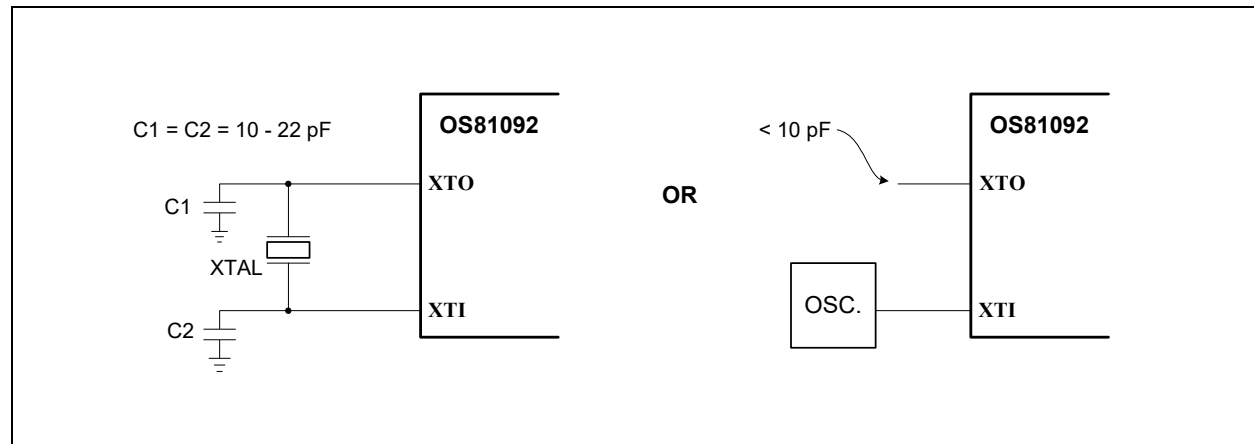
Note: A valid and continuous external reference clock (**XTI/XTO**) must be present for normal operation. If the reference clock is disabled during operation, INIC may become inoperable and/or the state of the **ERR/BOOT** pin may become invalid. A reset and valid reference clock is required to recover from this condition.

11.2 Crystal Pins (XTI/XTO)

The crystal oscillator should be in a fundamental, parallel resonant mode. [Figure 11-1](#) depicts the external circuitry connected to the OS81092 oscillator circuit. Since the internal inverter/amplifier is operated in its linear region, external series resistors should not be used, as they will lower the gain and could cause start-up problems. Several factors must be considered when selecting a crystal, including load capacitance, oscillator margin, cut, and operating temperature. For more information on crystals, see [Section 16.7 "Crystal Oscillator Selection"](#). The crystal frequency must be 384 times the desired operational network frequency (F_s).

If an external clock is used in lieu of a crystal oscillator, it must be connected to **XTI**. The clock must be stable prior to the rising edge of **RST** and remain stable for proper operation. In addition, **XTO** should float and have minimal capacitance (see [Figure 11-1](#)).

FIGURE 11-1: CRYSTAL OSCILLATOR INPUT



12.0 GENERAL PURPOSE INPUT/OUTPUT (GPIO) PORT

Note: The GPIO Port is only available when using OS81092 Revision C1D or later. Refer to [Appendix C: "Feature Matrix"](#) for more information.

The OS81092 allows certain pins to be reprogrammed from their default function to support general purpose input/output (GPIO) functionality. Each **GP_n** pin and its default function is listed in [Table 12-1](#). The user may enable the GPIO Port using the *INIC Configuration String*. See the [OS81092 INIC API User's Manual \[2\]](#) for more information.

TABLE 12-1: GPIO PIN AVAILABILITY

GPIO	Default Function	Notes
GP0	$\overline{\text{INT}}$	The Control Port is not available in slave mode when the GPIO Port is used.
GP3	TST1	This pin does not need to be tied to ground when the GPIO Port is used.
GP4	PS0	The INIC External Power Management functionality is not available when the GPIO Port is used.
GP5	PS1	
GP7	TST2	This pin does not need to be tied to ground when the GPIO Port is used.

12.1 GPIO Classes

GPIO pins are highly configurable which provides the user with the flexibility to support various applications. The following GPIO pin modes are available:

- *Output (Push-Pull)* - High Level or Low Level
- *Output (Open-Drain)* - default state Hi-Z or default state driven low. Trigger: Rising Edge, Falling Edge, High Level, Low Level.
- *Input* - Trigger: Rising Edge, Falling Edge, High Level, Low Level.
- *Sticky Input* - Trigger: Rising Edge, Falling Edge
- *Debounced Input* - Trigger: Rising Edge, Falling Edge, High Level, Low Level

At OS81092 power-up/reset, all GPIO pins are high-impedance inputs, except **GP7**, which is an *Open-Drain Output* configured to be Hi-Z by default. During GPIO initialization, the user specifies the pin configuration for each GPIO pin that is used and configures any other relevant details, such as debounce settings. Refer to [Chapter 15.0 "Electrical Characteristics"](#) for GPIO details such as voltage levels and timing. Further details for each GPIO mode are included in the following subsections and the [OS81092 INIC API User's Manual \[2\]](#).

12.1.1 PUSH-PULL OUTPUT

The *Push-Pull Output* mode provides a user-controlled output signal that is actively driven both high and low. The default output state of the pin can be configured either high or low level.

Because all GPIO pins are high-impedance inputs following power-up and reset, an external resistor to either ground or power is recommended to hold the pin at the desired initial state before the GPIO is configured as a *Push-Pull Output*.

12.1.2 OPEN-DRAIN OUTPUT

The *Open-Drain Output* mode provides a user-controlled output signal that is actively driven low and set to high-impedance during the logic high state. An external pull-up resistor is required to establish the logic high level when INIC releases the pin. This GPIO mode is useful for ganged signals for which more than one device can drive the line active (low).

Event notification (triggers) can also be configured on outputs similarly to inputs. For *Open-Drain Outputs*, this functionality is also used to notify the higher-level application that another device has driven the line active.

Because all GPIO pins are high-impedance inputs following power-up and reset, the required external pull-up resistor holds the pin at a logic high level before the GPIO is configured as an *Open-Drain Output*.

12.1.3 INPUT

A GPIO pin that is configured as an input can generate a notification (trigger) when an logic level (high-level or low-level) is detected or when a signal transition (edge) is detected.

12.1.4 STICKY INPUT

A *Sticky Input* allows the user to capture very brief pulses that may occur using dedicated hardware mechanisms. Once an input pulse has been captured, the condition remains active until cleared. The 'stuck' value is referred to as the sticky state and is independent of the actual state of the hardware pin.

The *Sticky Input* GPIO mode is appropriate for high speed inputs where active conditions may only exist for brief periods of time. Once the active condition has been captured, the input is 'stuck' and INIC does not report any further event conditions. Therefore, the captured state must be manually cleared by a higher-level application to re-enable the reporting of event conditions. The main advantage of a *Sticky Input* is the ability to capture narrow pulses that cannot be detected when using other GPIO modes.

Edge-based trigger events on *Sticky Input* pins remain valid as long as the input signal remains at the active level. Once the event is cleared, INIC will immediately report another event if the active level remains present on the GPIO pin. This behavior is similar to the *Level-Sensitive Input* GPIO mode; however, in the case of *Sticky Inputs*, this action may result in additional edge-based events being reported when no further edges have actually occurred.

A limitation of the *Sticky Input* mode is that the captured state must be manually cleared by a higher-level application in order to re-enable the reporting of subsequent event notifications. Another limitation is the additional complexities (described above) associated with handling an input event that is maintained at the active level rather than appearing as a narrow pulse.

12.1.5 DEBOUNCED INPUT

A *Debounced Input* is an *Input* that does not immediately report an event. Rather, the value on the pin is captured again after a period of time and compared to the initially captured value. If the values match, the signal is assumed to be stable (valid) and an event is reported. If the values do not match, the original edge event is considered non-qualified and is ignored.

The *Debounced Input* GPIO mode is useful for capturing events generated by mechanical switches that normally suffer from contact bounce. The time delay allows the signal to settle before reporting the event thereby only reporting a single, clean event to the monitoring application.

Refer to the [OS81092 INIC API User's Manual \[2\]](#) for more information on configuring a *Debounced Input*.

12.2 GPIO Triggers

12.2.1 LEVEL-SENSITIVE TRIGGERS

A *Level-Sensitive Trigger* is a trigger that monitors the state of the pin and generates an event notification when either high-level or low-level logic is detected. The event notification remains valid as long as the signal stays active (e.g. high or low, depending on configuration).

The *Level-Sensitive Trigger* is appropriate for signals where active levels are maintained until serviced. An advantage to this trigger mode is that subsequent event triggers are signaled once the controlling device re-enables event notifications, provided the active level has been maintained. This behavior is desirable in the following application scenarios:

- the peripheral attached to the OS81092 GPIO pin does not de-assert its hardware interrupt between events
- interrupt lines from multiple hardware peripherals are ganged together (e.g. using open-drain logic) onto a single OS81092 GPIO pin

A limitation of the *Level-Sensitive Trigger* is that a higher-level application must manually clear events in order to re-enable the reporting of subsequent event notifications.

12.2.2 EDGE-SENSITIVE TRIGGERS

An *Edge-Sensitive Trigger* generates an event notification when signal transitions are detected and are appropriate for detecting simple toggle events on an input signal. The advantage to using this GPIO mode is that subsequent signal transitions are immediately reported to the controlling application.

13.0 JTAG PORT

The OS81092 implements a standard 4-pin Test Access Port (TAP) interface (**TDI**, **TDO**, **TCK**, and **TMS**), as defined by the IEEE 1149.1 Test Access Port standard. The optional test-reset input signal (TRST) is not supported by INIC. Therefore, INIC should be reset through **RST** when a reset at the TAP interface is needed. This port can also be used for configuration and debug, as defined in [Section 16.6 “Configuration”](#).

The test-data input (**TDI**) and test-data output (**TDO**) pins provide the means of shifting data into and out of the OS81092 boundary-scan register (BSR). The TAP interface is controlled by the test clock (**TCK**) and the test-mode select (**TMS**) pins. When **TMS** is low, the OS81092 TAP interface is enabled and the rising edge of **TCK** clocks in data from **TMS** and **TDI**. After reset, the **TDO** pin is high-impedance. **TDO** is driven by the TAP controller only when in the *Shift-IR* or *Shift-DR* state.

For proper operation, all OS81092 TAP interface pins should be pulled high through resistors. A 4.7 k Ω pull-up resistor is recommended for **TCK**, **TDI**, and **TDO** per the IEEE 1149.1 standard. A 47 k Ω resistor is recommended for **TMS**.

For more information on the IEEE 1149.1 Test Access Port standard, see [IEEE Standard Test Access Port and Boundary-Scan Architecture \[9\]](#).

The OS81092 TAP interface supports all the mandatory boundary-scan instructions, as specified in the IEEE 1149.1 standard. Supported instructions include:

- **BYPASS** - Selects the bypass register (BYR) to be connected for serial access between **TDI** and **TDO**.
- **EXTEST** - Selects the boundary-scan register (BSR) to be connected for serial access between **TDI** and **TDO**.
- **SAMPLE/PRELOAD** - Captures a snapshot of the data at the digital pins during normal operation, and loads a data pattern into the boundary-scan register (BSR) prior to a new boundary-scan operation.
- **IDCODE** - (optional IEEE 1149.1 instruction) Used to read the Manufacturer ID from the identification register (IDR).

The OS81092 must be placed in *Boot Mode* before accessing the JTAG Port. In *Boot Mode*, the *INIC Software Stack* is disabled and the OS81092 waits for manual access through the JTAG Port. In this mode, standard communication channels (e.g. Control Port, MediaLB Port) are unavailable. The following sequence should be followed when using the OS81092 JTAG Port:

- Place the OS81092 in *Boot Mode*:
 - Hold the OS81092 in reset (drive **RST** pin low),
 - Hold the **ERR/BOOT** pin low,
 - Release the OS81092 from reset (**RST** pin high),
- Perform JTAG operations,
- Release the **ERR/BOOT** pin, and
- Reset the OS81092 to resume a normal mode of operation.

Note: The **ERR/BOOT** pin may be used in the boundary scan after the Configuration pin hold time (t_{cphrs}) given in [Section 15.4.2 “Reset and Power Management”](#) is met.

13.1 Instruction Register

The JTAG Instruction Register (IR) is accessed when the TAP controller is in the *Select-IR-scan*, *Capture-IR*, *Shift-IR*, *Exit1-IR*, *Pause-IR*, *Exit2-IR*, or *Update-IR* state.

REGISTER 13-1: JTAG INSTRUCTION REGISTER (IR)

Bit Range	Bit 3	Bit 2	Bit 1	Bit 0
3:0	IR[3]	IR[2]	IR[1]	IR[0]

The 4-bit JTAG Instruction Register (IR) contains the instruction code for the next operation. Supported instructions include:

bit 3-0	IR[3:0]: EXTEST: Instruction Code - Default: 0001
	0000 = SAMPLE/PRELOAD Enables EXTEST operation
	0001 = IDCODE Enables SAMPLE or PRELOAD operation
	0010 = BYPASS Enables shifting out of the Manufacturer ID
	1111 = Reserved Enables BYPASS
	All other codes Not supported

13.2 Identification Register

The JTAG Identification Register (IDR) is accessed when the TAP controller is in the *Select-DR-scan*, *Capture-DR*, *Shift-DR*, *Exit1-DR*, *Pause-DR*, *Exit2-DR*, or *Update-DR* state.

REGISTER 13-2: JTAG IDENTIFICATION REGISTER (IDR)

Bit Range	Bits 31/23/15/7	Bits 30/22/14/6	Bits 29/21/13/5	Bits 28/20/12/4	Bits 27/19/11/3	Bits 26/18/10/2	Bits 25/17/9/1	Bits 24/16/8/0
31:24	REV[0]	REV[1]	REV[2]	REV[3]	rsvd	rsvd	rsvd	rsvd
23:16	PID[0]	PID[1]	PID[2]	PID[3]	PID[4]	PID[5]	PID[6]	PID[7]
15:8	rsvd	rsvd	rsvd	rsvd	MID[0]	MID[1]	MID[2]	MID[3]
7:0	MID[4]	MID[5]	MID[6]	MID[7]	MID[8]	MID[9]	MID[10]	-

The JTAG IDR register is read out lsb first, bit 0 to bit 31. The bits below are listed in reverse order from the way they are read out.

bit 31-28	REV[3:0]: 4-bit Hardware Revision code †
	0001 = Revision A
	0010 = Revision B
	0011 = Revision C – The data sheet is applicable to this hardware revision.
bit 27-24	rsvd: Reserved - Default: 0000
bit 23-16	PID[7:0]: 8-bit Part Identifier - Default: 00011000
	82h = OS81092
bit 15-12	rsvd: Reserved - Default: 0000
bit 11-1	MID[10:0]: 11-bit Manufacturer ID - Default: 01000100010
	222h = Microchip Manufacturer ID
bit 0	Least significant bit (must be set per IEEE 1149.1 specification) - Default: 1
†	The default value of REV[3:0] is dependent on the actual hardware revision.

14.0 ELECTRICAL PHYSICAL (ePHY) INTERFACE

The OS81092 Electrical Physical (ePHY) interface supports electrical transmission of the network bit-stream between network devices, at a data rate up to 50 Mbps. An electrical MOST Network running at this data rate is referred to as a MOST50 Network. The ePHY interface is provided as an alternative to the traditional optical physical interface found in other MOST Network transceivers.

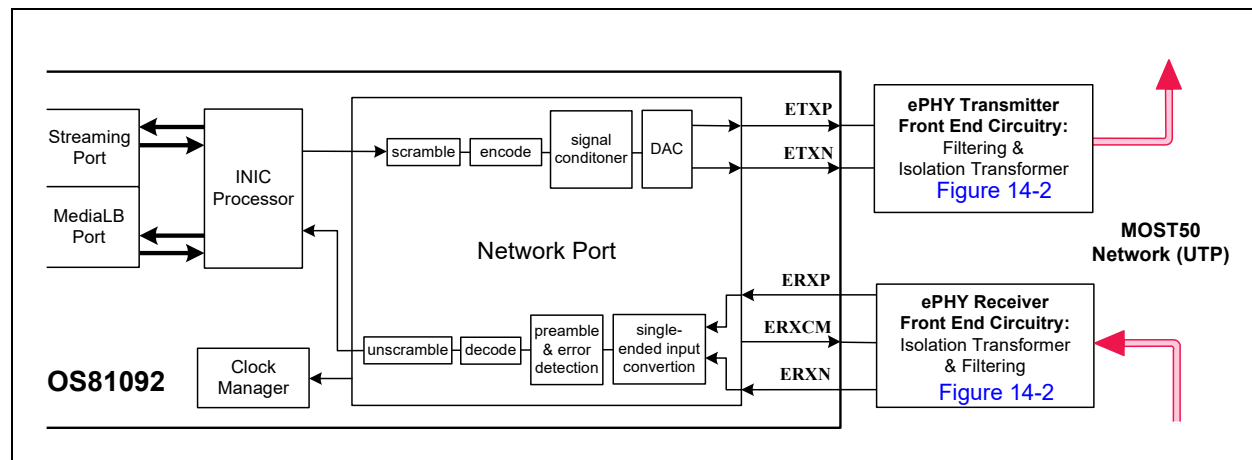
The ePHY interface consists of:

- **ETXP** - positive transmitter output
- **ETXN** - negative transmitter output
- **ERXP** - positive receiver input
- **ERXN** - negative receiver input
- **ERXCM** - common-mode set voltage

The ePHY system is a method of serial data communication between two or more compatible nodes using an electrical cable, rather than the traditional optical fiber. The OS81092 ePHY interface and the external ePHY Front End Circuitry described in this data sheet are optimized for use with unshielded twisted pair (UTP) wire.

Figure 14-1 illustrates the connection between the OS81092 ePHY interface and UTP MOST Network. A large portion of the Transmitter and Receiver Front End Circuitry is used to reduce electromagnetic emissions and increase the overall system immunity to EMI.

FIGURE 14-1: ePHY INTERFACE - BLOCK DIAGRAM



When transmitting data onto the network, the Network Port accepts data bytes from the INIC Processor, scrambles the data to reduce data-dependent jitter, conditions the signal for transmission over UTP, and transmits the encoded bit-stream onto **ETXP/ETXN**. These pins are driven by an internal DAC, which outputs a low-pass filtered version of the bit-stream by sourcing a total of 40 mA into external load resistors (R2 and R3 in Figure 14-2). The **ETXP/ETXN** outputs interface with the Transmitter Front End Circuitry, which provides an analog low-pass filter and a 1:1.5 isolation transformer that drives the UTP cable. The UTP cable has a nominal impedance of 130 Ω (+10 Ω /-30 Ω).

Network data at the receiver end is applied to a 1:1 isolation transformer (see the ePHY Receiver Front End Circuitry in Figure 14-2). The transformer increases the noise immunity by providing a high level of common-mode noise rejection. The transformer output feeds low-pass analog filters, which drive the **ERXP** and **ERXN** differential inputs. Resistors at these inputs are used to terminate the line with the cable's characteristic impedance (R201 and R202 in Figure 14-2). The **ERXCM** pin supplies a DC voltage (approximately 1.2 V) to the center of the resistors, in order to provide the differential input signal with the required common-mode voltage for data recovery.

When the OS81092 receives the serial network bit-stream at **ERXP/ERXN**, it converts the differential input to a single-ended signal. The Network Port detects the preamble in the received bit-stream, detects transmission errors, and works in conjunction with the Clock Manager to recover the clock from the bit-stream. The Network Port then decodes and unscrambles the bit-stream, and sends the network data to the INIC Processor as data bytes.

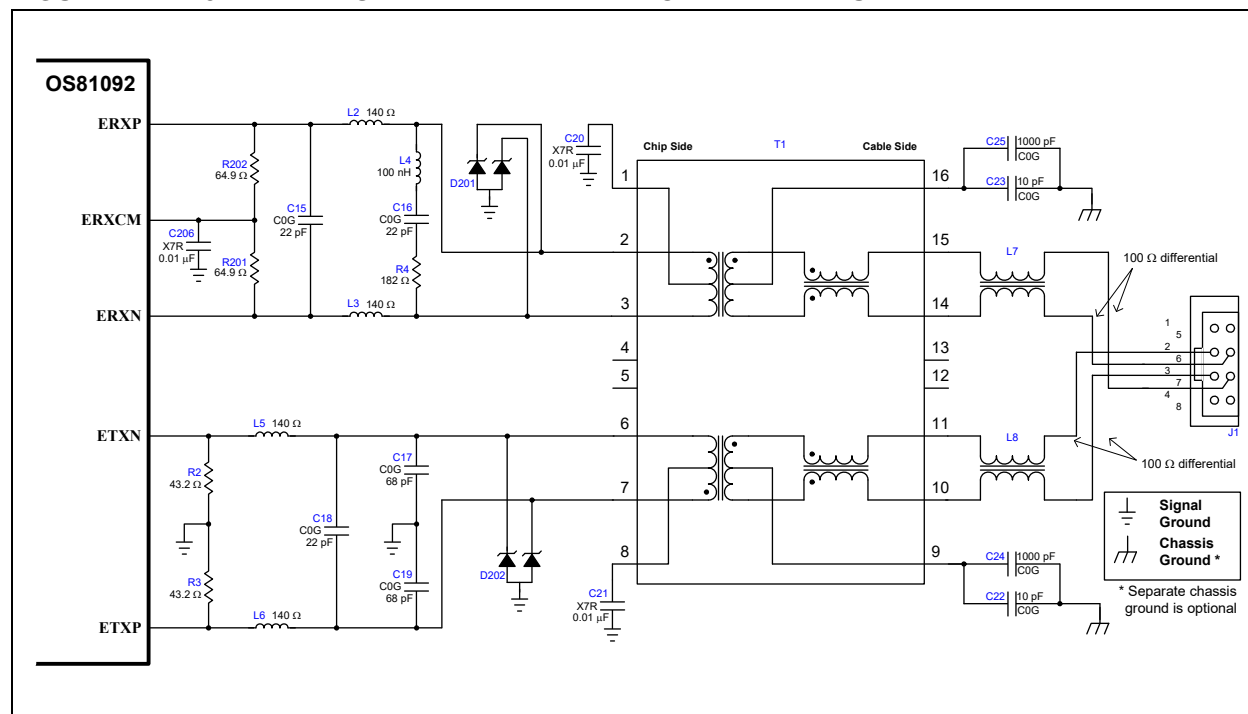
When the OS81092 is configured as a timing-slave, the Network Port is synchronized to the bit-stream recovered clock. When configured as the timing-master, the Network Port controls network timing, based on the external clock input. The transmitted network frames are aligned to this clock.

14.1 ePHY Front End Circuitry

In order for data transmission to be reliable over a wide operating range, all signals along the transmission path need to be relatively noise free, with minimal jitter and pulse width distortion. As a result, both the Transmitter and Receiver Front End Circuits are designed for reducing electrical emissions, increasing system immunity to EMI, and minimizing reflections.

Figure 14-2 shows the recommended Transmitter and Receiver Front End Circuitry at the ePHY interface. Circuit components used must comply with the guidelines provided in Section 16.8 “Physical Layer”. Use of an isolated chassis ground on the cable side of the transformer (as shown in Figure 14-2) is optional.

FIGURE 14-2: ePHY TRANSMITTER/RECEIVER FRONT END CIRCUITRY



14.1.1 TRANSMITTER FRONT END

The Transmitter Front End Circuitry in Figure 14-2 includes the following components: C17, C18, C19, C21, C22, C24, J1 (pins 2 and 3), L5, L6, L8, R2, R3, and T1 (bottom half).

The external load resistors on ETXP and ETXN (R2 and R3), in conjunction with the external load resistance reflected back through the transformer, convert the current loops into a differential voltage that is approximately 1.2 V peak-to-peak. Ferrite beads (L5 and L6), in conjunction with capacitors C17, C18, and C19, form a low-pass filter that removes out-of-band frequencies caused by the digitally switched current loops. C17 and C19 also help reduce common-mode noise. The transmission signal is passed through the 1:1.5 transformer (T1), with center tapped primary and secondary windings. The primary center tap is connected to signal ground through capacitor C21; the secondary center tap is connected to chassis ground through C22 and C24.

Transient voltage suppression devices (D202) are used for ESD protection on ETXP and ETXN. These devices are avalanche breakdown diodes that offer protection against high voltage, fast-edge transients. If further ESD protection is required to meet OEM specifications, an additional suppression device (not shown in Figure 14-2) may be placed on the chip-side transformer center tap, in parallel with C21.

These circuits minimize the common-mode noise that may be picked up by the cable in a harsh electrical environment and coupled into the system. Common-mode noise in both directions is also attenuated by a common-mode choke built into the transmit portion of the transformer and by an additional common-mode choke (L8) on the cable side of the transformer.

14.1.2 RECEIVER FRONT END

The Receiver Front End Circuitry in [Figure 14-2](#) includes the following components: C15, C16, C20, C23, C25, C206, J1 (pins 6 and 7), L2, L3, L4, L7, R4, R201, R202, and T1 (top half).

The differential receive signal on the cable (connected at J1), is passed through a common-mode choke (L7) and is applied to the 1:1 isolation transformer (T1). A choke internal to the transformer (on the cable side) blocks incoming common-mode noise. The center tap of the chip side of the transformer is connected to a capacitor (C20) which attenuates any residual common-mode noise passed through the transformer.

Transient voltage suppression devices (D201) are also used for ESD protection on **ERXP** and **ERXN**. If further ESD protection is required to meet OEM specifications, additional suppression devices (not shown in [Figure 14-2](#)) may be placed on the chip-side transformer center tap (in parallel with C20) and on the **ERXCM** pin (in parallel with C206).

The output of the transformer is low-pass filtered by C15, L2, and L3 before reaching **ERXP** and **ERXN**. This low-pass filtering causes the input impedance (looking into J1) to vary across the full frequency band of interest. As a result, shunt components C16, L4, and R4 act as a tuning circuit, designed to keep the input impedance constant over the applicable frequency range. This minimizes cable reflections and results in higher signal quality.

The receiver input is terminated with a pair of resistors R201 and R202, to match the cable impedance. These resistors are connected directly between **ERXCM** and **ERXN/ERXP**. A DC voltage of approximately 1.2 V is generated by the OS81092 at **ERXCM**, which sets the common-mode operating voltage of the received signal. This applied voltage makes the received signal compatible with the requirements of the differential reception circuitry.

15.0 ELECTRICAL CHARACTERISTICS

Specifications are subject to change without notice.

15.1 Absolute Maximum Ratings

TABLE 15-1: ABSOLUTE MAXIMUM RATINGS ELECTRICAL CHARACTERISTICS

Parameter ¹		Min	Max	Unit
Storage Temperature		-65	150	°C
Junction Temperature			150	°C
Power Supply Voltage:	Core (VDDCn) ²	-0.2	2.5	V
	Analog (VDDA18) ²	-0.2	2.5	V
	Analog (VDDA33) ³	-0.2	3.9	V
	ePHY (VDDEn)	-0.2	3.9	V
	Peripheral (VDDPn)	-0.2	3.9	V
	Continuous (VDDU)	-0.2	3.9	V
Power Dissipation: ⁴	Revision C1C and earlier		796	mW
	Revision C1D and later		843	mW
DC Current to any Pin Except Power ⁵			±10	mA
Maximum Input Voltage:				
XTI		-0.5	VDDPn+0.3	V
Other digital pins		-0.5	VDDPn+0.3	V

Note 1: Operation at or above these limits may damage the device.

2: The VDDCn and VDDA18 supplies must never be more than 0.5 V above the VDDEn, VDDPn, VDDA33, and VDDU supplies. This requirement applies to power-up, power-down, and normal operation.

3: The VDDA33 and VDDEn supplies must never be more than 0.5 V above the VDDU supply. This requirement applies to power-up, power-down, and normal operation.

4: Maximum power dissipation is calculated using the maximum power supply currents (listed in [Section 15.3 “DC Characteristics \(non-MediaLB pins\)”](#)) and the maximum power supply voltages under operating conditions (listed in [Section 15.2 “Guaranteed Operating Conditions”](#)).

5: Excludes the ETXP and ETXN pins, on which output current is specified in [Section 15.5 “ePHY Network Characteristics”](#).

15.2 Guaranteed Operating Conditions

TABLE 15-2: GUARANTEED OPERATING CONDITIONS ELECTRICAL CHARACTERISTICS

Parameter	Min	Max	Unit
Junction Temperature ¹	-40	125	°C
Power Supply Voltage:			
Core (VDDCn)	1.71	1.89	V
Analog (VDDA18)	1.71	1.89	V
Analog (VDDA33)	3.135	3.465	V
ePHY (VDDEn)	3.135	3.465	V
Peripheral (VDDPn)	3.135	3.465	V
Continuous (VDDU)	3.135	3.465	V
Voltage applied to pins:			
XTI	0	VDDPn	V
Other digital pins	0	VDDPn	V
Operating Network Frame Rate (Fs) ²	47.9	48.1	kHz
Digital Input Transition Rise/Fall rate ^{3,4}		10	ns/V

Note 1: Write operations to OTP memory are guaranteed from 0 to 125 °C.

2: The *targeted* operating network frame rate (Fs) is 48 kHz.

3: Measured between V_{IL} and V_{IH}, unless otherwise noted.

4: Excludes digital input pins with transition times explicitly noted elsewhere in this chapter, as well as the RST pin, on which input hysteresis is specified ([Section 15.4.2](#)).

15.3 DC Characteristics (non-MediaLB pins)

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$;
 $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; Load Capacitance = 30 pF ; PLL locked at 48 kHz ; unless otherwise noted.

TABLE 15-3: DC ELECTRICAL CHARACTERISTICS (OTHER THAN MediaLB)

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Low-Level Input Voltage: XTI ¹ All other digital input pins	V_{IL}			0.75 0.8	V V	
High-Level Input Voltage: XTI ¹ All other digital input pins	V_{IH}	2.0 2.0			V V	
Input Leakage Current	I_L			± 10	μA	$0 < V_{in} < V_{DDPn}$
Low-Level Output Voltage	V_{OL}			0.4	V	$I_{OL} = 4\text{ mA}$
High-Level Output Voltage	V_{OH}	$V_{DDPn} - 0.4$			V	$I_{OH} = -4\text{ mA}$
Digital Input Pin Capacitance				10	pF	
Analog Supply Current: VDDA18 VDDA33 VDDEn	I_{AC} I_{AP} I_E			30 25 65	mA mA mA	$F_s = 48\text{ kHz}$ Outputs unloaded
Digital Supply Current: VDDCn - Revision C1C and earlier VDDCn - Revision C1D and later VDDPn	I_{DDC} I_{DDC} I_{DDP}			125 150 55	mA mA mA	$F_s = 48\text{ kHz}$ Outputs unloaded
Continuous Supply (VDDU) Current: Active Mode Sleep Mode ²	I_{UA} I_{US}			90 6	μA μA	

Note 1: Applicable only when an external clock driver is used.

2: When all other power supplies are off.

15.4 AC Characteristics (non-MediaLB pins)

15.4.1 CLOCKS

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$; $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; Load Capacitance = 30 pF ; PLL locked at 48 kHz ; unless otherwise noted.

TABLE 15-4: AC ELECTRICAL CHARACTERISTICS (OTHER THAN MediaLB)

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Output Rise Time: ERR, TDO RMCK, SX0, SX1, FSY, SCK GP0, GP3, GP4, GP5, GP7	t_{hdr}			12 5 5	ns ns ns	V_{OL} to V_{OH}
Output Fall Time: ERR, SDA, SCL, INT, RSOUT TDO, DSDA, DSCL, PWROFF RMCK, SX0, SX1, FSY, SCK GP0, GP3, GP4, GP5, GP7	t_{hdf}			12 12 5 5	ns ns ns ns	V_{OH} to V_{OL}
GPIO Debounce Time Delay	t_{dbdelay}	1		65,535	ms	
GPIO Debounce Time Resolution	t_{bdres}		1		ms	
GPIO Minimum Pulse Width (Normal Input)	$t_{\text{minpwnrml}}$	200			μs	
GPIO Minimum Pulse Width (Sticky Input)	$t_{\text{minpwstky}}$	60			ns	
Clocks:						
RMCK Master Clock Output	f_{rmck}	18,000		$1536 \times F_s$	Hz	when enabled
External clock/crystal frequency ¹	f_{osc}		$384 \times F_s$		Hz	
External clock/crystal deviation ¹	Δf_{osc}			± 200	ppm	
Jitter Tolerance (timing-master): XTI ²	t_{jm}	0.8			ns (pp)	

Note 1: The *MOST Specification* requires each node have a crystal-based master-clock source to support *Ring-Break Diagnosis* mode, where nodes normally configured as timing-slaves can be re-configured as the timing-master.

2: Jitter Tolerance: The OS81092 tolerates *at least* the amount of clock jitter defined by the t_{jm} (minimum) specification without errors.

15.4.2 RESET AND POWER MANAGEMENT

$T_J = -40$ to 125 °C; $V_{DDCn}, V_{DDA18} = 1.8$ V ± 5 %; $V_{DDA33}, V_{DDEn}, V_{DDPh}, V_{DDU} = 3.3$ V ± 5 %; $GNDn, GNDAn, GNDE, GNDU = 0.0$ V; Load Capacitance = 30 pF; PLL locked at 48 kHz; unless otherwise noted.

TABLE 15-5: RESET AND POWER MANAGEMENT ELECTRICAL CHARACTERISTICS

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
PS[1:0] transition time ^{1, 2, 3}	$t_{pstrans}$			0.75	ms	
PS[1:0] hold time	t_{pshold}	2.2			ms	
$\overline{RST}$ rising to PS[1:0] recognition	t_{stp}	5		100	ms	
$\overline{RST}$ input hysteresis	$\Delta V_{hys-rst}$	200			mV	
$\overline{RST}$ pulse width ^{4, 5, 6}	t_{rspw}	150			ns	
Configuration pin setup to $\overline{RST}$ rising: ^{4, 5, 6} $\overline{INT}$ $\overline{ERR/BOOT}$	t_{cpsrs}	0 0			ns ns	
Configuration pin hold from $\overline{RST}$ rising: ^{4, 5, 6} $\overline{INT}$ $\overline{ERR/BOOT}$	t_{cphrs}	25 2.5			ms ms	
$\overline{RST}$ rising to ETXP/ETXN output enabled ⁷	t_{inicit}		20	30	ms	
$\overline{RST}$ rising to PWROFF asserted low	t_{pal}		10	100	μ s	
Activity on ERXP/ERXN input to ETXP/ETXN output enabled ^{7, 8}	$t_{NtwStartup}$		1.2	6	ms	
Loss of activity on ERXP/ERXN input to ETXP/ETXN output disabled ⁹	$t_{NtwShutdown}$		0.5	2	ms	

Note 1: Maximum time allowed by hardware for PS[1:0] to settle. Longer transition times may cause hardware to recognize an erroneous power management state.

2: Minimum time required for an external power management state (PS[1:0]) to be recognized by hardware. The INIC Software Stack may define a longer hold time for responding to an external power event.

3: Immediately following power-up (or reset), the PS[1:0] state is recognized after t_{stp} . This requires t_{pshold} be at least $t_{stp}(\max)$ after power-up (or reset).

4: After both the 3.3 V and 1.8 V power supplies have stabilized (excluding VDDU), INIC must be reset for normal operation. A proper reset is when both supplies are ≥ 90 % VDD; however, electrical specifications are applicable only within ± 5 % VDD.

5: Power-up configuration pins include: $\overline{ERR/BOOT}$ and $\overline{INT}$; however, $\overline{INT}$ is only used as a configuration pin when `PortConfiguration.DefPort()` is set to `INT_Select`. When used as a configuration pin, an external component (e.g. resistor) must hold $\overline{INT}$ in a valid state (high or low) for at least t_{cphrs} or until $\overline{INT}$ is driven low by INIC, whichever occurs first.

6: The $\overline{RST}$ pulse width indicates the minimum time required to reset the part; however, the configuration pins can take longer to settle to their default state, based on the trace capacitance and size of the on-board pull-up or pull-down resistor. Therefore, the $\overline{RST}$ pulse width must be long enough to allow the external configuration pins to achieve their default state.

7: When INIC's ETXP/ETXN output is enabled, the INIC transceiver becomes *visible* and INIC enters the network (e.g. obtains a node address).

8: The $t_{NtwStartup}$ parameter is only valid if t_{inicit} has elapsed, otherwise the time from activity on ERXP/ERXN until ETXP/ETXN enabled is determined by t_{inicit} . $t_{NtwStartup}$ also assumes activity has been absent for at least the minimum $t_{Restart}$ time (refer to the [MOST Specification \[1\]](#)). The $t_{NtwStartup}$ parameter represents the INIC component of the *MOST Specification* t_{WakeUp} parameter.

9: The $t_{NtwShutdown}$ parameter assumes t_{inicit} has previously expired. $t_{NtwShutdown}$ represents the INIC component of the *MOST Specification* $t_{ShutDown}$ parameter.

FIGURE 15-1: POWER-UP AND RESET TIMING

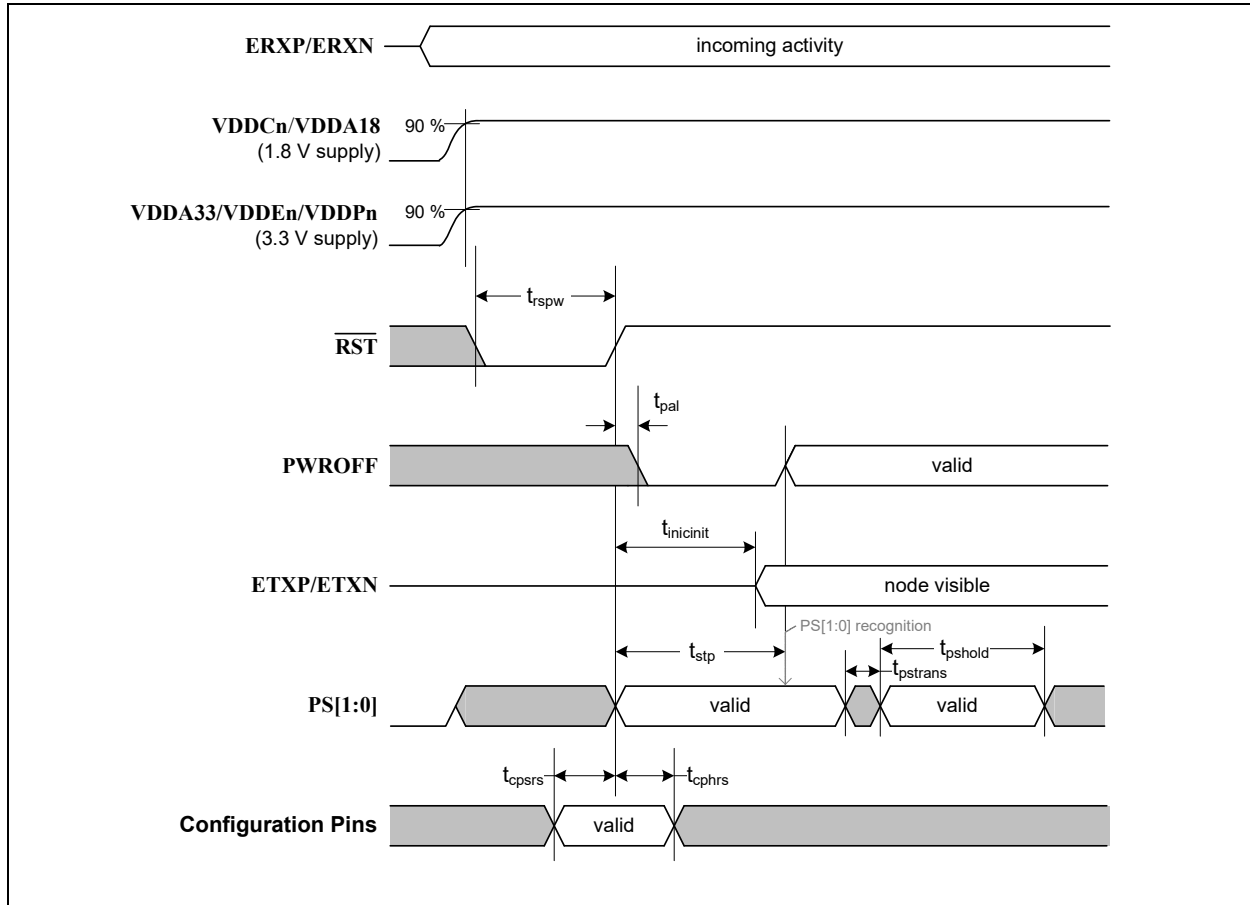
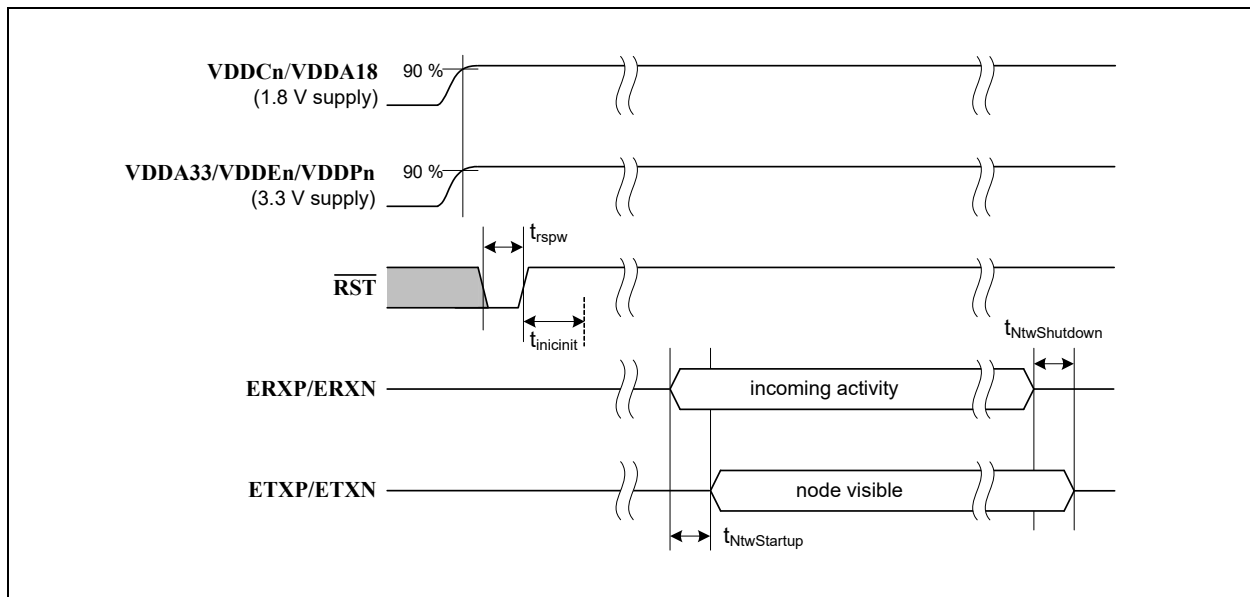


FIGURE 15-2: NETWORK STARTUP AND SHUTDOWN TIMING



15.5 ePHY Network Characteristics

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPh}, V_{DDU} = 3.3\text{ V} \pm 5\%$; $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; unless otherwise noted.

TABLE 15-6: ePHY NETWORK ELECTRICAL CHARACTERISTICS

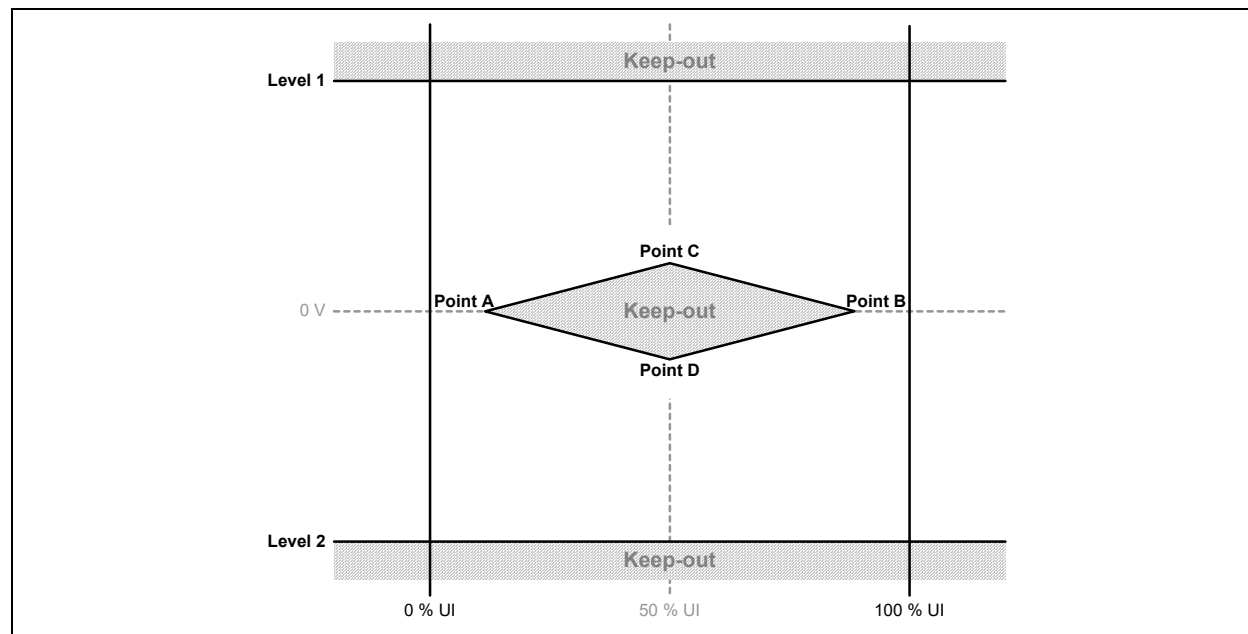
Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Output drive current	I_L		36		mA	
ETX Common-Mode output voltage	V_{CMT}		0.85		V	at center tap
ERX Common-Mode input voltage	V_{CMR}	1.00		1.70	V	
ERXCM output voltage	V_{OCM}	1.05		1.65	V	

Note: Test setup for ePHY electrical specifications was performed in accordance with the [MOST Specification of Electrical Physical Layer \[10\]](#).

15.5.1 ETXP/ETXN - EYE MASK PATTERN

The eye mask pattern in [Figure 15-3](#) defines the minimum transmit eye opening area and maximum transmit signal amplitude for ETXP/ETXN. The OS81092 ETXP/ETXN differential signal that is transmitted does not intrude on any of the keep-out areas defined for this eye mask.

FIGURE 15-3: ETXP/ETXN EYE MASK PATTERN



[Table 15-7](#) defines the boundaries of the ETXP/ETXN eye mask shown in [Figure 15-3](#).

TABLE 15-7: ETXP/ETXN EYE MASK BOUNDARIES

Parameter	Voltage	Time
Point A	0.00 V	11.7 % UI
Point B	0.00 V	88.3 % UI
Point C	+0.20 V	50 % UI
Point D	-0.20 V	50 % UI
Level 1	+1.25 V	(N/A)
Level 2	-1.25 V	(N/A)

15.6 ERXP/ERN - Eye Mask Pattern

The OS81092 ERXP/ERN receiver will tolerate an input signal that runs up to the keep-out boundaries of the eye mask pattern in Figure 15-4. A differential signal received at ERXP/ERN that fills the eye mask (up to the keep-out boundary) represents the worst-case signal that is tolerable by the OS81092 receiver.

FIGURE 15-4: ERXP/ERN EYE MASK PATTERN

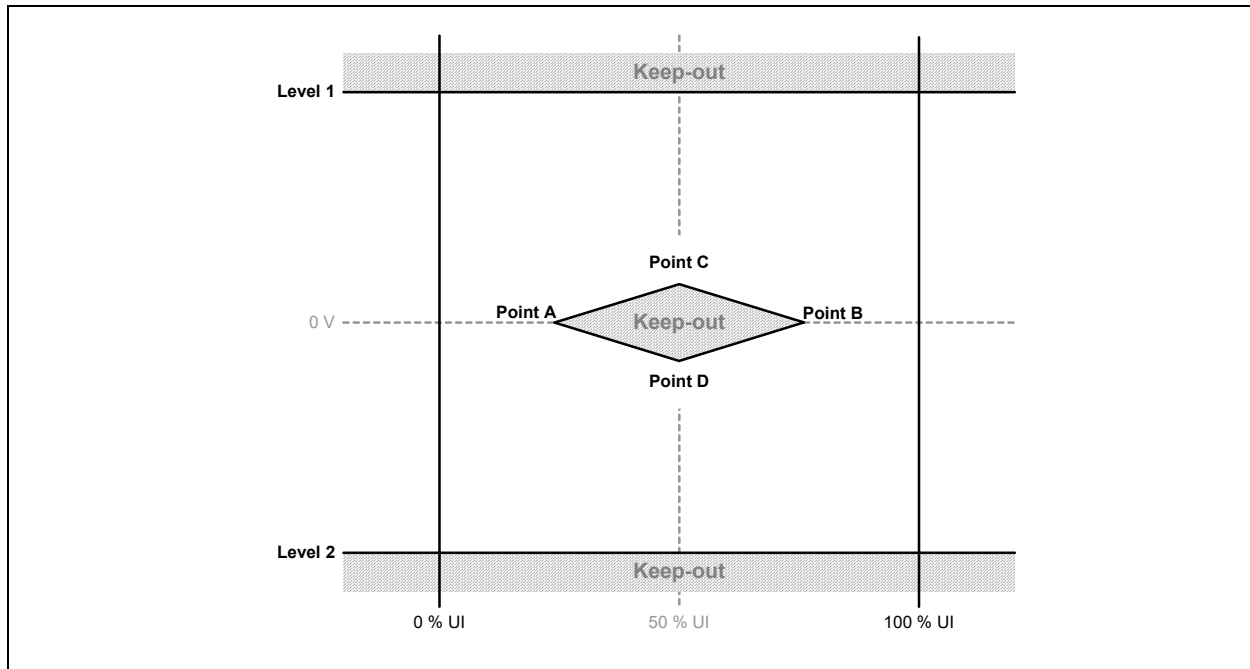


Table 15-8 defines the boundaries of the ERXP/ERN eye mask shown in Figure 15-4.

TABLE 15-8: ERXP/ERN EYE MASK BOUNDARIES

Parameter	Voltage	Time
Point A	0.00 V	23.5 % UI
Point B	0.00 V	76.5 % UI
Point C	+0.15 V	50 % UI
Point D	-0.15 V	50 % UI
Level 1	+1.25 V	(N/A)
Level 2	-1.25 V	(N/A)

15.7 MediaLB Port (3-pin)

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$;
 $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; Load Capacitance = 60 pF ; **MediaLB Speed of $256 \times F_s$ or $512 \times F_s$** ;
 PLL locked at 48 kHz ; unless otherwise noted.

TABLE 15-9: MediaLB PORT ELECTRICAL CHARACTERISTICS (3-Pin, $256 \times F_s$ or $512 \times F_s$)

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Low-Level Input Voltage	V_{IL}			0.7	V	
High-Level Input Voltage	V_{IH}	1.8			V	
Input Leakage Current	I_L			± 1	μA	$0 < V_{in} < V_{DDPn}$
Low-Level Output Voltage	V_{OL}			0.4	V	$I_{OL} = 8\text{ mA}$
High-Level Output Voltage	V_{OH}	2.4			V	$I_{OH} = -8\text{ mA}$
MLBCLK Operating Frequency ^{1, 2}	f_{mck}	12.2624	12.288	12.3136 12.5952	MHz MHz	$256 \times F_s$ $256 \times F_s$ PLL unlocked
		24.5248	24.576	24.6272 25.1904	MHz MHz	$512 \times F_s$ $512 \times F_s$ PLL unlocked
MLBCLK rise time	t_{mckr}			3	ns	V_{IL} to V_{IH}
MLBCLK fall time	t_{mckf}			3	ns	V_{IH} to V_{IL}
MLBCLK cycle time	t_{mckc}		81 40		ns ns	$256 \times F_s$ $512 \times F_s$
MLBCLK low time	t_{mckl}	30 14	35.5 16.5		ns ns	$256 \times F_s$ $512 \times F_s$
MLBCLK high time	t_{mckh}	30 14	36.5 16.5		ns ns	
MLBSIG/MLBDAT receiver input valid to MLBCLK falling	t_{dsmcf}	1			ns	
MLBSIG/MLBDAT receiver input hold from MLBCLK low	t_{dhmcf}	t_{mdzh}			ns	
MLBSIG/MLBDAT output high impedance from MLBCLK low ³	t_{mcfdz}	0		t_{mckl}	ns	
Bus hold from MLBCLK low ³	t_{mdzh}	4			ns	
MLBSIG/MLBDAT output valid from MLBCLK high	t_{mcdrv}			8	ns	

Note 1: The OS81092 can shut off MLBCLK to place MediaLB in a low-power state.

2: When the PLL is locked, MLBCLK operating frequencies are based on a 48.0 kHz *targeted* operating network frame rate (F_s) with $\pm 0.1\text{ kHz}$ variation. During PLL unlock, F_s may drift up to 49.2 kHz .

3: The MediaLB driver can release MLBSIG/MLBDAT (e.g. high-impedance) as soon as MLBCLK is low; however, the logic state of the final driven bit on the line must remain on the bus for t_{mdzh} . Therefore, coupling must be minimized while meeting the maximum capacitive load listed.

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$;
 $G_{NDn}, G_{NDAn}, G_{NDE}, G_{NDU} = 0.0\text{ V}$; Load Capacitance = 40 pF ; **MediaLB Speed of $1024 \times F_s$** ;
 PLL locked at 48 kHz ; unless otherwise noted.

TABLE 15-10: MediaLB PORT ELECTRICAL CHARACTERISTICS (3-Pin, $1024 \times F_s$)

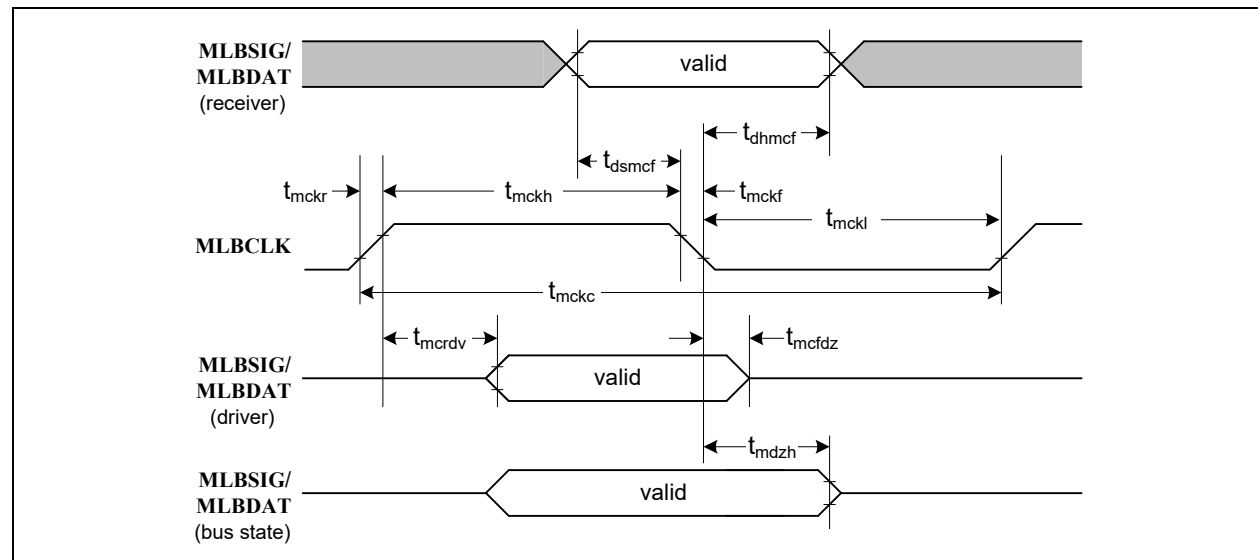
Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Low-Level Input Voltage	V_{IL}			0.7	V	
High-Level Input Voltage	V_{IH}	1.8			V	
Input Leakage Current	I_L			± 1	μA	$0 < V_{in} < V_{DDPn}$
Low-Level Output Voltage	V_{OL}			0.4	V	$I_{OL} = 8\text{ mA}$
High-Level Output Voltage	V_{OH}	2.4			V	$I_{OH} = -8\text{ mA}$
MLBCLK Operating Frequency ^{1, 2}	f_{mck}	49.0496	49.152	49.2544 50.3808	MHz MHz	$1024 \times F_s$ $1024 \times F_s$ PLL unlocked
MLBCLK rise time	t_{mckr}			1	ns	V_{IL} to V_{IH}
MLBCLK fall time	t_{mckf}			1	ns	V_{IH} to V_{IL}
MLBCLK cycle time	t_{mckc}		20.3		ns	
MLBCLK low time	t_{mckl}	6.1	7.3		ns	
MLBCLK high time	t_{mckh}	9.3	10.2		ns	
MLBSIG/MLBDAT receiver input valid to MLBCLK falling	t_{dsmcf}	1			ns	
MLBSIG/MLBDAT receiver input hold from MLBCLK low	t_{dhmcf}	t_{mdzh}			ns	
MLBSIG/MLBDAT output high impedance from MLBCLK low ³	t_{mcfdz}	0		t_{mckl}	ns	
Bus hold from MLBCLK low ³	t_{mdzh}	2			ns	
MLBSIG/MLBDAT output valid from MLBCLK high	t_{mcrdv}			7	ns	

Note 1: The OS81092 can shut off MLBCLK to place MediaLB in a low-power state.

2: When the PLL is locked, MLBCLK operating frequencies are based on a 48.0 kHz *targeted* operating network frame rate (F_s) with $\pm 0.1\text{ kHz}$ variation. During PLL unlock, F_s may drift up to 49.2 kHz .

3: The MediaLB driver can release MLBSIG/MLBDAT (e.g. high-impedance) as soon as MLBCLK is low; however, the logic state of the final driven bit on the line must remain on the bus for t_{mdzh} . Therefore, coupling must be minimized while meeting the maximum capacitive load listed.

FIGURE 15-5: MediaLB 3-PIN TIMING



15.8 MediaLB Port (5-pin)

$T_J = -40$ to $125\text{ }^\circ\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$;
 $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; Load Capacitance = 40 pF ; **MediaLB Speed of $256 \times F_s$ or $512 \times F_s$** ;
 PLL locked at 48 kHz ; unless otherwise noted.

TABLE 15-11: MediaLB PORT ELECTRICAL CHARACTERISTICS (5-Pin, $256 \times F_s$ or $512 \times F_s$)

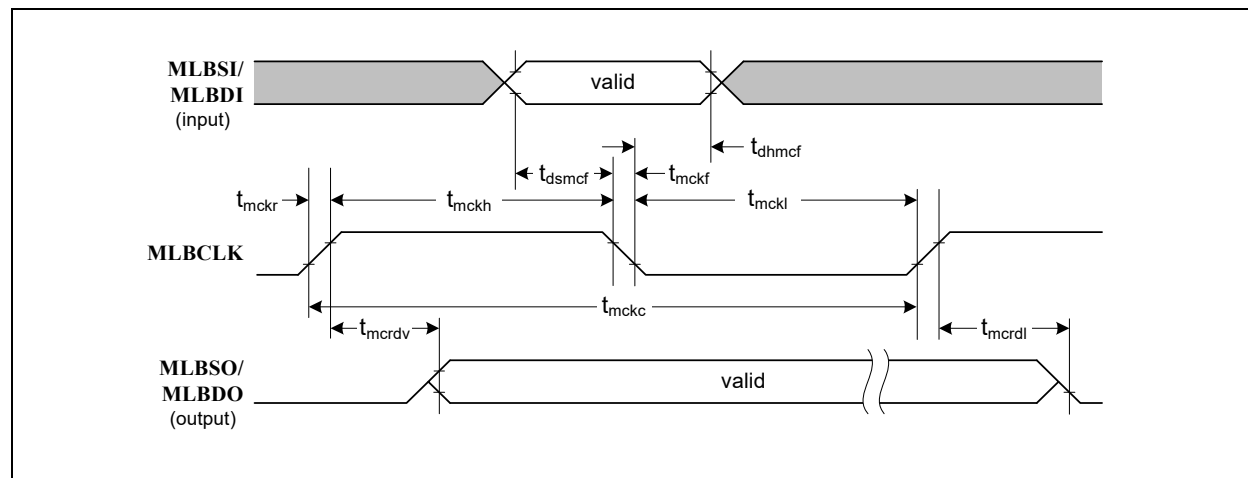
Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Low-Level Input Voltage	V_{IL}			0.7	V	
High-Level Input Voltage	V_{IH}	1.8			V	
Input Leakage Current	I_L			± 1	μA	$0 < V_{in} < V_{DDPn}$
Low-Level Output Voltage	V_{OL}			0.4	V	$I_{OL} = 8\text{ mA}$
High-Level Output Voltage	V_{OH}	2.4			V	$I_{OH} = -8\text{ mA}$
MLBCLK Operating Frequency ^{1, 2}	f_{mck}	12.2624	12.288	12.3136 12.5952	MHz MHz	$256 \times F_s$ $256 \times F_s$ PLL unlocked
		24.5248	24.576	24.6272 25.1904	MHz MHz	$512 \times F_s$ $512 \times F_s$ PLL unlocked
MLBCLK rise time	t_{mckr}			3	ns	V_{IL} to V_{IH}
MLBCLK fall time	t_{mckf}			3	ns	V_{IH} to V_{IL}
MLBCLK cycle time	t_{mckc}		81		ns	$256 \times F_s$
			40		ns	$512 \times F_s$
MLBCLK low time	t_{mckl}	30	35.5		ns	$256 \times F_s$
		15	16.5		ns	$512 \times F_s$
MLBCLK high time	t_{mckh}	30	36.5		ns	$256 \times F_s$
		15	16.5		ns	$512 \times F_s$
MLBSI/MLBDI valid to MLBCLK falling ³	t_{dsmcf}	3			ns	
MLBSI/MLBDI hold from MLBCLK low	t_{dhmcf}	5			ns	
MLBSO/MLBDO low from MLBCLK high	t_{mcrdl}			20	ns	$256 \times F_s$
				10	ns	$512 \times F_s$
MLBSO/MLBDO valid from MLBCLK high	t_{mcrdv}			8	ns	

Note 1: The OS81092 can shut off MLBCLK to place MediaLB in a low-power state.

2: When the PLL is locked, MLBCLK operating frequencies are based on a 48.0 kHz *targeted* operating network frame rate (F_s) with $\pm 0.1\text{ kHz}$ variation. During PLL unlock, F_s may drift up to 49.2 kHz .

3: Gate delays due to OR'ing logic on the pins must be taken into consideration.

FIGURE 15-6: MediaLB 5-PIN TIMING



15.9 Streaming Port

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$;
 $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; Load Capacitance = 30 pF ; PLL locked at 48 kHz ; unless otherwise noted.

TABLE 15-12: STREAMING PORT ELECTRICAL CHARACTERISTICS

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
FSY frequency ¹	f_{fsy}	47.9	48	48.1	kHz	
SCK frequency ¹	f_{sck}	$8 \times F_s$		$256 \times F_s$	Hz	
SCK low time	t_{sckl}	50 35			ns ns	all other speed modes $256 \times F_s$
SCK high time	t_{sckh}	50 35			ns ns	all other speed modes $256 \times F_s$
SR[1:0] valid to SCK rising ^{2, 4}	t_{srs}	12			ns	
SR[1:0] hold from SCK rising ^{2, 4} Master Mode Slave Mode	t_{srh}	10 10			ns ns	
SX[1:0] valid from SCK falling ^{2, 4}	t_{sxv}			20	ns	
SCK and FSY Outputs (Master Mode):						
SCK duty cycle	t_{sckdc}		50		%	
SCK falling to FSY valid ^{2, 3}	t_{fsyv}	-5		5	ns	
SCK and FSY Inputs (Slave Mode):						
FSY valid to SCK rising ^{2, 3}	t_{fsys}	5			ns	
FSY hold from SCK rising ^{2, 3}	t_{fsyh}	5			ns	

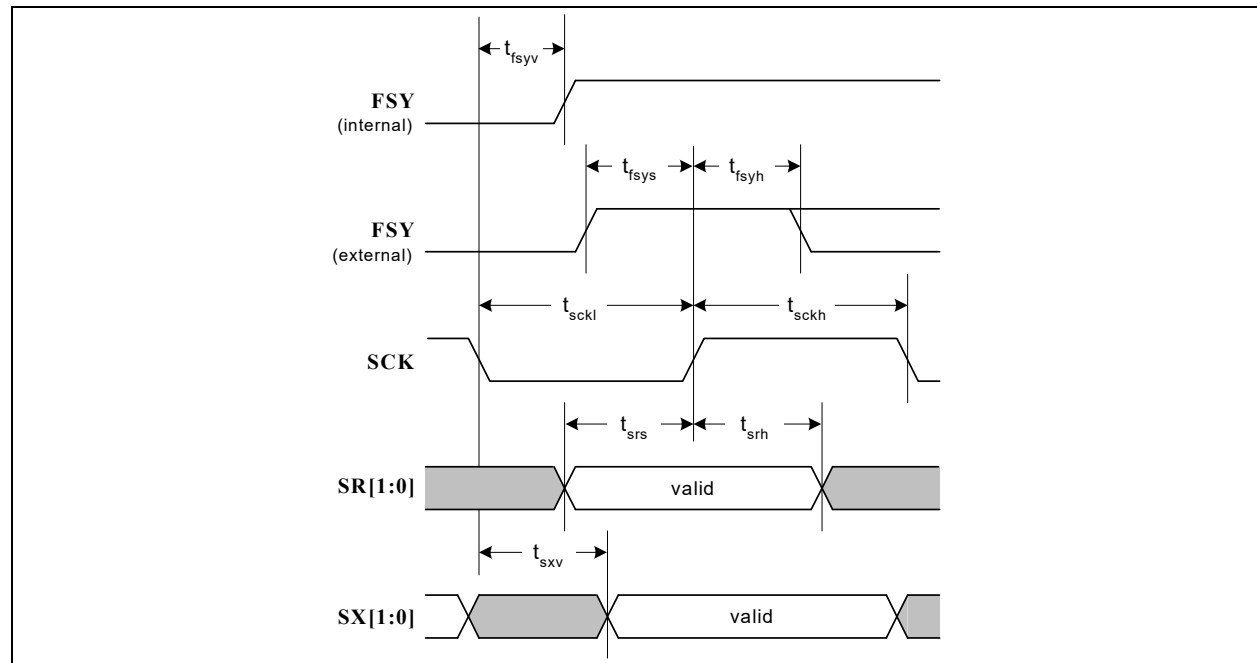
Note 1: If SCK and FSY are inputs, they must be frequency locked to the master clock (RMCK output clock).

2: The SCK edge at which data is stable and not changing is the rising edge for all supported formats.

3: FSY polarity is determined by the data format selected when the SCM `OpenPort()` function is called.

4: The MSB of SRn/SXn is the first or the second bit after FSY changes, based on the format selected.

FIGURE 15-7: STREAMING PORT TIMING



15.10 Control Port

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$;
 $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; Load Capacitance = 30 pF ; PLL unlocked; unless otherwise noted.

TABLE 15-13: CONTROL PORT ELECTRICAL CHARACTERISTICS

Parameter	Symbol	Min	Max	Unit
Slave Mode				
SCL frequency	f_{scl}		400	kHz
Bus free between transmissions (SDA high time between start and stop)	t_{buf}	1.3		μs
Start condition hold time (SDA falling to SCL falling)	t_{stah}	0.6		μs
SCL low	t_{scll}	1.3		μs
SCL high	t_{sclh}	0.6		μs
SDA input hold from SCL falling	t_{sdah}	0	900	ns
SDA output valid from SCL falling	t_{ov}		900	ns
SDA input valid to SCL rising	t_{sdas}	100		ns
(repeated) start condition setup time	t_{stas}	0.6		μs
SDA and SCL rise time ¹	t_r		300	ns
SDA and SCL fall time ¹	t_f		300	ns
Stop condition setup time (SCL rising to SDA rising)	t_{stps}	0.6		μs

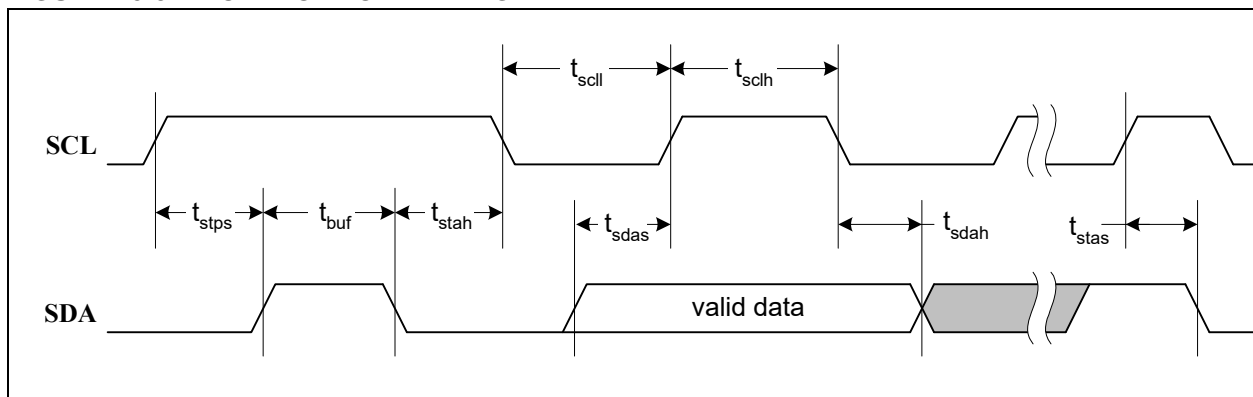
Note 1: Rise and fall time specifications are based on the V_{IL} and V_{IH} values given in the [I²C-Bus Specification \[5\]](#) and not the V_{IL} and V_{IH} specifications given in [Table 15-3](#).

TABLE 15-13: CONTROL PORT ELECTRICAL CHARACTERISTICS (CONTINUED)

Parameter	Symbol	Min	Max	Unit
Master Mode (Standard-mode)				
SCL frequency	f_{scl}		100	kHz
Bus free between transmissions (SDA high time between start and stop)	t_{buf}	4.7		μs
Start condition hold time (SDA falling to SCL falling)	t_{stah}	4.0		μs
SCL low	t_{scll}	4.7		μs
SCL high	t_{sclh}	4.0		μs
SDA input hold from SCL falling	t_{sdah}	0	3.45	μs
SDA output valid from SCL falling	t_{ov}		3.45	μs
SDA input valid to SCL rising	t_{sdas}	250		ns
(repeated) start condition setup time	t_{stas}	4.7		μs
SDA and SCL rise time ¹	t_r		1000	ns
SDA and SCL fall time ¹	t_f		300	ns
Stop condition setup time (SCL rising to SDA rising)	t_{stps}	4.0		μs
Master Mode (Fast-mode)				
SCL frequency	f_{scl}		400	kHz
Bus free between transmissions (SDA high time between start and stop)	t_{buf}	1.3		μs
Start condition hold time (SDA falling to SCL falling)	t_{stah}	0.6		μs
SCL low	t_{scll}	1.3		μs
SCL high	t_{sclh}	0.6		μs
SDA input hold from SCL falling	t_{sdah}	0	900	ns
SDA output valid from SCL falling	t_{ov}		900	ns
SDA input valid to SCL rising	t_{sdas}	100		ns
(repeated) start condition setup time	t_{stas}	0.6		μs
SDA and SCL rise time ¹	t_r		300	ns
SDA and SCL fall time ¹	t_f		300	ns
Stop condition setup time (SCL rising to SDA rising)	t_{stps}	0.6		μs

Note 1: Rise and fall time specifications are based on the V_{IL} and V_{IH} values given in the [I²C-Bus Specification \[5\]](#) and not the V_{IL} and V_{IH} specifications given in [Table 15-3](#).

FIGURE 15-8: CONTROL PORT TIMING



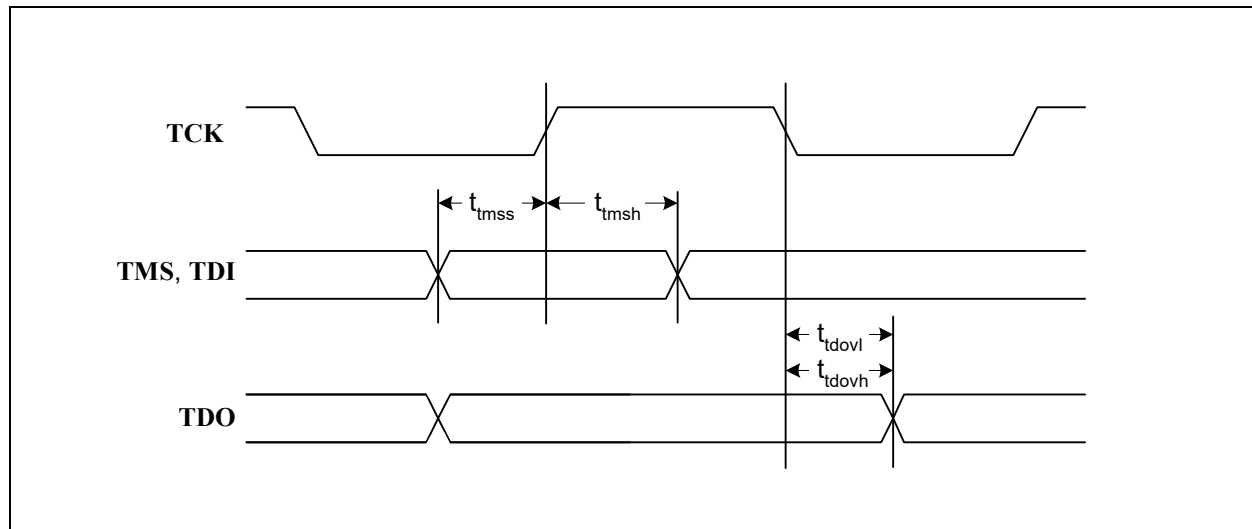
15.11 JTAG Timing

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDA18} = 1.8\text{ V} \pm 5\%$; $V_{DDA33}, V_{DDEn}, V_{DDPn}, V_{DDU} = 3.3\text{ V} \pm 5\%$; $GNDn, GNDAn, GNDE, GNDU = 0.0\text{ V}$; Load Capacitance = 30 pF ; unless otherwise noted.

TABLE 15-14: JTAG TIMING ELECTRICAL CHARACTERISTICS

Parameter	Symbol	Min	Max	Unit
Maximum TCK Frequency	f_{tck}		10	MHz
TDO output valid high from TCK falling edge	t_{tdovl}		25	ns
TDO output valid low from TCK falling edge	t_{tdovh}		25	ns
TMS and TDI input setup to TCK rising edge	t_{tmss}	5		ns
TMS and TDI input hold from TCK rising edge	t_{tmsh}	5		ns

FIGURE 15-9: JTAG TIMING



16.0 APPLICATION INFORMATION

The OS81092 Evaluation Board provides a platform for comprehensive understanding of a MOST50 Network device and implements various application concepts discussed in this chapter. Refer to [OS81092 Evaluation Board User's Guide \[11\]](#) for more information.

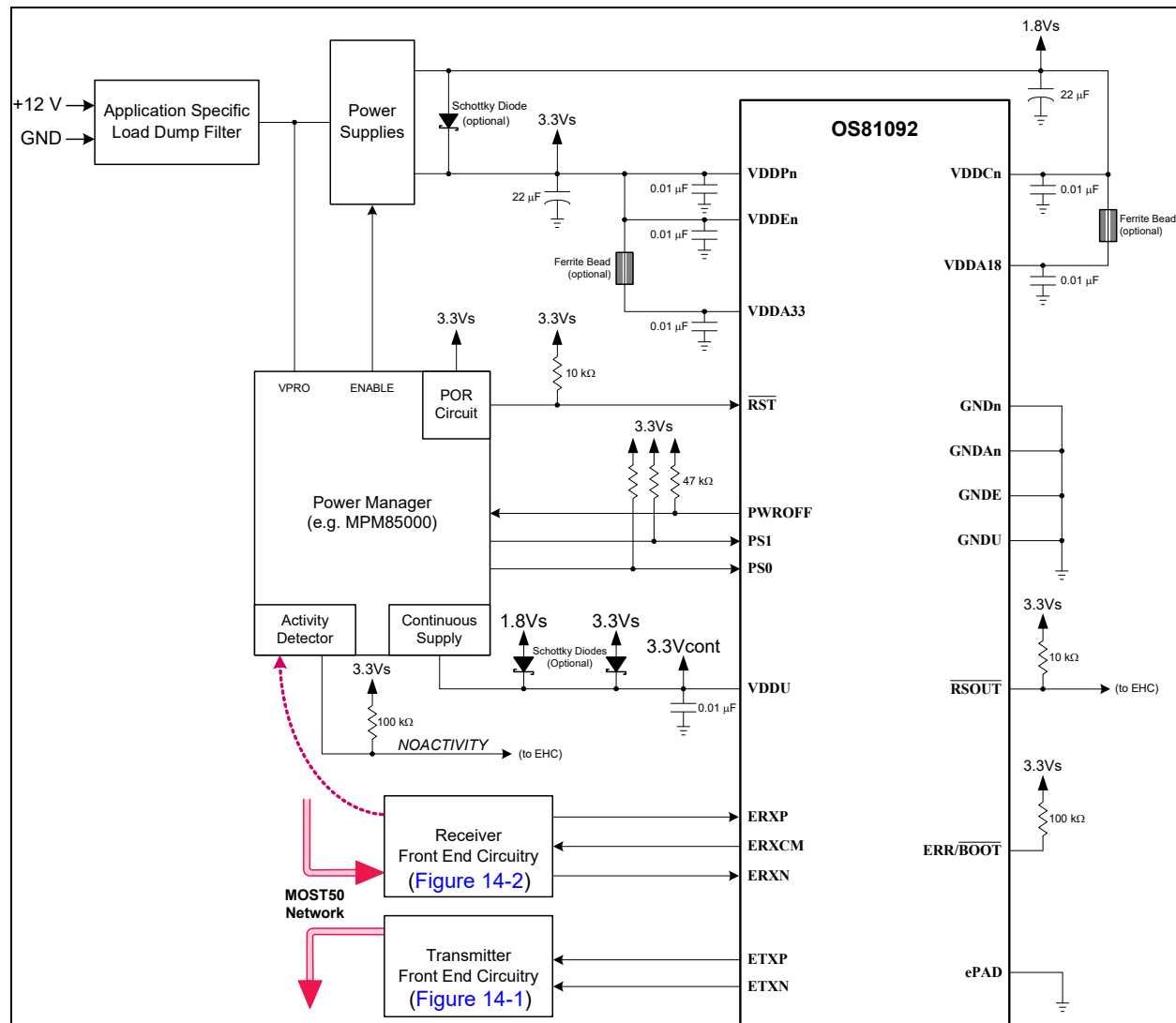
16.1 Power

The OS81092 requires 3.3 V power for the **VDDA33**, **VDDEn**, and **VDDPn** pins, and 1.8 V power for the **VDDA18** and **VDDCn** pins. In addition, for INIC to maintain critical network parameters while in a low power mode, a continuous 3.3 V power supply should be connected to **VDDU** to power a small block of persistent memory. See [Section 16.5 "Persistent Memory"](#) for more information.

The voltage on **VDDA18** and **VDDCn** must never be more than 0.5 V above the **VDDA33**, **VDDEn**, **VDDPn**, and **VDDU** pins. Furthermore, digital input pins must never be driven to more than the **VDDPn** supply. These requirements are applicable to power-up and power-down, as well as normal operating conditions.

[Figure 16-1](#) illustrates a typical power supply and decoupling architecture for the OS81092 required to minimize jitter effects. More information on system-wide INIC power management architectures can be found in the [MOST INIC Hardware Concepts Specification \[8\]](#). Although [Figure 16-1](#) illustrates connections to other parts, only logical connections are shown and the respective data sheet should be consulted for complete information on external components (such as proper decoupling).

FIGURE 16-1: TYPICAL POWER SUPPLY CONNECTION DIAGRAM



16.2 Optional Components

Figure 16-1 shows Schottky diodes between the 1.8 V and 3.3 V power supplies and ferrite beads on the analog power supply pins. The use of these components is optional and should be considered on a case-by-case basis, depending on application requirements and limitations.

As noted in Section 16.1 “Power”, the 1.8 V supply must never exceed the 3.3 V supplies by more than 0.5 V. Additionally, the 3.3 V analog supply must never exceed the 3.3 V continuous supply by more than 0.5 V. One approach to satisfying these power sequencing requirements is to connect Schottky diodes between the supplies to prevent a particular supply from exceeding another by more than a diode drop. If used, these diodes should be sized according to the power requirements of the 3.3 V supply.

Ferrite beads can be added to **VDDA18** and **VDDA33** for increased noise immunity in EMI-sensitive applications.

16.3 Reset

After both the 3.3 V and 1.8 V power supplies have stabilized, the OS81092 must be reset ($\overline{\text{RST}}$ low) for normal operation. A proper reset is when both supplies are $\geq 90\%$ VDD; however, electrical specifications are applicable only within $\pm 5\%$ VDD. Once initialized, INIC is typically reset by the EHC only when a fatal error occurs, or when loading the *INIC Configuration String* (see the [OS81092 INIC API User's Manual \[2\]](#)).

If the external POR circuit does not drive the $\overline{\text{RST}}$ line high (e.g. open-drain output), an external pull-up resistor to 3.3 V should be used. If the external POR circuit drives the $\overline{\text{RST}}$ line high and low (e.g. push-pull output), a series resistor should be used between the POR circuit and the OS81092 $\overline{\text{RST}}$ pin in lieu of the pull-up. This series resistor allows INIC to be reset by something other than the POR circuit, such as the EHC or the *INIC Explorer* tool. The pull-up or series resistor should be at least 10 k Ω .

The $\overline{\text{RST}}$ pulse width should be long enough to allow the external configuration pins ($\overline{\text{ERR/BOOT}}$ and $\overline{\text{INT}}$) to achieve their default state. The transition time of $\overline{\text{RST}}$ is not relevant, provided the noise level does not exceed the $\Delta V_{\text{hys-rst}}$ parameter specified in Section 15.4.2 “Reset and Power Management”.

16.4 Other Application Pins

For normal operation, the $\overline{\text{ERR/BOOT}}$ pin should be pulled to 3.3 V. To support the *Customer Configuration Interface*, the on-board pull-up resistor on $\overline{\text{ERR/BOOT}}$ must not be less than 10 k Ω ; however, the actual resistor value should be chosen based on the input leakage of the INIC pin and on the other devices connected to the line. The JTAG Port pins **TMS**, **TCK**, **TDI**, and **TDO** require pull-up resistors to 3.3 V, regardless of whether or not the JTAG interface is used.

The OS81092 $\overline{\text{RSOUT}}$ pin is an optional output that allows INIC to reset the EHC. When used, the $\overline{\text{RSOUT}}$ pin should be pulled high by an on-board resistor (10 k Ω recommended) and connected to the Reset pin of the EHC. INIC drives $\overline{\text{RSOUT}}$ low to reset the EHC when either:

- the `Reset.Start(Target)` INIC function is called and the `Target` parameter specifies the EHC as the device to be reset, or
- the watchdog timer expires and the `WatchdogMode()` INIC function indicates the EHC should be reset.

The length of the reset pulse on $\overline{\text{RSOUT}}$ is 10 ms. For more information on the $\overline{\text{RSOUT}}$ pin, refer to the [OS81092 INIC API User's Manual \[2\]](#) and the [MOST INIC Hardware Concepts Specification \[8\]](#).

When an external crystal is used, the **XTI** and **XTO** pins should be connected directly to the crystal (e.g. no series resistance). The crystal used must be 384×Fs and should comply with the specifications outlined in Section 16.7 “Crystal Oscillator Selection”. If an external clock is used in lieu of a crystal oscillator, it must be connected to **XTI** and support the drive levels listed in the Chapter 15.0 “Electrical Characteristics” chapter.

When external power management is used, the **PWROFF** pin must be pulled high through an external resistor. After initialization, INIC drives **PWROFF** low during normal operation and releases **PWROFF** when the INIC Processor is ready to be shutdown. Refer to Chapter 10.0 “External Power Management” for more information.

16.5 Persistent Memory

The OS81092 may optionally use persistent memory to maintain certain internal values when the OS81092 is placed in a low-power mode (e.g. MOST Sleep Mode). To support this functionality, the application should supply 3.3 V continuous power to **VDDU**, which powers the RAM-based persistent memory, independent of all other switched power supplies (**VDDA18**, **VDDA33**, **VDDCn**, **VDDEn**, and **VDDPn**). Refer to the [OS81092 INIC API User's Manual \[2\]](#) for more information on using OS81092 persistent memory.

The Microchip *Automotive Power Management Device* (MPM85000) provides a 3.3 V continuous output which can be used to power OS81092 persistent memory (e.g. **VDDU**). Integrating both the OS81092 and MPM85000 on a MOST Network node minimizes application circuitry and software complexity, allowing the application designer to focus on the application, rather than on MOST compliance. Contact Microchip for more information on the MPM85000.

Depending on OEM requirements for robustness and flexibility, the use of OS81092 persistent memory may not be required in all applications. When an application does not utilize persistent memory to store MOST Network parameters during a low-power mode of operation, the OS81092 **VDDU** pin should be connected to 3.3 V switched power (e.g. **VDDP**).

Note: When 3.3 V power is removed from the OS81092 **VDDU** pin, the content of the persistent memory block becomes invalid.

16.6 Configuration

The OS81092 provides *One-Time Programmable* (OTP) memory that is used to store power-up configuration settings. OTP memory is divided into two sections, which allows configuration settings to be written twice. When OTP memory is written once, the first section stores the configuration data and the second section is unused. When OTP memory is written again, the second section stores the configuration data and all data previously written into the first section is ignored. After power-up/reset, configuration settings are loaded from the appropriate OTP section into RAM.

The power-up configuration settings stored in OTP memory are collectively referred to as the *INIC Configuration String* (or *ConfigString*). The *INIC Configuration String* includes initial hardware configuration information such as:

- Control Port I²C address,
- Default port for INIC-EHC communication,
- MediaLB Port default mode and speed,
- Enable option for GPIO Port and Control Port I²C master mode,
- **RMCK** configuration, and
- Network device mode (timing-master/timing-slave).

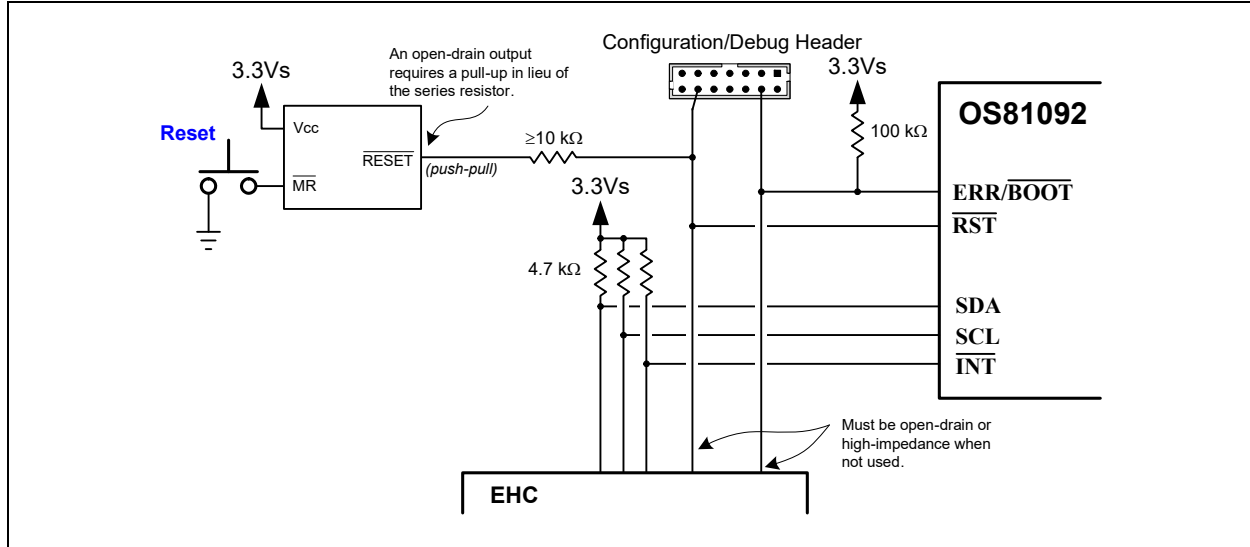
If OTP memory is not initialized, OS81092 Factory default settings are used in its place. OTP memory only needs to be programmed when the Factory defaults are not adequate for the application. Refer to the [OS81092 INIC API User's Manual \[2\]](#) for the OS81092 Factory defaults, as well as a complete listing of *INIC Configuration String* settings.

The *INIC Configuration String* can be programmed into OTP memory by the EHC via the Control Port. This approach is suitable for production-level applications. See [Section 16.6.1 "EHC via Control Port"](#) for more information. Alternatively, The Microchip [INIC Explorer Interface Box \[4\]](#) tool can load OTP memory via the *Customer Configuration Interface*. This technique is suitable for development-level applications. See [Section 16.6.2 "INIC Explorer via JTAG Port"](#) for more information.

16.6.1 EHC VIA CONTROL PORT

The *INIC Configuration String* data can be written into OTP memory by the EHC via the OS81092 Control Port. To support this feature, the EHC must be connected to the INIC Control Port (*SDA*, *SCL*, and *INT*) and be capable of controlling the *ERR/BOOT* and *RST* pins. The required connections are shown in [Figure 16-2](#).

FIGURE 16-2: CONNECTION DIAGRAM FOR CONFIGURATION VIA CONTROL PORT



The sequence required of the EHC to load configuration data into OTP memory is described in the [OS81092 Programming Guide \[12\]](#).

16.6.2 INIC EXPLORER VIA JTAG PORT

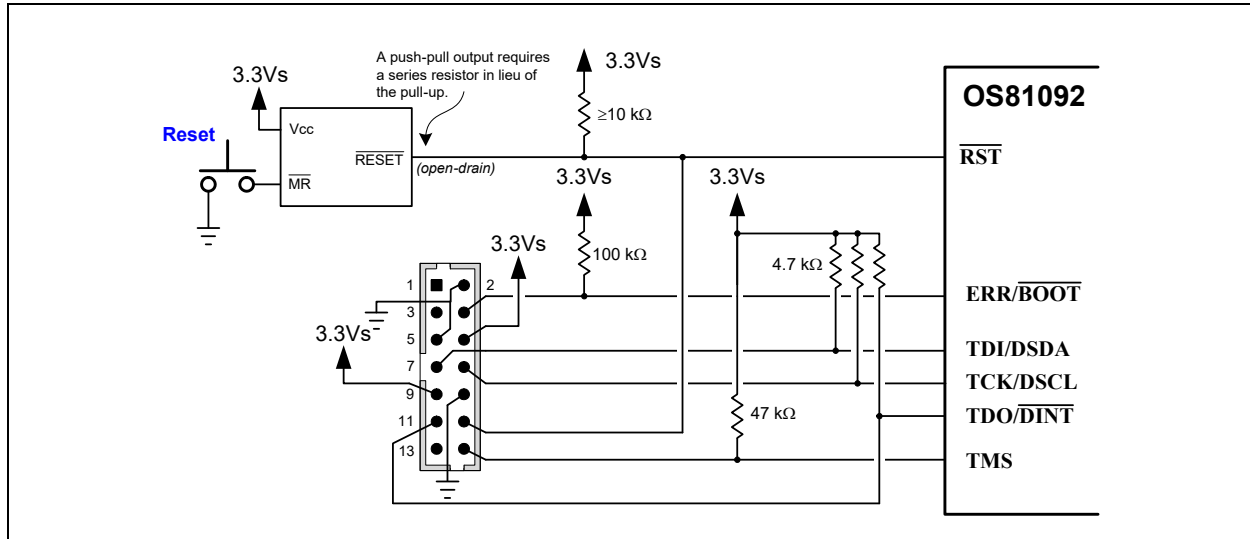
The OS81092 JTAG Port can be converted into a configuration interface for use with the Microchip [INIC Explorer Interface Box \[4\]](#) tool. *INIC Explorer* requires that the application PCB provide a standard 14-pin (2x7) 2 mm header (such as Molex 87332-1420 or equivalent). The interface between the *INIC Explorer* tool and the OS81092 is also referred to as the *Customer Configuration Interface*.

In addition to writing *INIC Configuration String* data into OTP memory, the *INIC Explorer* platform provides debugging capabilities by allowing users to access INIC (via PC software) for:

- Exploring internal INIC properties and states while it is operating in the target platform,
- Viewing internal INIC states and routing configurations graphically, and
- Creating an INIC data memory snapshot as a file dump.

Figure 16-3 illustrates the necessary connections between the OS81092 and the configuration/debug header that accommodates *INIC Explorer*. To allow the header to drive the **ERR/BOOT** and **RST** pins during configuration, the pull-up resistors on these lines should be greater than 10 k Ω . If the **ERR/BOOT** and **RST** lines are also connected to the EHC (as described in [Section 16.6.1 “EHC via Control Port”](#)), the EHC must use open-drain logic or set its outputs as high-impedance when the *INIC Explorer* connects to the OS81092 via the configuration/debug header.

FIGURE 16-3: CONFIGURATION/DEBUG HEADER



Contact Microchip for more information on the *INIC Explorer* platform.

16.7 Crystal Oscillator Selection

Several factors must be considered when selecting a crystal, including: load capacitance, operating temperature, cut, and oscillator margin. Oscillator margin is a measure of the stability of an oscillator circuit, and is defined as the ratio of the oscillator's negative resistance (R_{NEG}) to the crystal's ESR (R_{ESR}), or:

$$\text{Margin} = \frac{|R_{NEG}|}{R_{ESR}} = \frac{|R_{VAR}| + R_{ESR}}{R_{ESR}}$$

The negative resistance can be measured by placing a variable resistor (R_{VAR}) in series with the crystal and finding the largest resistor value where the crystal still starts up properly. This point would be just below where the oscillator does not start-up or where the start-up time is excessively long.

Ideally, oscillator margin should be greater than 10, and should be at least 5. Smaller oscillator margin can affect the ability of the oscillator to start up.

The load capacitance, specified when ordering the crystal, is the series combination of the capacitance on each leg of the crystal. This capacitance includes not only the added capacitors, but also PCB trace (shunt) capacitance and chip pin capacitance. Larger capacitors also have a negative effect on oscillator margin. In general, the external capacitors on each leg (C1 and C2 in [Figure 11-1](#)) should be in the 10 to 22 pF range.

TABLE 16-1: CRYSTAL OSCILLATOR SPECIFICATIONS

Name	Value	Description
Frequency	384×Fs	Frequency relative to operating network frame rate (Fs)
Correlation	Parallel Resonant	Mode of oscillation
Osc. Mode	Fundamental	Oscillation mode or operation mode
C_L	10-22 pF	Load capacitance (typical)
C_O	7 pF	Recommended maximum shunt (parallel) capacitance
ESR	100 Ω	Recommended maximum equivalent series resistance
Drive Level	50 μ W	Drive level (typical)
Cut	AT	AT cut produces the best temperature stability
Tolerance	±200 ppm	Frequency tolerance over all conditions (typical)

The crystal cut and tolerance value listed in [Table 16-1](#) are *typical* values and may be changed to suit differing system requirements. Higher ESR values (than those listed in the [Table 16-1](#)) run the risk of having start-up problems and should be thoroughly tested before being used. Contact the crystal manufacturer for more information.

16.8 Physical Layer

Capacitors in the signal path should be C0G type for accuracy and stability. These capacitors include: C15, C16, C17, C18, and C19. The C22, C23, C24 and C25 capacitors may be C0G type, as C0G is more commonly available for this value. Additionally, these capacitors are AC grounds for the transformer center taps, thus should be rated for 50 V.

Listed below are all the parts shown in [Figure 14-2](#). Only critical parts have a manufacturer and part number listed. Other components are generic and may be obtained from various suppliers.

TABLE 16-2: EPHY FRONT END CIRCUITRY - COMPONENT SELECTION

Reference	Part Number	Manufacturer	Description
C15, C16, C18			Capacitor, 22 pF, C0G, 50 V, 5 %
C17, C19			Capacitor, 68 pF, C0G, 50 V, 5 %
C20, C21			Capacitor, 0.01 μ F, X7R, 50 V, 10 %
C22, C23			Capacitor, 10 pF, C0G, 50 V, 5 %
C24, C25			Capacitor, 1000 pF, C0G, 50 V, 5 %
C206			Capacitor, 0.01 μ F, X7R, 16 V, 10 %
D201, D202	ESD7C5.0DT5G	ON SEMI	TVS Device, $V_{BR} = 11$ V, $C_{MAX} = 6.2$ pF
J1	1376350-1	TYCO/AMP	Connector, 8-pin, 2x4, Right-Angle
L2, L3, L5, L6	BLM18BB141SN1	MURATA	Ferrite Bead, 140 Ω @100 MHz, 200 mA, 0603
L4	LQW18ANR10J00	MURATA	High Freq Chip Coil, 100 nH, 5 %, 220 mA, 0603
L7, L8	ACM2012-121-2P-T001	TDK	Common-Mode Choke Coil, 20 V, 370 mA, 120 Ω @100 MHz, SMD
R2, R3			Resistor, 43.2 Ω , 0603, 1/16 W, 1 %, 200 ppm
R4			Resistor, 182 Ω , 0805, 1/10 W, 1 %, 100 ppm
R201, R202			Resistor, 64.9 Ω , 0603, 1/16 W, 1 %, 200 ppm
T1	TG100-MOST15N2LF	HALO	MOST Transformer, SMD (see Table 16-3)
	TG100-MOST9N2LF	HALO	
	TLA-8T101WLF(-T)	TDK	
	TLA-8T102WLF(-T)	TDK	

TABLE 16-3: EPHY FRONT END CIRCUITRY - TRANSFORMER SPECIFICATIONS

Parameter	Value	Units/Notes
Open Circuit Inductance	200	μ H (100 kHz, 0.1 V_{rms})
Insertion Loss	-1	dB _{max} (100 kHz to 50 MHz)
Isolation Voltage	1500	$V_{rms-min}$
Common-Mode Rejection	-40	dB _{min} (0.1 to 100 MHz)
DC Winding Resistance	0.6	Ω_{max}
Winding to winding capacitance	18	pF _{max}
Turns Ratio (windings)	1:1 (± 3 %) 1:1.5 (± 3 %)	Reception Signal Transmission Signal
Common to Diff Rejection	-40	dB _{min} (0.1 to 100 MHz)
100 Ω Return Loss	-12.0	dB _{min} (at 50 MHz)
Operating Temperature	-40 / +85	$^{\circ}$ C

FIGURE 16-5: EXAMPLE LAYOUT - TOP AND BOTTOM LAYERS SUPERIMPOSED

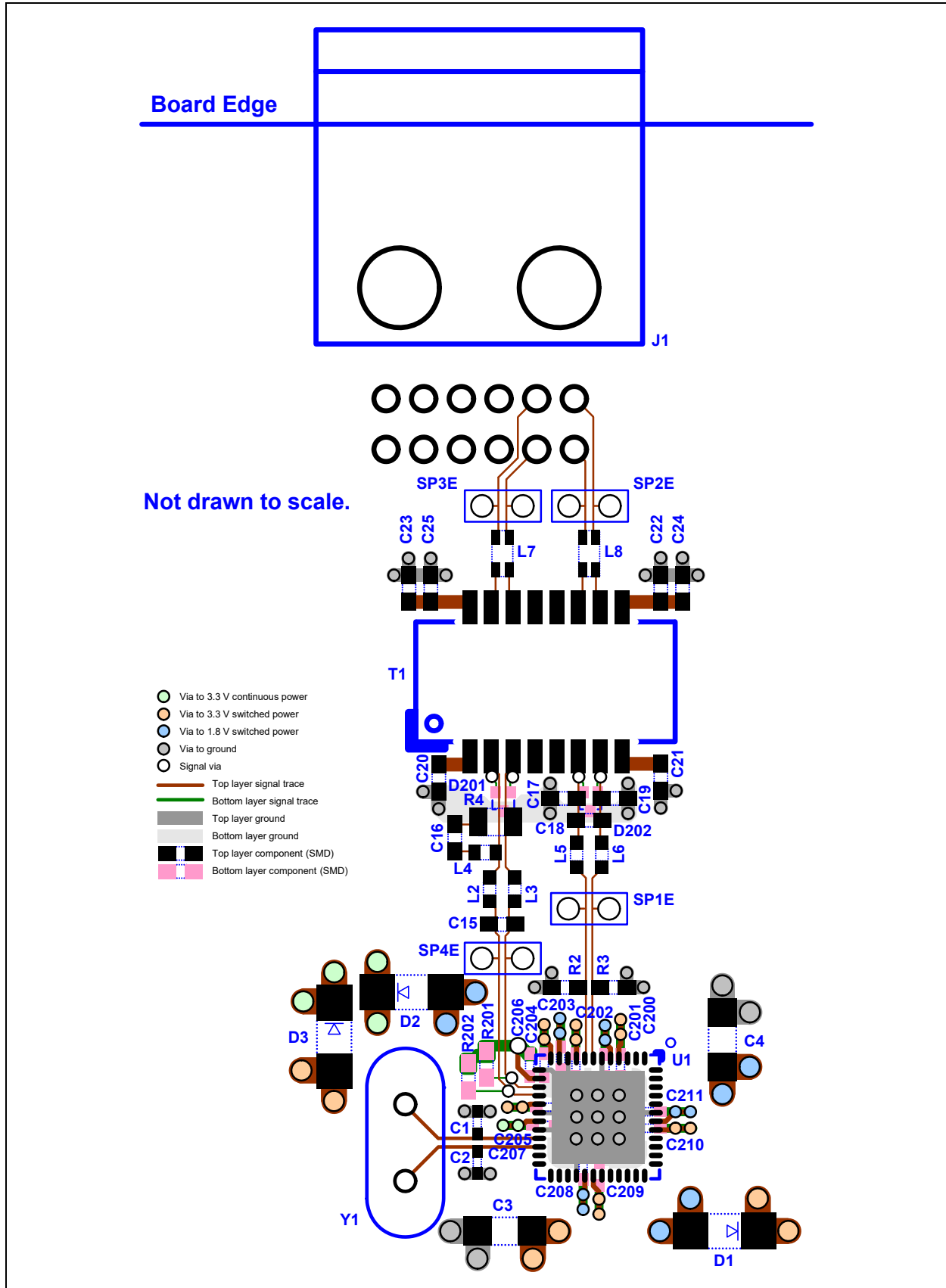


FIGURE 16-6: EXAMPLE LAYOUT - TOP LAYER

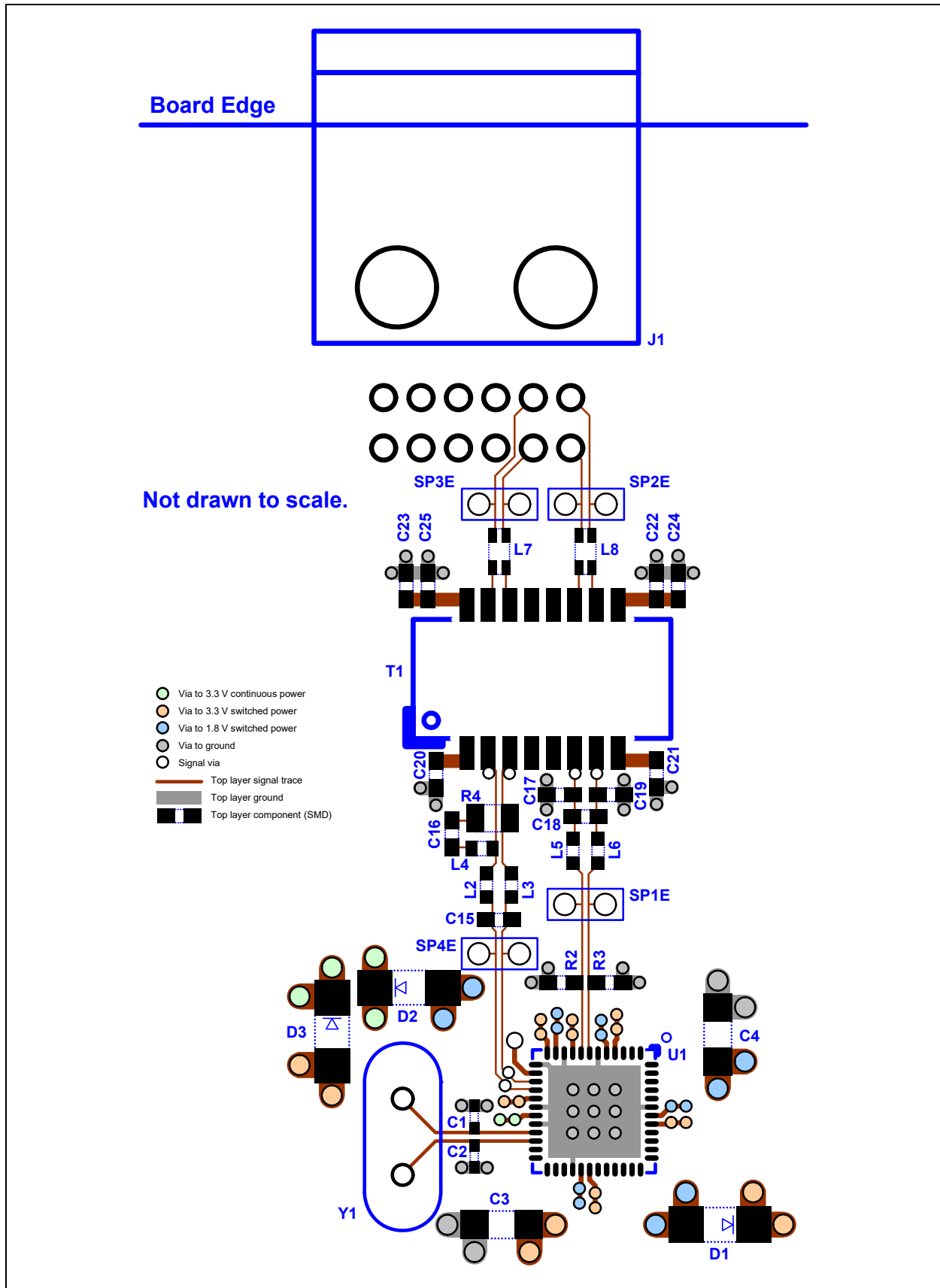


FIGURE 16-7: EXAMPLE LAYOUT - BOTTOM LAYER

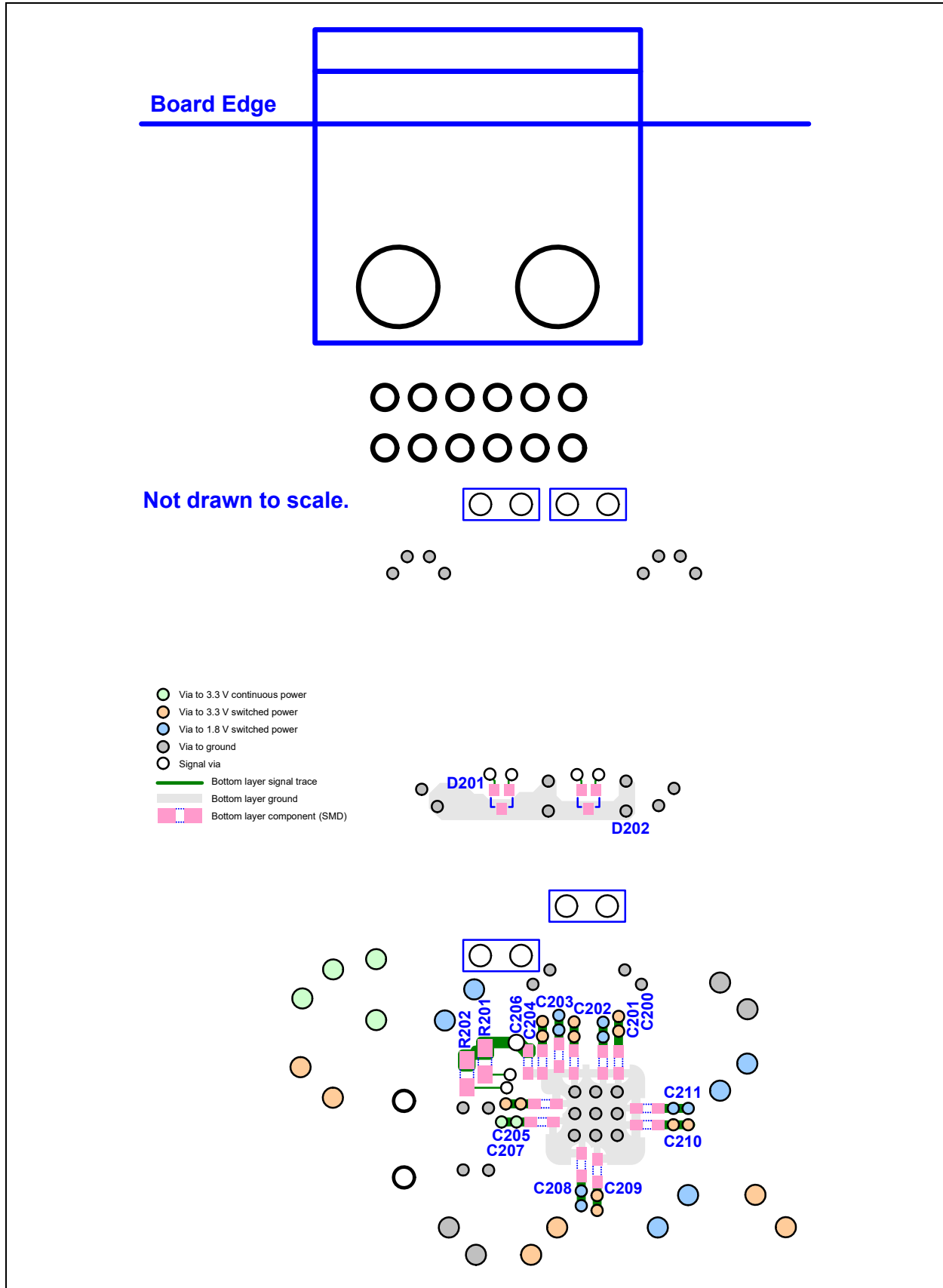


FIGURE 16-8: EXAMPLE LAYOUT - GROUND LAYER

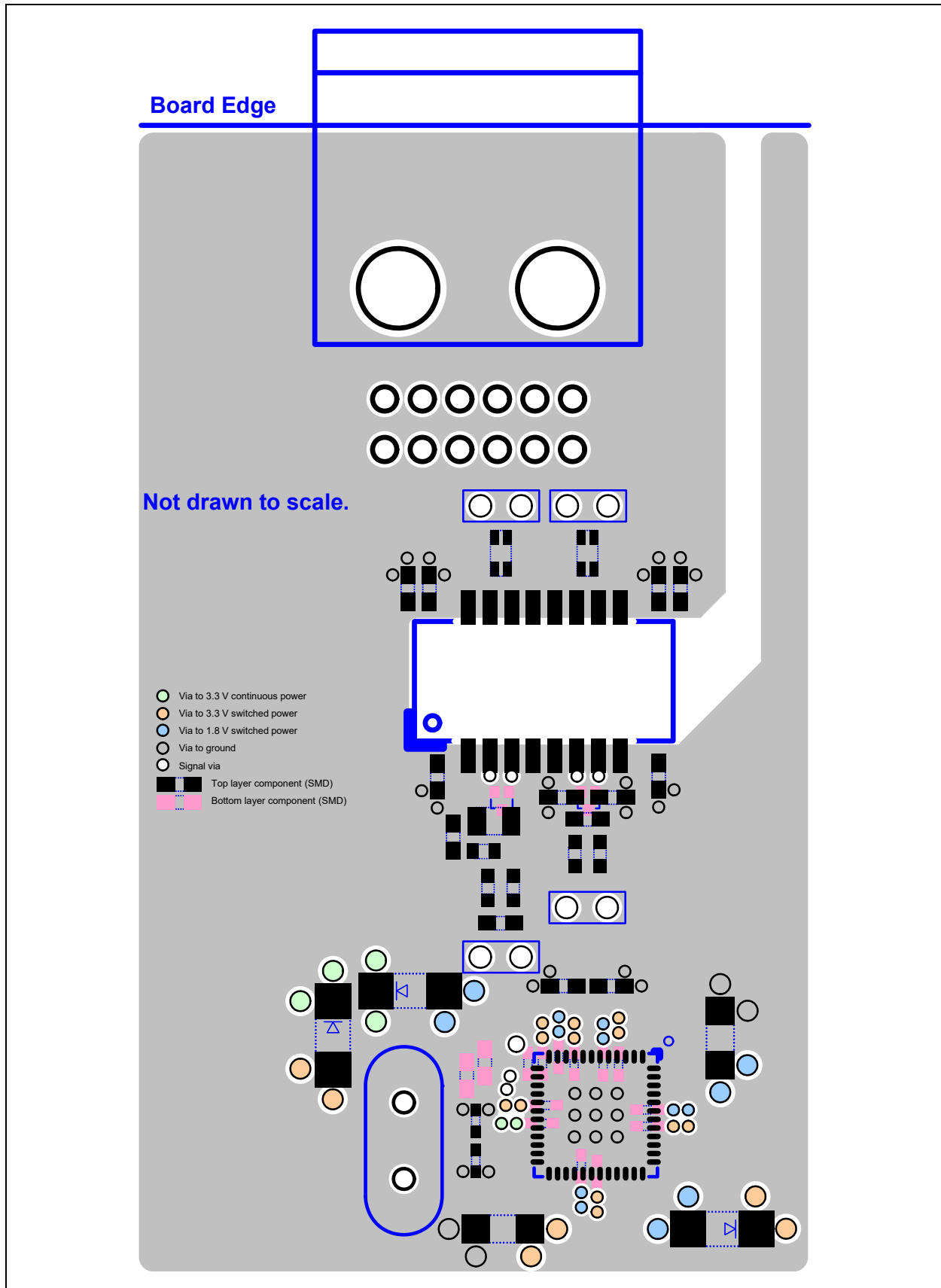


FIGURE 16-9: EXAMPLE LAYOUT - INIC PERIPHERAL POWER PLANE

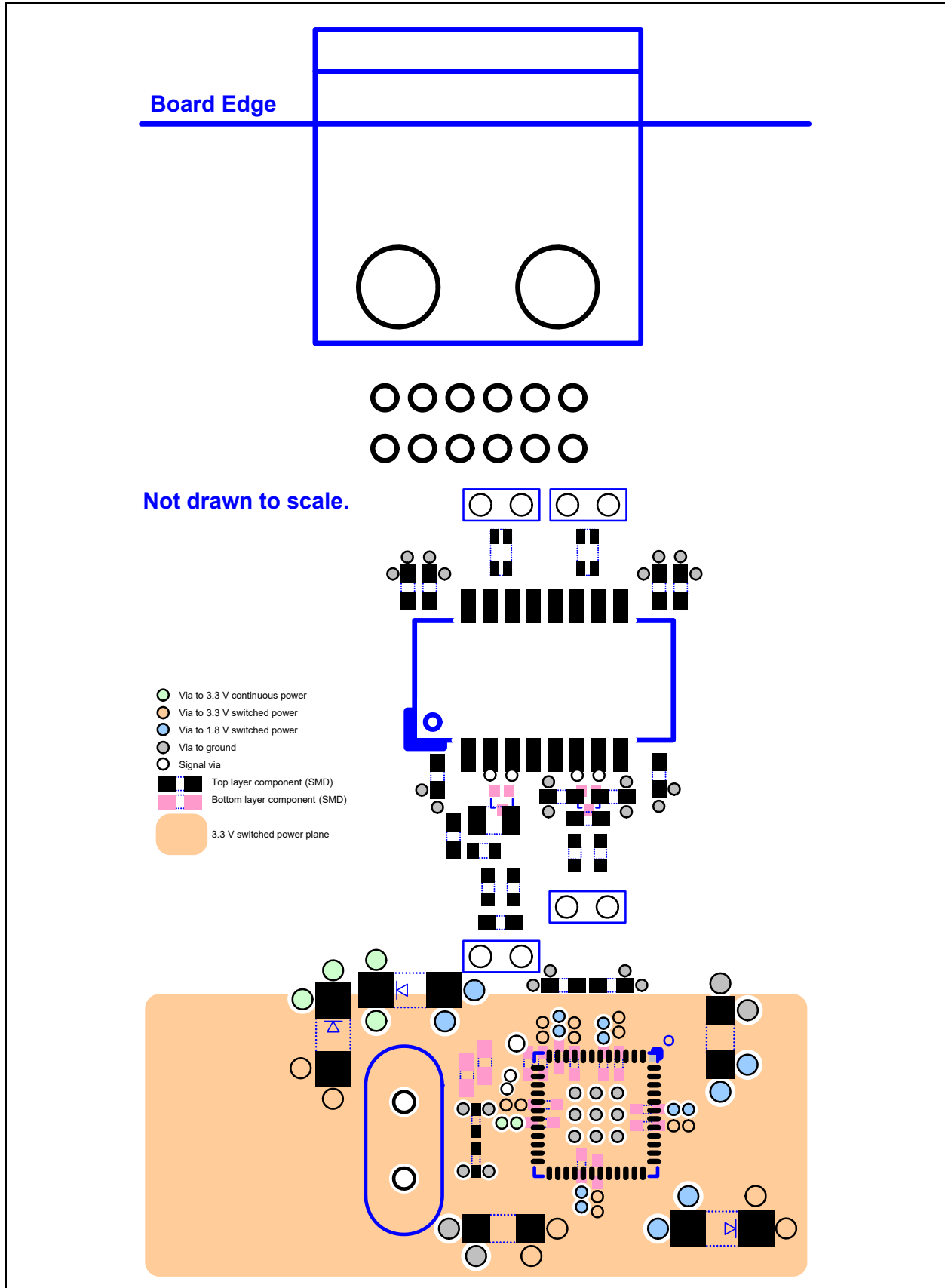
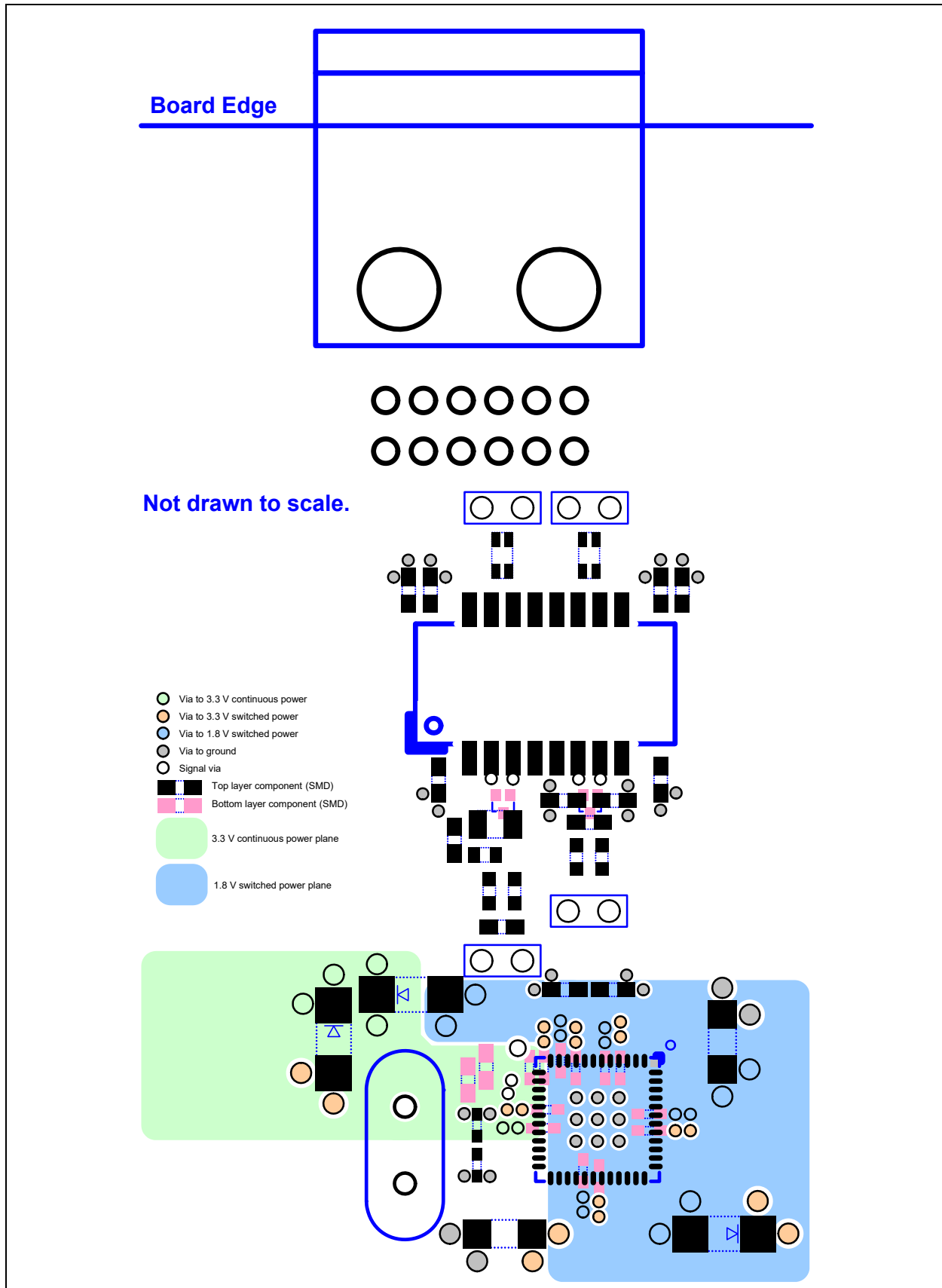


FIGURE 16-10: EXAMPLE LAYOUT - INIC CORE POWER AND CONTINUOUS POWER PLANES



16.9.1 POWER AND DECOUPLING

A 0.01 μ F decoupling capacitor should be used at each OS81092 power pin for localized decoupling. For EMI-sensitive applications C0G capacitors are strongly recommended for their superior performance at high frequencies. To reduce trace impedance and keep the loop areas small, decoupling capacitors should be positioned as close as possible to the OS81092. Two vias to the ground plane should be used to minimize inductance.

The recommended placement of decoupling capacitors is on the same side of the PCB as the OS81092. However, decoupling capacitors may be mounted on the opposite side of the PCB to reduce the distance between the capacitor and the pin, as inductance from vias is less harmful than long traces that may allow external interference.

In situations where both large (e.g. electrolytic or tantalum) and small (e.g. ceramic) capacitors are used, the smaller capacitor should take priority for placement and layout optimization.

As in all low-noise circuits, two vias should be used from power and ground traces to the respective power and ground planes to reduce parasitic inductance, whenever possible. Additionally, power traces should be as wide as practical and all OS81092 ground pins (digital, analog, continuous, and ePHY) should be tied to the same ground plane.

Ground-plane fill is used under the part to minimize the impedance between ground pins, which also helps minimize EMI. If ground-plane fill must be broken by a trace, multiple stitching vias should connect each section of the fill to a reference ground plane on another layer. Vias that connect high frequency decoupling capacitors to the ground plane should be as close as possible to the OS81092 to minimize loop area back to the chip ground pins.

16.9.2 PHYSICAL LAYER

For optimum performance, attention should be given to the routing and placement of physical layer components. Meeting the stringent emission and interference requirements of the automotive environment requires strict adherence to good circuit layout practices. The following recommendations represent steps taken by Microchip to meet the required automotive standards. All components listed in this section refer to reference designators shown in the OS81092 Reference Schematic (Figure 16-4), as well as the ePHY Front End Circuit Diagram (Figure 14-2).

Note: Circuit configurations different from those shown in Figure 14-2 and Figure 16-4, and layout configurations different from those shown in this section may produce different results. Customers are responsible for designing and testing their unique systems for emission and interference adherence.

The ePHY Front End circuitry (including the MOST Network connector) should reside on the same PCB as (and in close proximity to) the OS81092. The distance between the external circuitry that represents the receiving physical interface (shown in Figure 14-2) and the OS81092 should be as short as possible to optimize signal integrity and improve EMC.

The OS81092 transmit and receive lines (ETXP/ETXN and ERXP/ERXN) are differential signal pairs, which must be implemented on the PCB as impedance controlled (e.g. 100 Ω) traces. Differential pairs should be matched in length and tightly coupled, with constant spacing maintained between them over their entire length. When possible, the signals should be routed on a single layer and the use of vias should be minimized. Additionally, traces should be routed over a continuous reference plane, preferably a ground plane. To minimize reflections, two 45 degree turns should be used rather than a single 90 degree turn. Crosstalk can be avoided by not running other signals in parallel to the transmit and receive lines for long distances.

The differential signal pairs should maintain balanced symmetry throughout the ePHY Front End circuitry. The isolation transformer (T1) separates cable-side components from chip-side components. The placement and orientation of C20, C21, C22, C23, C24, C25, and T1 shown in Figure 16-5 allows currents to be steered through the filtering circuitry. A void should be constructed under the transformer (on all layers) to reduce magnetic coupling between internal planes and the cores of the transformer.

The length of the traces between the capacitor (C15) and the reception termination components (R201, R202, and C206) is less critical; however, the reception termination components must be placed as close to the OS81092 as possible.

A chassis ground plane (not shown) may exist beneath the MOST Network connector and the power input connector, if needed to suppress EMC issues. The chassis ground must be separated from the signal ground by a moat at least 3 mm wide. The moat is an area containing no planes or traces on any layer of the board. In the circuit, signals may only cross the moat through the transformer (T1) and the common-mode chokes (L7 and L8). The components on the chas-

OS81092

sis ground side of the moat, include: C22, C23, C24, C25, J1, L7, L8, and the connector side of the transformer. Additionally, a metal shield may cover the connector, the connector side of the transformer, and the power input connector (not shown). The shield must be grounded to the chassis.

To aid in MOST Compliance testing, test points should be included for SP1E, SP2E, SP3E, and SP4E, as shown in Figure 16-5.

16.9.3 MediaLB INTERFACES

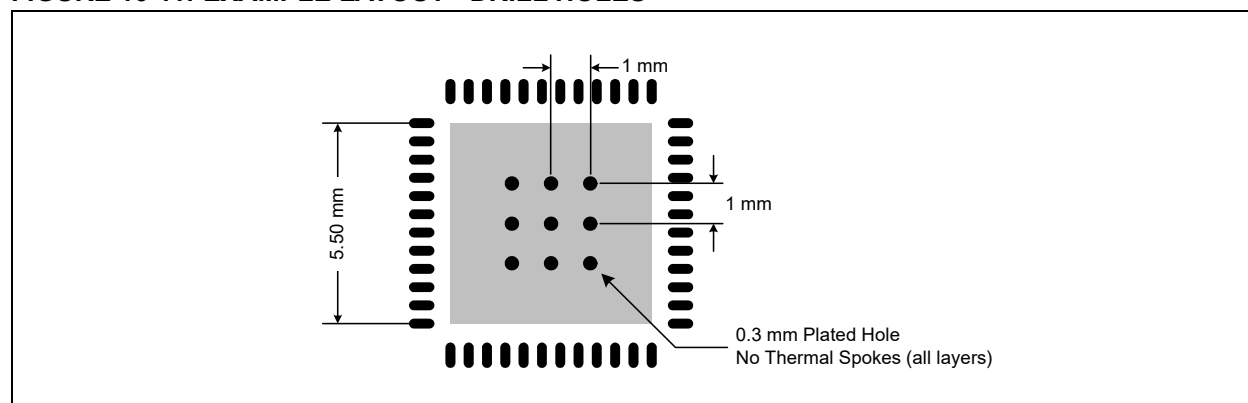
The OS81092 *Media Local Bus* (MediaLB) 3-pin and 5-pin interfaces are provided to facilitate high-speed data exchange between INIC and the EHC. MediaLB is intended for use on a single PCB. For effective operation, especially at higher bus speeds, signal routing and component placement is extremely important.

Note: Refer to the [MediaLB Specification \[3\]](#) for application guidelines, including layout examples.

16.9.4 THERMAL CONSIDERATIONS

The paddle of the OS81092 QFN package should be soldered to the thermal land (thermal pad) for efficient heat dissipation. The thermal pad should be at least as large as the exposed portion of the package paddle and be connected to the ground plane of the board using heat tubes (vias without thermal reliefs). At least nine heat tubes are recommended, with a drill hole size no larger than 0.33 mm (13 mils).

FIGURE 16-11: EXAMPLE LAYOUT - DRILL HOLES



16.9.5 MISCELLANEOUS

Floating areas of copper fill near INIC should be avoided due to the risk of noise coupling into the system. A keep-out can be used around the OS81092 and its immediate peripheral circuitry (e.g. decoupling capacitors and external crystal) to ensure islands are not created when copper fill is poured. If a keep-out is not used, multiple stitching vias should tie any floating copper fills to an appropriate reference on other power or ground layers.

The XTI and XTO traces should be equal in length. Additionally, the distance between INIC and the external crystal should be minimized.

APPENDIX A: REFERENCES

Documents listed below and referenced within this publication are current as of the release of this publication and may have been reissued with more current information. To obtain the latest releases of Microchip documentation please visit the Microchip website. Please note, some Microchip documentation may require approval. Contact information can be found at www.microchip.com.

All non-Microchip documentation should be retrieved from the applicable website locations listed below. Microchip is not responsible for the update, maintenance or distribution of non-Microchip documentation.

Because the Internet is a constantly changing environment, all Internet links mentioned below and throughout this document are subject to change without notice.

- [1] MOST Specification
MOST Cooperation. www.mostcooperation.com.
- [2] OS81092 INIC API User's Manual
Microchip. www.microchip.com.
- [3] MediaLB Specification
TB0400AN4V2: Microchip. www.microchip.com.
- [4] INIC Explorer Interface Box
Microchip. www.microchip.com.
- [5] I²C-Bus Specification
UM10204_3. NXP (formerly a division of Philips). www.ics.nxp.com.
- [6] MediaLB Monitor Adapter User's Manual
Microchip. www.microchip.com.
- [7] MediaLB Analyzer User's Manual
Microchip. www.microchip.com.
- [8] MOST INIC Hardware Concepts Specification
DS60001264: Microchip. www.microchip.com.
- [9] IEEE Standard Test Access Port and Boundary-Scan Architecture
IEEE Std. 1149.1 Institute of Electrical and Electronics Engineers (IEEE). www.ieee.org.
- [10] MOST Specification of Electrical Physical Layer
MOST Cooperation. www.mostcooperation.com.
- [11] OS81092 Evaluation Board User's Guide
DS60001239: Microchip. www.microchip.com.
- [12] OS81092 Programming Guide
Microchip. www.microchip.com.

APPENDIX B: REVISION HISTORY

The most extensive and pertinent application changes are listed in [Table B-1](#), although various other differences may be observed between document revisions.

TABLE B-1: DATA SHEET REVISION SUMMARY

Location	Description of Changes
DS60001271B	
General	- "Confidential" designation has been removed from this Data Sheet. No other content changes made.
Package Outline	- Update to 48 Pin QFN Package drawing
DS60001271A	
General	- Converted entire document to Microchip corporate documentation template
Package Outline	- Updated package drawing (Figure 1-1) with coplanarity data
Pinout	- Added GPIO Port pins (GP0, GP3, GP4, GP5 and GP7) to device pinout (Figure 2-1 and Table 2-1)
Control Port	- Added details describing Control Port operation in I ² C master mode
Streaming Port	- Added 3 additional sequential data formats to Table 9-2 for bit clock rates of 32×Fs, 16×Fs and 8×Fs
External Power Management	- Added details describing new options available for Revision C1D devices
GPIO Port	- New chapter added
Electrical Characteristics	- Updated power supply sequencing requirements as published in previous errata document - Updated maximum power dissipation specification for Revision C1D devices - Updated I_{DDC} specification for Revision C1D devices - Added output rise and fall times for GPIO pins to <i>AC Characteristics</i> section - Added various GPIO timing specifications to <i>AC Characteristics</i> section - Updated minimum RMCK frequency specification in <i>AC Characteristics</i> section - Updated minimum SCK frequency specification in <i>Streaming Port</i> section - Added various timing specifications for I²C Master Mode to <i>Control Port</i> section
DS81092AP3:	
General	- Added Microchip TrueAuto Quality marker and automotive quality process overview
Pinout	- Corrected Figure 2-1 to show proper orientation of package markings
JTAG Port	- Removed OSSPRI instruction from Instruction Register (IR) - Updated the Identification Register (IDR) revision code for hardware Revision C devices
Electrical Physical (ePHY) Interface	- Changed UTP cable nominal impedance to match MOST Cooperation specification - Added description of ESD suppression to ePHY Transmitter/Receiver Front End sections

TABLE B-1: DATA SHEET REVISION SUMMARY (CONTINUED)

Location	Description of Changes
Electrical Characteristics	- Added VDDA33 power sequencing requirement in <i>Absolute Maximum Ratings</i> section - Added Power Dissipation specification in <i>Absolute Maximum Ratings</i> section - Added Note 1 to <i>Guaranteed Operating Conditions</i> defining temperature range for OTP writes - Changed High-Level Input Voltage (V_{IH}) specification for XTI - Updated current specifications in <i>DC Characteristics (non-MediaLB pins)</i> section, as follows: - Added maximum current specifications (I_{AC}, I_{AP}, I_E, I_{DDC}, I_{DDP}, I_{UA}, and I_{US}) - Removed all typical current specifications - Added minimum jitter tolerance (t_{jm}) specification - Added the following specifications to the <i>Reset and Power Management</i> section: - RST rising to ETXP/ETXN output enabled (t_{inicit}) - RST rising to PWROFF asserted low (t_{pal}) - Activity on ERXP/ERXN input to ETXP/ETXN output enabled ($t_{NtwStartup}$) - Loss of activity on ERXP/ERXN input to ETXP/ETXN output disabled ($t_{NtwShutdown}$) - Updated the <i>ePHY Network Characteristics</i> section as follows: - Corrected V_{CMT} typical value - Replaced V_{CMR} and V_{OCM} typical values with minimum and maximum values - Updated various specifications in MediaLB Port (3-pin) and MediaLB Port (5-pin) sections (for all MediaLB speeds), as follows: - Changed MLBCLK operating frequency (f_{mck}) specifications - Changed MLBCLK low time (t_{mckl}) specifications for worst-case timing - Changed MLBCLK high time (t_{mckh}) specifications for worst-case timing - Removed MLBCLK pulse width variation (t_{mpwv}) specification - Added typical SCK duty cycle (t_{sckdc}) specification to <i>Streaming Port</i> section
Package Outline	- Removed minimum and maximum theta parameter from package dimensions table and drawing - Added typical Junction to Package (Ψ_{JT}) and typical Junction to Ambient (θ_{JA}) specifications
Application Information	- Added optional Schottky diode recommendation for VDDA33 power sequencing requirement - Updated <i>Persistent Memory</i> section for clarity - Updated reference schematic and layout diagrams in <i>Layout Guidelines</i> section
DS81092AP2:	
Pinout	- Included more information regarding power-up/reset pin types - Updated pinout descriptions and external pull-up/pull-down requirements
Control Port	- Added Factory default information for uninitialized OTP memory - Added clarification for I²C clock stretching
MediaLB Port	- Added Factory default information for uninitialized OTP memory
External Power Management	- Clarified PWROFF functionality based on the <code>PMIConfig()</code> property
JTAG Port	- Added Boot Mode as a prerequisite for accessing the JTAG Port - Updated the Identification Register (IDR) for hardware Revision B
Electrical Physical (ePHY) Interface	- Added ESD protection devices to ePHY Front End Circuit Diagram
Electrical Characteristics	- Added distinct V_{IL} for the XTI pin - Corrected minimum RST input hysteresis value - Excluded VDDU from the 90% threshold prior to the POR - Corrected <i>minimum</i> f_{mclk} values to accurately reflect 256×Fs (where Fs = 47.9 kHz)
Package Outline	- Added D2, E2, and L package dimensions
Application Information	- Added frequency to crystal oscillator specifications table - Added layout recommendations including typical schematic and example layout plots
DS81092AP1:	
	First Release - AP Data Sheet for OS81092

APPENDIX C: FEATURE MATRIX

Some hardware ports and features on the OS81092 INIC are only available with specific Product Revisions. A list of available device features and the applicable revision codes is provided in [Table C-1](#).

TABLE C-1: OS81092 FEATURE MATRIX

Device Feature	Product Revision Codes *	
	OS81092AM-C1C and earlier	OS81092AM-C1D and later
Control Port operating in I ² C slave mode	x	x
Control Port operating in I ² C master mode		x
MediaLB Port	x	x
Streaming Port	x	x
External Power Management	x	x
Clock Manager	x	x
GPIO Port		x
JTAG Port	x	x
ePHY Interface	x	x
All other features	x	x

* Product Revision Codes are specified when ordering. Contact Microchip for more information.

APPENDIX D: LIST OF TABLES

TABLE 2-1:	OS81092 Pin Allocation Table	7
TABLE 4-1:	Hardware Port Data Transport.....	13
TABLE 8-1:	Available MediaLB Bandwidth	29
TABLE 9-1:	Streaming Port vs. MediaLB Port Availability	35
TABLE 9-2:	Streaming Data Formats.....	36
TABLE 10-1:	External Power Management States.....	39
TABLE 12-1:	GPIO Pin Availability.....	42
TABLE 15-1:	Absolute Maximum Ratings Electrical Characteristics	50
TABLE 15-2:	Guaranteed Operating Conditions Electrical Characteristics	51
TABLE 15-3:	DC Electrical Characteristics (other than MediaLB).....	52
TABLE 15-4:	AC Electrical Characteristics (other than MediaLB).....	53
TABLE 15-5:	Reset and Power Management Electrical Characteristics	54
TABLE 15-6:	ePhy Network Electrical Characteristics	56
TABLE 15-7:	ETXP/ETXN Eye Mask Boundaries	56
TABLE 15-8:	ERXP/ERXN Eye Mask Boundaries	57
TABLE 15-9:	MediaLB Port Electrical Characteristics (3-Pin, 256×Fs or 512×Fs).....	58
TABLE 15-10:	MediaLB Port Electrical Characteristics (3-Pin, 1024×Fs).....	59
TABLE 15-11:	MediaLB Port Electrical Characteristics (5-Pin, 256×Fs or 512×Fs).....	60
TABLE 15-12:	Streaming Port Electrical Characteristics.....	61
TABLE 15-13:	Control Port Electrical Characteristics.....	62
TABLE 15-14:	JTAG Timing Electrical Characteristics.....	64
TABLE 16-1:	Crystal Oscillator Specifications.....	70
TABLE 16-2:	ePHY Front End Circuitry - Component Selection	71
TABLE 16-3:	ePHY Front End Circuitry - Transformer Specifications.....	71
TABLE B-1:	Data Sheet Revision Summary.....	82
TABLE C-1:	OS81092 Feature Matrix.....	84

APPENDIX E: LIST OF FIGURES

FIGURE 1-1:	48-Pin QFN	5
FIGURE 2-1:	48-Pin QFN Diagram for OS81092	6
FIGURE 3-1:	Pin-equivalent for Analog I/O pin - A_{IO}	9
FIGURE 3-2:	Pin-equivalent for Digital Input pin - D_{IN}	9
FIGURE 3-3:	Pin-equivalent for Digital Output pin - D_{OUT}	9
FIGURE 3-4:	Pin-equivalent for Open-Drain Digital Output pin - D_{OUTD}	9
FIGURE 3-5:	Pin-equivalent for Digital Output pin with high-Z control - D_{OUTZ}	10
FIGURE 3-6:	Pin-equivalent for Digital I/O pin - $D_{I/O}$	10
FIGURE 3-7:	Pin-equivalent for Digital Input/Open-Drain Output pin - $D_{I/OD}$	10
FIGURE 4-1:	INIC Hardware/System Overview	11
FIGURE 4-2:	INIC-IOC Application Example	15
FIGURE 4-3:	Division of Services between INIC and the EHC	16
FIGURE 5-1:	INIC Software Stack	17
FIGURE 6-1:	Generic Port Message Format	19
FIGURE 6-2:	FIFO Status and Command Message Format	20
FIGURE 6-3:	FIFO Read and Write Message Format	20
FIGURE 7-1:	Control Port Pin Connections (Slave Mode)	22
FIGURE 7-2:	Control Port Write Sequence (Slave Mode)	23
FIGURE 7-3:	Control Port Read Sequence (Slave Mode)	24
FIGURE 7-4:	Control Port Pin Connections (Master Mode)	25
FIGURE 7-5:	Control Port Write Sequence (Master Mode)	26
FIGURE 7-6:	Control Port Read Sequence (Master Mode)	26
FIGURE 8-1:	MediaLB 3-pin Mode Connection Diagram - Example	28
FIGURE 8-2:	MediaLB Data Structure	29
FIGURE 8-3:	MediaLB 3-pin Mode to Parallel Interface	30
FIGURE 8-4:	MediaLB Interface (at $256 \times Fs$)	31
FIGURE 8-5:	MediaLB 5-pin Mode Two-Node Connection Diagram	32
FIGURE 8-6:	MediaLB 5-pin Mode Four-Node Connection Diagram	33
FIGURE 8-7:	MediaLB Debug Header	34
FIGURE 9-1:	Full Streaming Mode vs. MediaLB 5-pin Mode	35
FIGURE 9-2:	Partial Streaming Mode	36
FIGURE 9-3:	Left-Justified Streaming Data Format	37
FIGURE 9-4:	Right-Justified Streaming Data Format	37
FIGURE 9-5:	Delayed-Bit Streaming Data Format	38
FIGURE 9-6:	Sequential Streaming Data Format	38
FIGURE 10-1:	Typical Power Management Diagram	40
FIGURE 10-2:	Power Management Startup	40
FIGURE 11-1:	Crystal Oscillator Input	41
FIGURE 14-1:	ePHY Interface - Block Diagram	47
FIGURE 14-2:	ePHY Transmitter/Receiver Front End Circuitry	48
FIGURE 15-1:	Power-Up and Reset Timing	55
FIGURE 15-2:	Network Startup and Shutdown Timing	55
FIGURE 15-3:	ETXP/ETXN Eye Mask Pattern	56
FIGURE 15-4:	ERXP/ERXN Eye Mask Pattern	57
FIGURE 15-5:	MediaLB 3-pin Timing	59
FIGURE 15-6:	MediaLB 5-pin Timing	60
FIGURE 15-7:	Streaming Port Timing	61
FIGURE 15-8:	Control Port Timing	63
FIGURE 15-9:	JTAG Timing	64
FIGURE 16-1:	Typical Power Supply Connection Diagram	65
FIGURE 16-2:	Connection Diagram for Configuration via Control Port	68
FIGURE 16-3:	Configuration/Debug Header	69
FIGURE 16-4:	Example Layout - Reference Schematic	72

FIGURE 16-5:	Example Layout - Top and Bottom Layers Superimposed.....	73
FIGURE 16-6:	Example Layout - Top Layer	74
FIGURE 16-7:	Example Layout - Bottom Layer.....	75
FIGURE 16-8:	Example Layout - Ground Layer	76
FIGURE 16-9:	Example Layout - INIC Peripheral Power Plane	77
FIGURE 16-10:	Example Layout - INIC Core Power and Continuous Power Planes.....	78
FIGURE 16-11:	Example Layout - Drill Holes.....	80

Microchip Information

Trademarks

The “Microchip” name and logo, the “M” logo, and other names, logos, and brands are registered and unregistered trademarks of Microchip Technology Incorporated or its affiliates and/or subsidiaries in the United States and/or other countries (“Microchip Trademarks”). Information regarding Microchip Trademarks can be found at <https://www.microchip.com/en-us/about/legalinformation/microchip-trademarks>.

ISBN:979-8-3371-0695-3

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP "AS IS". MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer's risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip product is strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is “unbreakable”. Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.